



Practical Guide

Azure DevOps

Complete CI/CD Pipeline

By: Mukesh Kumar

Disclaimer & Copyright

Copyright © 2019 by [Mukesh Kumar](#)

All rights reserved. Share this eBook as it is, don't reproduce, republish, change or copy. This is a free eBook and you are free to give it away (in unmodified form) to whomever you wish. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission from the author.

The information provided within this eBook is for general informational purposes only. While we try to keep the information up-to-date and correct, there are no representations or warranties, express or implied, about the completeness, accuracy, reliability, suitability or availability with respect to the information, products, services, or related graphics contained in this eBook for any purpose. Any use of this information is at your own risk

The methods described in this eBook are the author's personal thoughts. They are not intended to be a definitive set of instructions for this project. You may discover there are other methods and materials to accomplish the same end result.

Mukesh Kumar

Author

(By: Mukesh Kumar)

About The Author



Mukesh Kumar is a **Software Developer** and **Microsoft MVP** who has a Master's degree in Computer Science. He is also **C# Corner MVP, Blogger, Writer** and has more than seven years of extensive experience designing and developing enterprise-scale applications on Microsoft Technologies like C#, Asp.Net, Web API, Asp.NET Core, Microsoft Azure, Angular 4+, JavaScript, Node.JS, Python etc.

He is a passionate author on www.mukeshkumar.net and believes in the motto "**Think for the new.**"

You can also follow him on [Facebook](#), [Twitter](#), [LinkedIn](#) and [Google+](#).

Table of Contents

1.	About DevOps	5
1.1	Definition	5
1.2	Why DevOps	6
1.3	DevOps Lifecycle	6
1.4	Prerequisites	8
2.	CI/CD Pipeline	10
3.	Why Azure DevOps	12
4.	DevOps Project Setup	13
4.1	Create Asp.Net Core Project	13
4.2	xUnit Test Project	22
4.3	Add Project to GitHub	27
5.	Create Organization and Project	30
6.	Continuous Integration	35
7.	Create Azure App Services	49
8.	Continuous Delivery	59
8.1	Create Dev Stage	59
8.2	Create QA Stage	66
8.3	Create Prod Stage	79
9.	Add Slot	90
10.	Run and Test Azure DevOps Pipeline	96

1. About DevOps

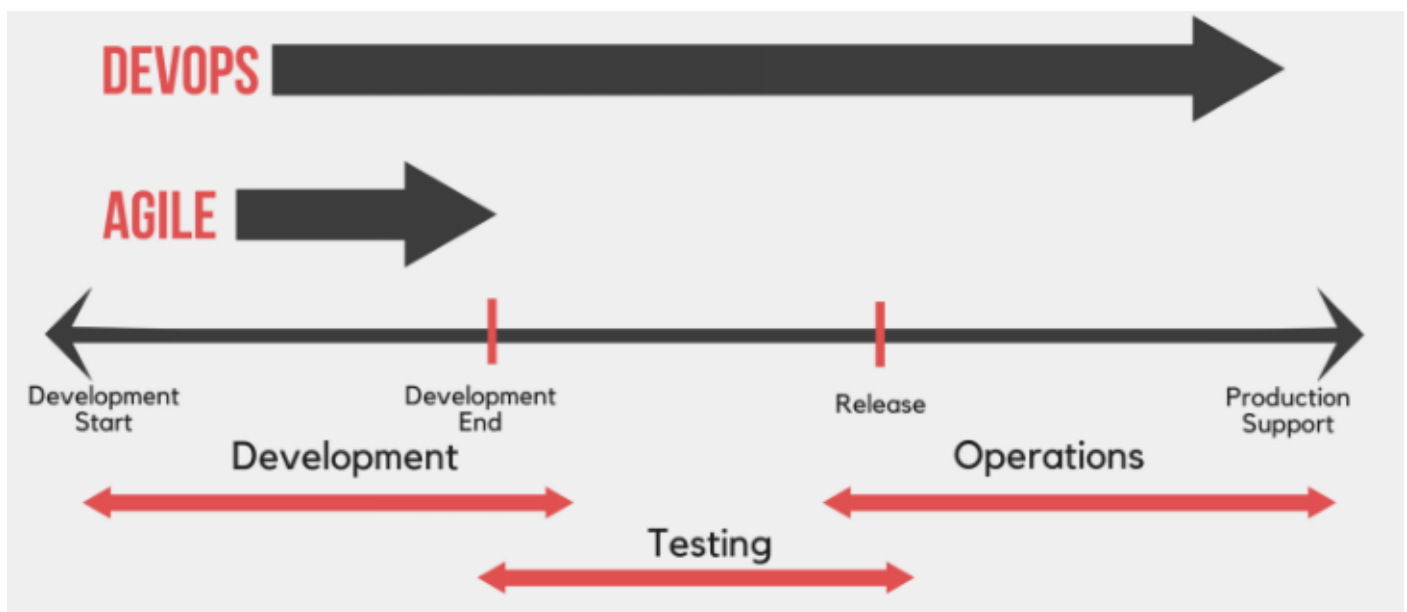
1.1 Definition

DevOps is a combination of Development and Operations. It means Dev + Ops = DevOps. It is a culture which automates the system and improves and accelerates delivery to the client in a repeated manner. It's basically a collaboration between the Development team and the Operations team for serving a better quality application. It is a culture for continuous integration and continuous delivery where we make the automated build system as well as automated deployment system. In other words, DevOps is practice collaboration between Development and Operations from the planning of a project to deployment on production. It involves the complete SDLC life cycle as well as monitoring.

Understanding DevOps, you should consider the following points as well.

- ✓ DevOps means only combining the two teams of Development and Operations.
- ✓ It is not a separate team.
- ✓ It is not a product or tool.
- ✓ DevOps people do not hire from outside, they are internal team members who are working either in the development phase or in the operations phase.

It is a group of people, processes and tools. It basically brings the two or more different teams like development and operations together with a well-defined process, using some great tools for automatic software delivery to the client. It is a set of practices which are used by the DevOps teams to speed up the quality delivery. There are different kinds of tools or set of tools which are used in **Continuous Integration (CI)** and **Continuous Delivery (CD)** where it performs restoring the code, building the processes, executing the test cases, and deployment on the stage environment, etc.



1.2 Why DevOps

To understand why DevOps is required, let's first understand, what happens without DevOps.

As an IT consulting firm, while initiating a new project, we have two different kinds of teams. First is the Development team, which is involved completely in developing and testing, including writing code and unit test cases. The other team is the Operations team which is involved in operating and monitoring the product.

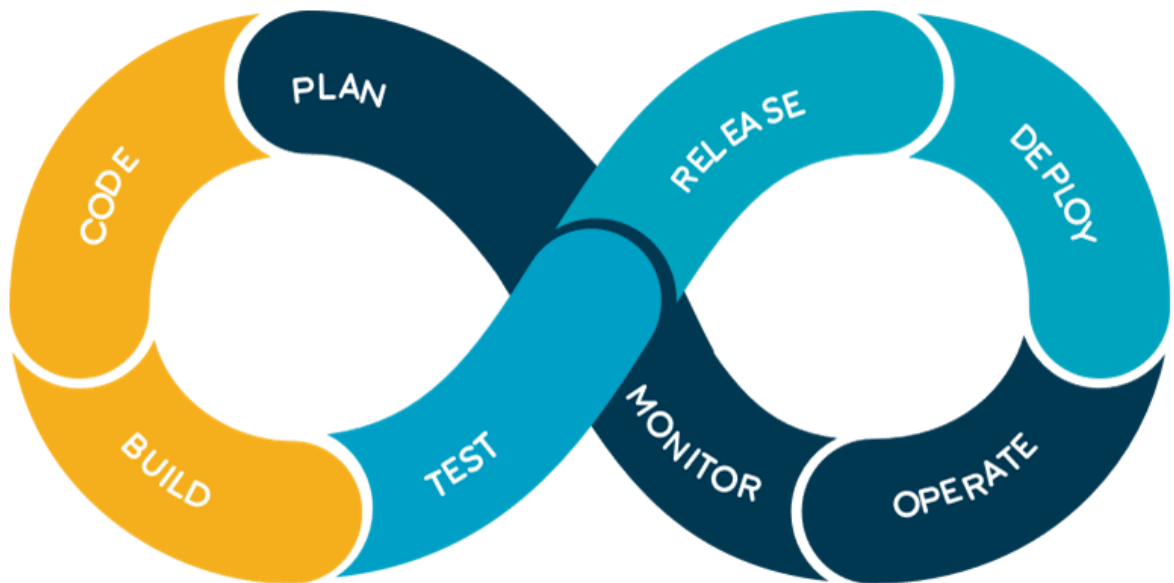
- ✓ Without DevOps, both teams (**Development and Operations**) work completely in an isolated manner.
- ✓ If DevOps is not there then the team spends most of the time in building the code and deploying on multiple environments.
- ✓ Each team waits for others to be done. This means, if development is going on then the testing team waits for the deployment of the code (Artifact) on the QA environment and in the same manner the operations team waits for the deployment on the Production environment. So, due to this, a large amount of time gets wasted.
- ✓ As a human being, we make mistakes. Every time, building the code and deploying on specific environments in a manual fashion can increase the issues and resolving it will take a lot of time. Rather than doing it manually, we can make it automated using DevOps.

So, the above points show that we face lots of human issues and system issues if we are not following DevOps. Now, let's see how we can make it more systematic using DevOps and how DevOps helps us to achieve the same tasks in less time without any errors.

- ✓ DevOps increases the higher success rate of new releases without any error.
- ✓ It helps us to simplify the whole development and deployment process.
- ✓ Automates the manual process like build process, release process, etc.
- ✓ Automates executing the test cases.
- ✓ Configures the continuous delivery and continuous deployment in the release cycle.
- ✓ Live monitoring.
- ✓ It also helps in team collaboration.
- ✓ Reduces the failures and rollbacks
- ✓ Provides continuous improvement

1.3 DevOps Lifecycle

DevOps is a culture where Development and Operations teams get involved. The development phase has its own lifecycle and the Operations phase has its own as well. If we combine both lifecycles, we get the lifecycle of DevOps. If an organization is not following or considering some of the points from the DevOps lifecycle then we can say, they are not following DevOps culture. From planning to deployment, we have several stages which are very important and we cannot skip any of them. So, let's understand the DevOps lifecycle.

**PLAN:**

It is the first stage of any new project. Here, we plan for a new project from requirement gathering from the customer and planning to deliver the final project to the customer.

- ✓ What the requirements are.
- ✓ What types of product we have to create.
- ✓ What the timelines are for different sprints and the final product.
- ✓ What technologies we will use.
- ✓ What tools we will use.
- ✓ How many team members will be available for this project.
- ✓ What process we are going to follow, like agile.

CODE:

In this stage, we do the coding for creating the product with actual functionality as discussed with the customer. We use different types of methodologies for achieving the goal, like Agile methodology. Here we group the tasks in the sprint and their estimation as well. Sprints are basically for 2-3 weeks. Unit Test cases are also to be a part of the coding.

BUILD:

In this stage, we build the code. Code building happens two times; first when a developer is writing the code, then he/she has to build the code every time in their own local system to see the functionality. The second time, when a team member checks in the code in source control repository, then it automates the build for the code and makes the artifact for deployment.

TEST:

Testing is the heart of any development process. We write the Unit Test cases along with code. In DevOps, test cases auto execute and validate the build process.

RELEASE:

In this stage, it collects the build artifact which can deploy further.

DEPLOY:

Here, we start the deployment on the respective stages which are configured in the Release Pipeline. Actually after testing and validating the build artifact, it auto starts the deployment on the respective environment using a continuous delivery process. But before deploying it to the production environment, it asks for approval which can be done manually.

We can also automate the whole deployment process using continuous deployment. This is basically used when we have small changes which can be deployed on production as well without any approvals.

OPERATE & MONITOR: After successful deployment on the production environment, we have to operate the whole system and monitor the application. This monitoring is not only the performance but also the functionality.

1.4 Prerequisites

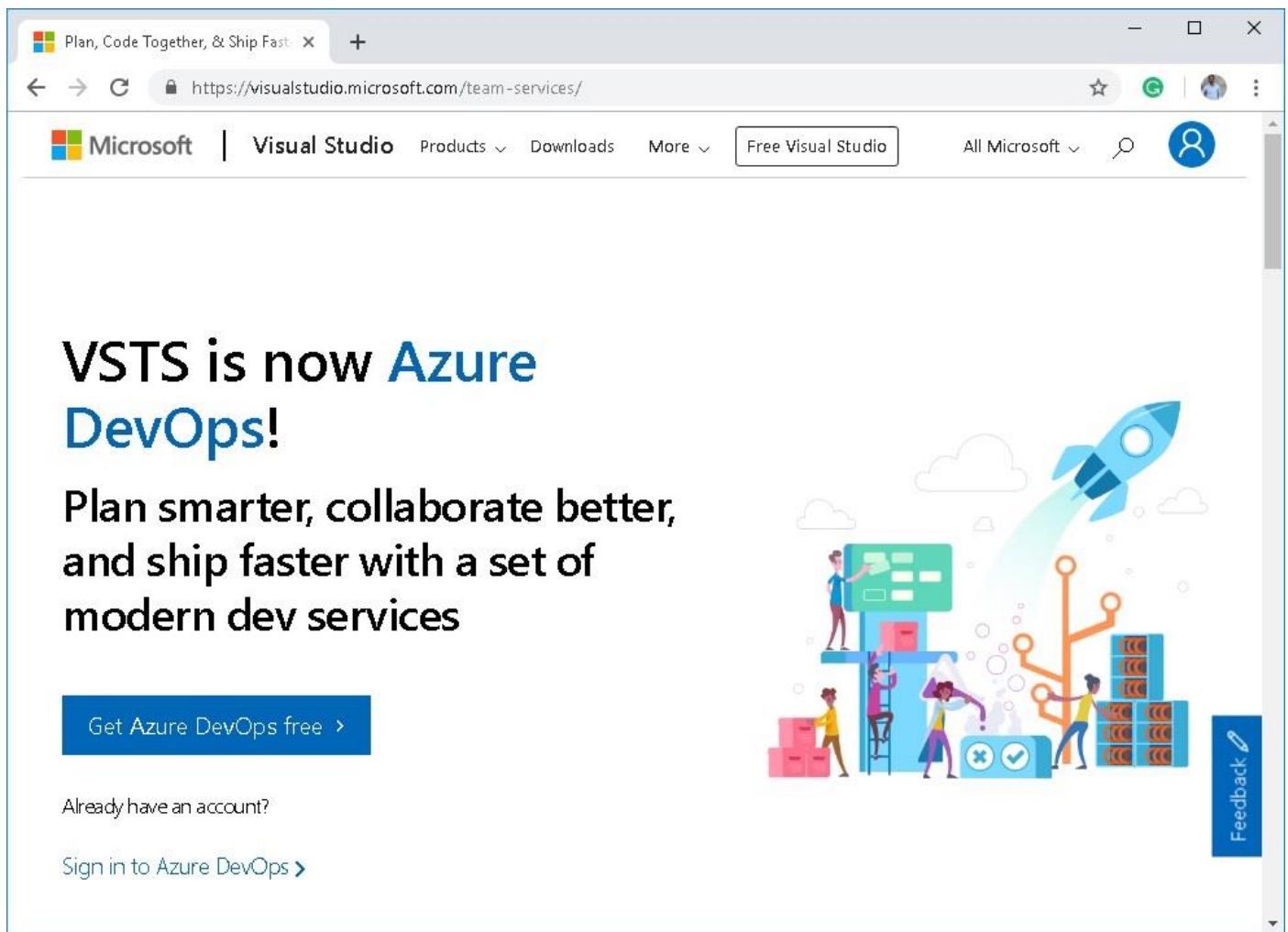
In order to do this practical demonstration, we will require a few tool and some accounts.

Visual Studio 2017 or Higher Version: It is a world class IDE which supports more than 40 programming languages. We will create the sample application along with a test project using Visual Studio 2017. So, before starting the demonstration, we will require Visual Studio 2017. If you would like to download Visual Studio 2017, we can download it from [HERE](#).

Alternatively,

1. We can also use the Visual Studio Code, which is totally open source and can be download from [HERE](#).
2. We can also create the Virtual Machine on the cloud (Cloud Account is required) and install Visual Studio 2017.

Azure DevOps: We need an Azure DevOps account. If we have an account with Azure DevOps that's fine; otherwise we can sign up from [HERE](#). Earlier it was known as VSTS (Visual Studio Team Service). We can also get the free trial Azure DevOps account. Just click on the button 'Get Azure DevOps Free' as follows.



Microsoft Azure Account: We also require Microsoft Azure account to access Cloud Services like VM, App Service, and Database. If we don't have any subscription for Microsoft Azure then we can go with a Free Trial. <https://portal.azure.com> is the portal for accessing all Microsoft Azure Services.

2. CI/CD Pipeline

Now a days, the delivery process in software development is rapid and fast. With the help of several tools, we try to customize our delivery process. If delivery process will be smooth then we can deliver the high quality of product to customer within timeline. Actually, customer wants to get the small change or small functionality to be added in minimum time. To enable to faster delivery process, we take the help of some of the mechanism like agile and various tools and people and it is called DevOps.

Let see what the old process in development are and what happens if some issues are there.

1. Get the requirements from Business.
2. Make a plan for converting the requirements into the actual functionality.
3. Developers do the code and check in it to a repository like TFS, GitHub etc.
4. After all code checks in to the repository, a developer executes the build process.
5. Run the test cases against the code.
6. If everything is fine, code gets deployed on the DEV server and then on QA.
7. If QA confirms the OK then deployment goes started on PROD.

Above is the general development workflows which are used mostly in small organizations. But is this process correct? Let understand the problem with the above development and delivery process.

1. Every time a developer check in the code into repository, but he/she doesn't know about Build. Is build created successfully or not?
2. Developers should wait for other developers to do the check in the code and building the code before deploying to DEV/QA server.
3. If a developer has completed his/her work and has checked in the code, they have to wait for others developers to check in the code as well.
4. If something wrong with anyone code then builds cycle will stop and wait again till the issue is resolved.
5. As a human being, we do a lot of mistakes. We can also do a mistake while doing the manual deployment.
6. Chances to get more issues from development to deployment.

As we can see with above points, there are several issues while using the old deployment process. Which also need more time as doing manual deployment, more resource for completing manual deployment and obviously more money. We can resolve all the above issues if we implement the **Azure DevOps**. In **Azure DevOps**, we have **Continuous Integration (CI)** and **Continuous Delivery (CD)**. These help us to make the whole process as automated. So, let understand what these are.

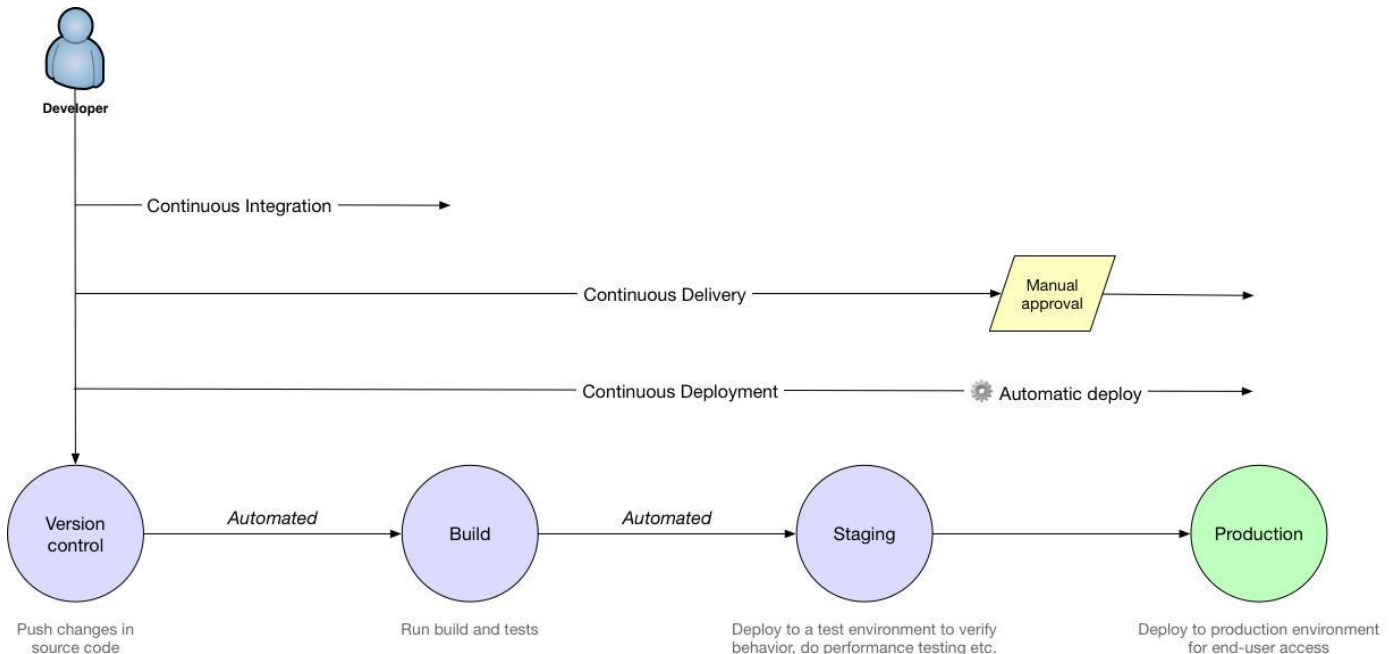
Continuous Integration (CI): It is an automated build process which starts automatically when a developer checks in the code into the respective branch. Once the code check in process is done, the Continuous Integration process starts. It fetches the latest code from the respective branch and restore the required packages and start building automatically. After build process, it auto starts executing the Unit Test Cases and at the end, build artifact is ready. This build artifact further will use for deployment in Release process.

Continuous Delivery (CD): It is the next process after Continuous Integration. Once build artifact is ready for deployment then release process gets started. As per the Azure DevOps Release Pipeline configuration, it starts deploying the build artifact on the different stage server (Environments) automatically. But in the Continuous

(By: Mukesh Kumar)

Delivery, the deployment on the production goes manual. Manual does not mean here that we will deploy the build artifact manually, but here we have to approve before deploying on the PROD.

Continuous Deployment: It is the combination of **CI + CD** and **deployment on the production without any approval**. It means, in the Continuous Deployment, everything goes automatically.



As per the above image, we can see that once a developer checks in the code into the Version Control, Azure DevOps Pipeline starts. From getting the code from Version Control, restoring it, restoring packages from NuGet, Maven etc., building the code, executing the unit tests cases are part of the **Continuous Integration (CI)**.

Including Continuous Integration, if build artifact gets auto-deployed on any staging server like DEV or QA and deploys on the Production after manual approval is called **Continuous Delivery (CD)**. If manual approval is converted into automatic deployment then it's called **Continuous Deployment**. So, basically Continuous Deployment is the same as Continuous Delivery but without any manual approval. All goes in auto approval mode.

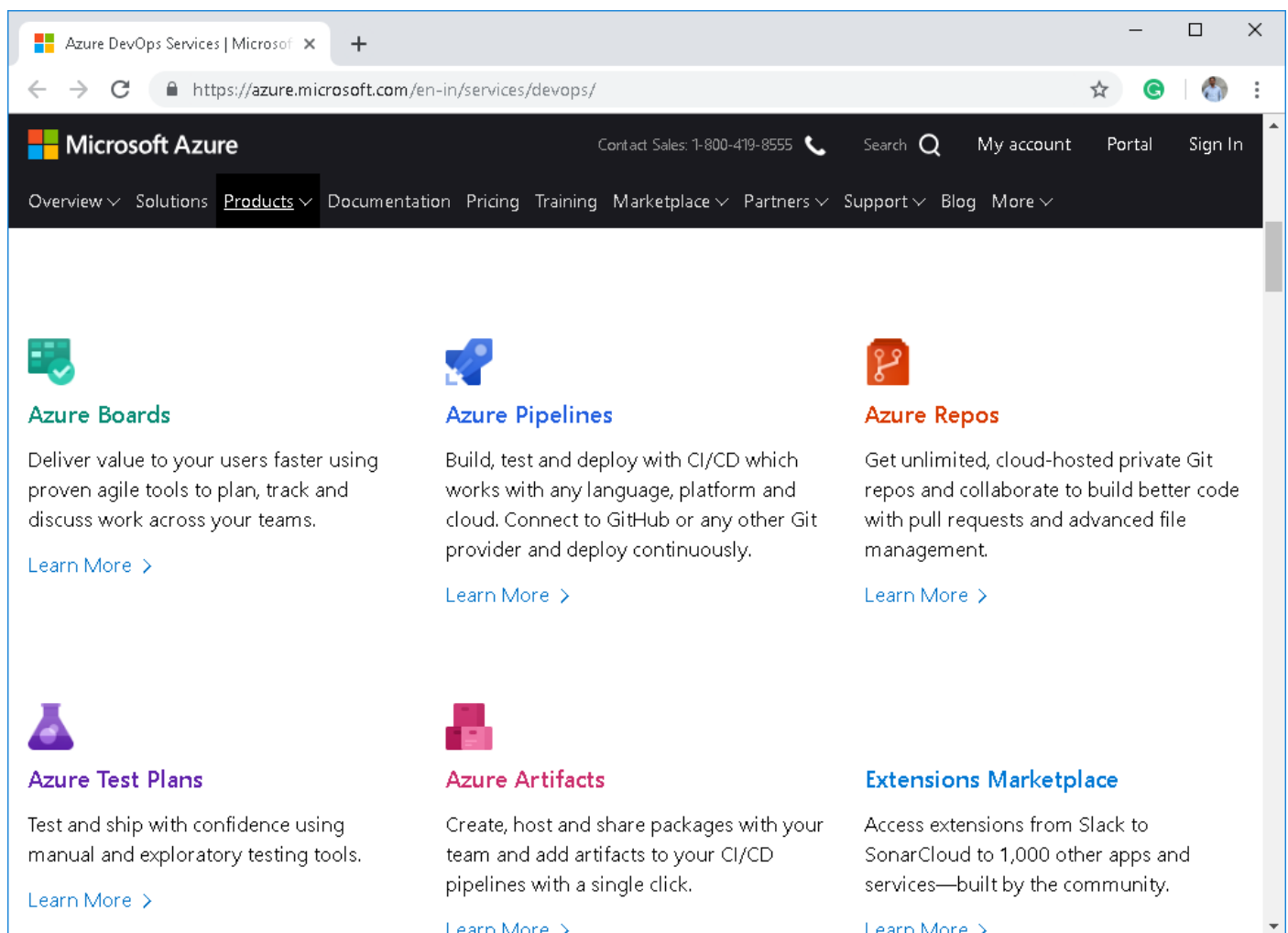
3. Why Azure DevOps

DevOps is a group of people, processes and tools which enable and automate the continuous delivery. Azure DevOps formally known as Visual Studio Team Service (VSTS) provides the repository management, project management, Build/Release Pipeline etc. Azure DevOps is free for a small project which has up to five users. But we can also use the paid version of it.

Azure DevOps provides unlimited cloud-hosted Git repos for a small or big project. Here developer can pull the code from Repos or push the code into Repos. It is basically a complete file management system.

While building the source code, it requires lots of third-party packages. Using Azure Artifacts, it enables the **NPM**, **MAVEN** or **NuGet** packages will available for private or public source code.

The most important feature of Azure DevOps is Azure pipeline. It enables the automated build and release pipeline. Azure Pipeline helps us to achieve Continuous Integration and Continuous Delivery for the project. Know more update about Azure DevOps [HERE](#).



Microsoft Azure DevOps is most popular and widely use throughout the world because **it is free for Open Source Project and Small Teams project**. We can use lots of Azure DevOps features with free version like Azure Pipeline, Azure Boards, Azure Repos, and Azure Artifacts etc. It means, we don't need to go for paid version of Azure DevOps, if and only if we want to use those features which are in free versions.

(By: Mukesh Kumar)

Pricing for Azure DevOps

Start with Only Azure Pipelines or move to Azure DevOps Services, Azure Boards, On-premises repos and more

Open source projects

Free

Unlimited users and build time

- **Azure Pipelines:** 10 parallel jobs with unlimited minutes for CI/CD
- **Azure Boards:** Work item tracking and Kanban boards
- **Azure Repos:** Unlimited public Git repos

Small Teams

Free

Start free with up to 5 users

- **Azure Pipelines:** 1 hosted job with 1,800 minutes per month for CI/CD and 1 self-hosted job
- **Azure Boards:** Work item tracking and Kanban boards
- **Azure Repos:** Unlimited private Git repos
- **Azure Artifacts:** Package management (5 users free)
- Load testing (20,000 VUMs/month)
- Unlimited stakeholders

Teams of any size

₹1,982.89/mo

10 Users ▼

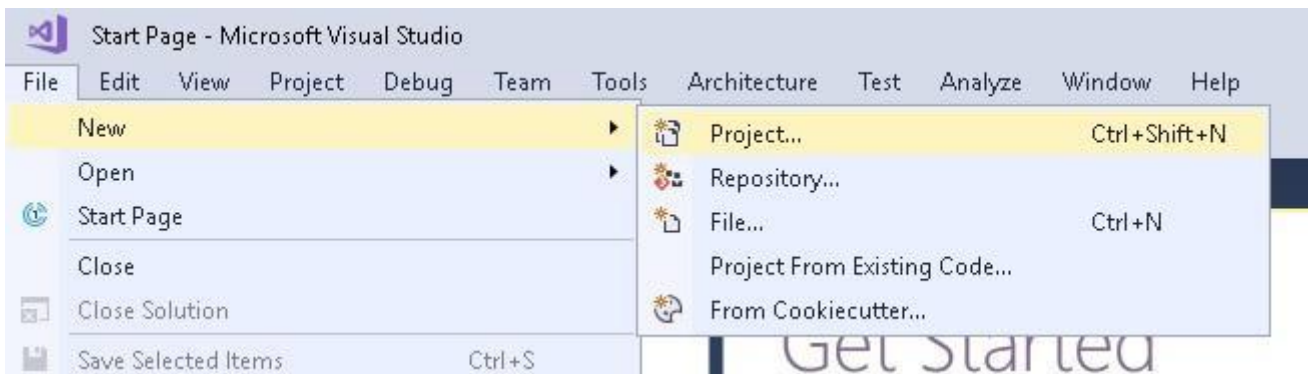
- **Azure Pipelines:** 1 hosted job with 1,800 minutes per month for CI/CD and 1 self-hosted job
- **Azure Boards:** Work item tracking and Kanban boards
- **Azure Repos:** Unlimited private Git repos
- **Azure Artifacts:** Package management (5 users free)
- Load testing (20,000 VUMs/month)
- Unlimited stakeholders
- Visual Studio subscribers included free

4. DevOps Project Setup

In this session, we will create a project in Visual Studio 2017 or in higher version which will be used for this demonstration. Apart from the main project, it will also include a testing project, where all required Unit Test Cases will write. For this demonstration, we are using Visual Studio 2017 but anyone can use higher version of Visual Studio for creating the Asp.NET Core. So, let start creating the Asp.Net Core project along with xUnit testing project for DevOps Demo.

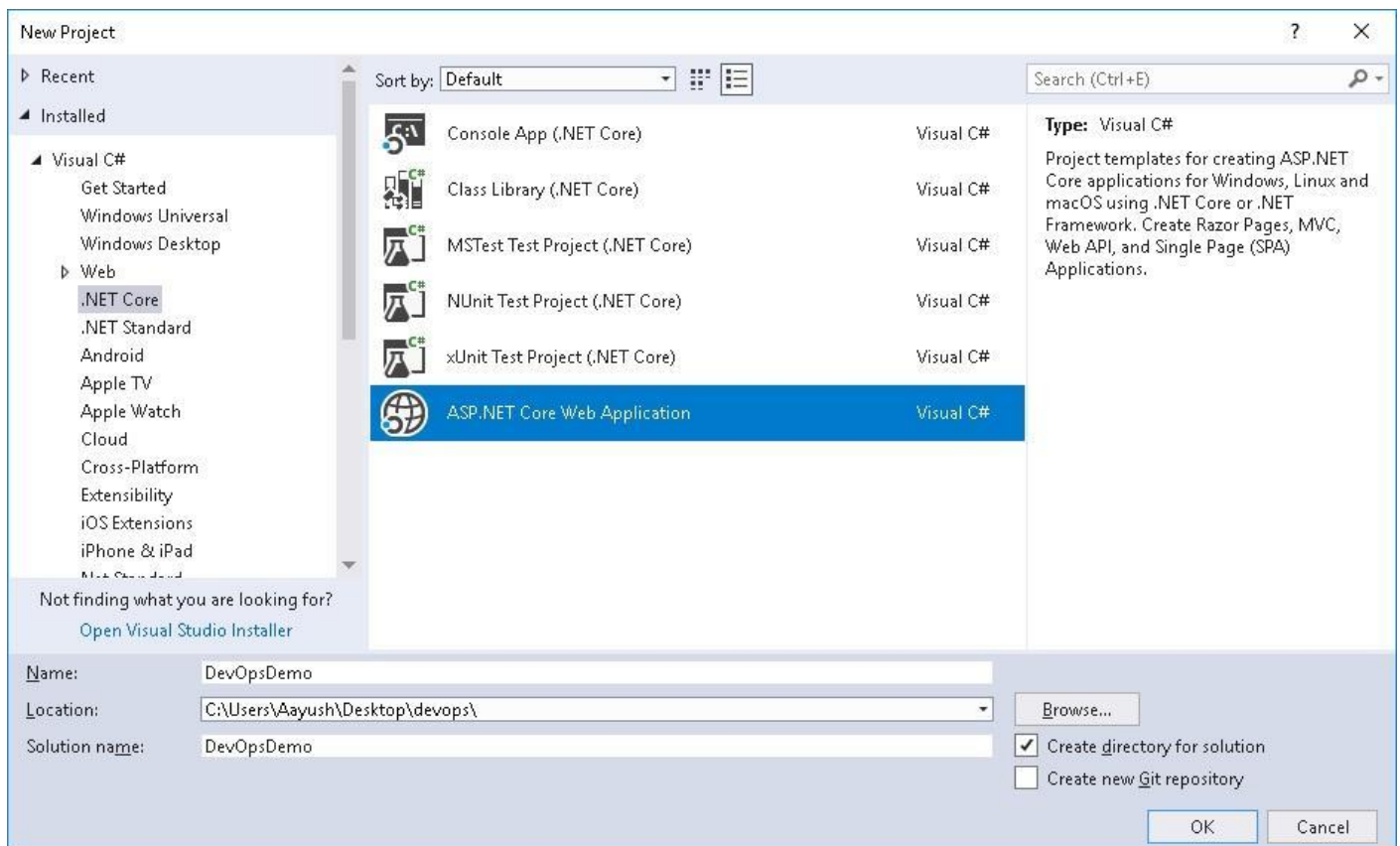
4.1 Create Asp.Net Core Project

Let create an **Asp.Net Core Project** in Visual Studio. Open **Visual Studio 2017 or higher version** and go to the **File** menu and choose **New** and then **Project**.

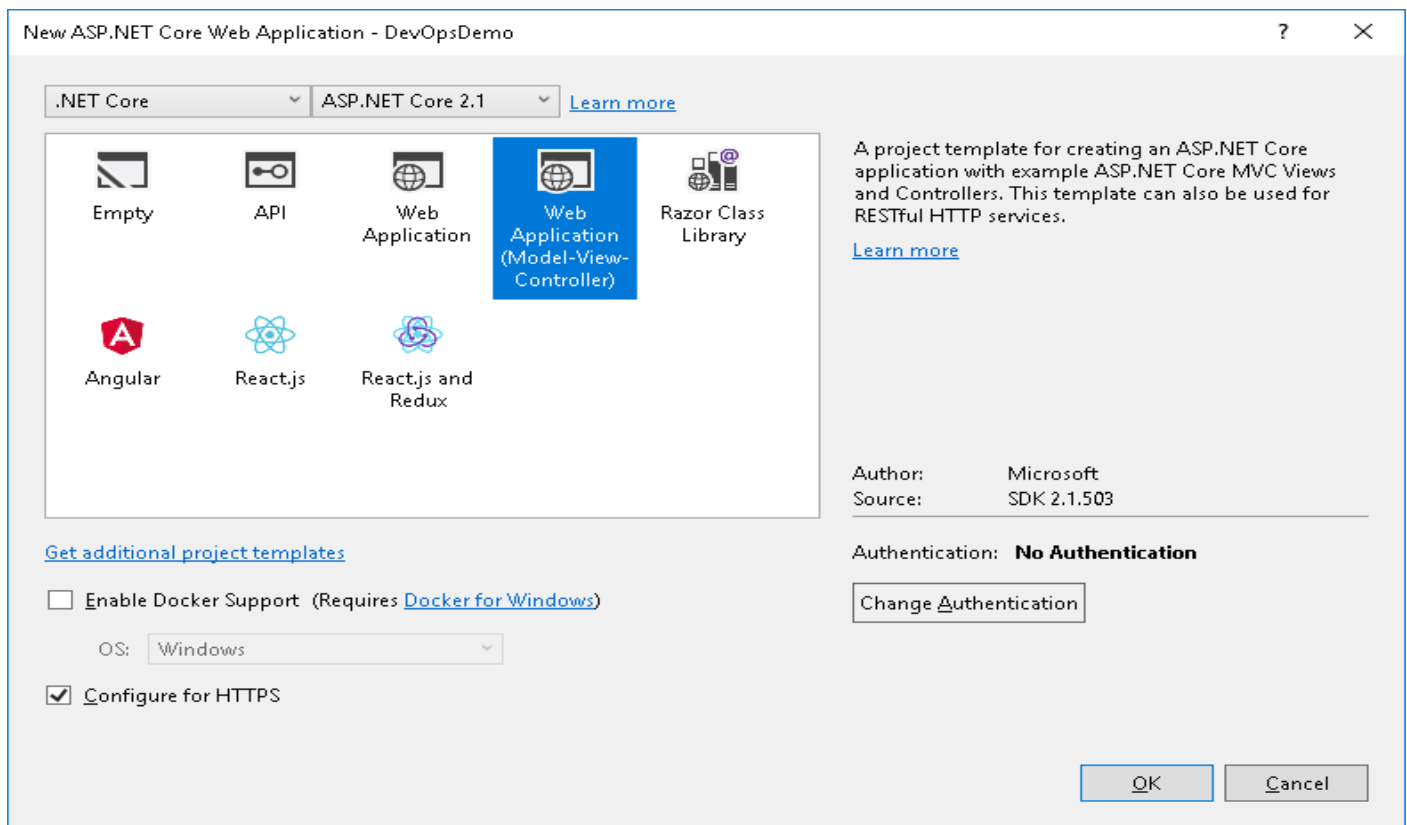


A “**New Project**” dialog window will open as follows. As we are going to create **.Net Core application**. So, move to the first panel and from the **Installed**, choose **Visual C# > Web > .Net Core**. Here we will find different kinds of **.Net Core application** template. We will select **ASP.NET Core Web Application**.

After selection the **.Net Core application** template, provide the **suitable name for the project** as well as a solution name like **DevOpsDemo**. This project is being created for DevOps demonstration, so let keep the simple name as DevOpsDemo. Don't forget the provide the project **location** and **click to OK**.

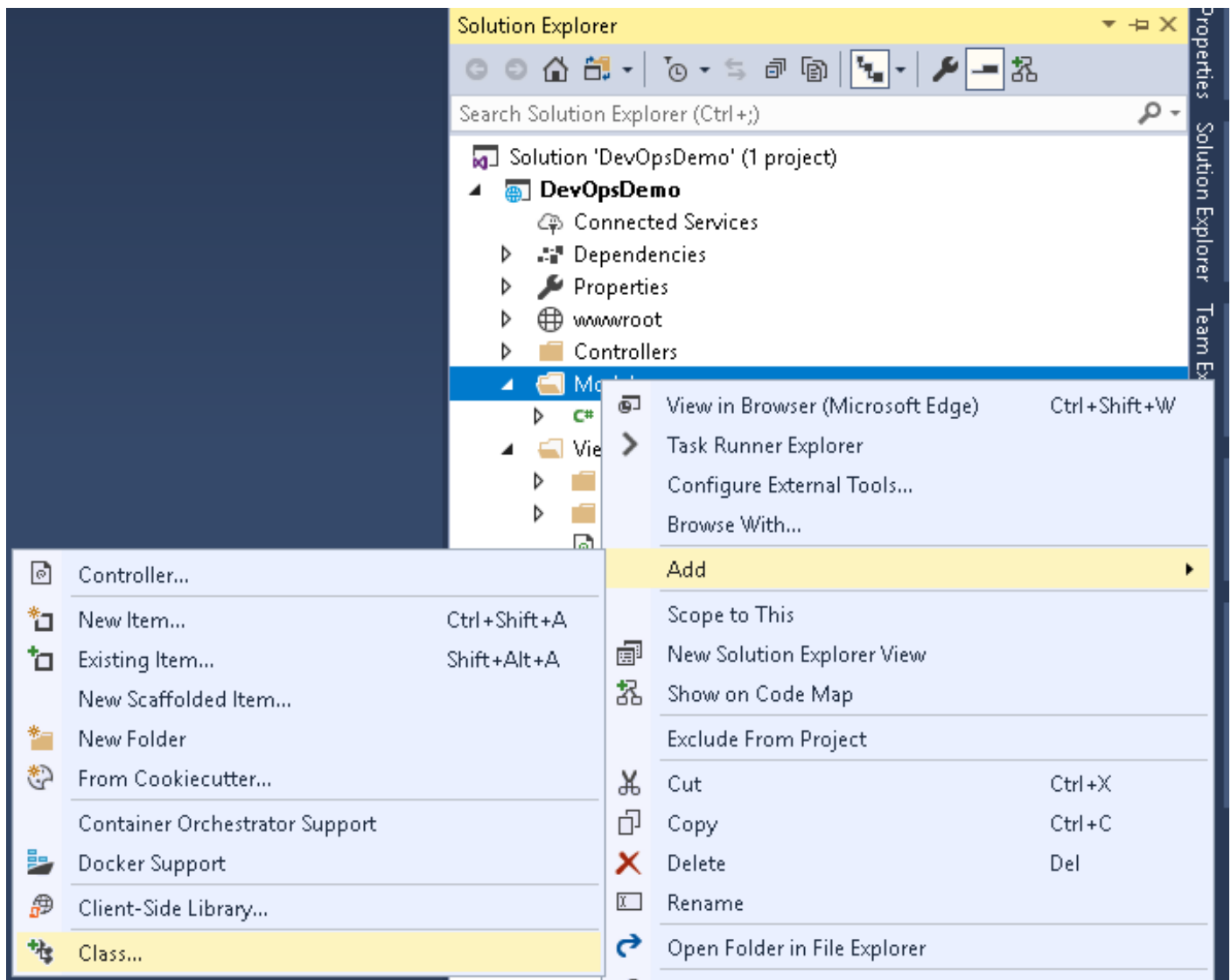


Next dialog window will ask to choose the **Project template**, here, we are working on **ASP.NET Core 2.1**. We have different kinds of project templates are available like **API**, **Web Application**, **Model View Controller** etc. Here we will select **Web Application (Model-View-Controller)** which enable the functionality of MVC. Apart from this, we require two more things, first check on checkbox for configuring the HTTPS and change authentication and choose **No Authentication**. Now click to **OK**.

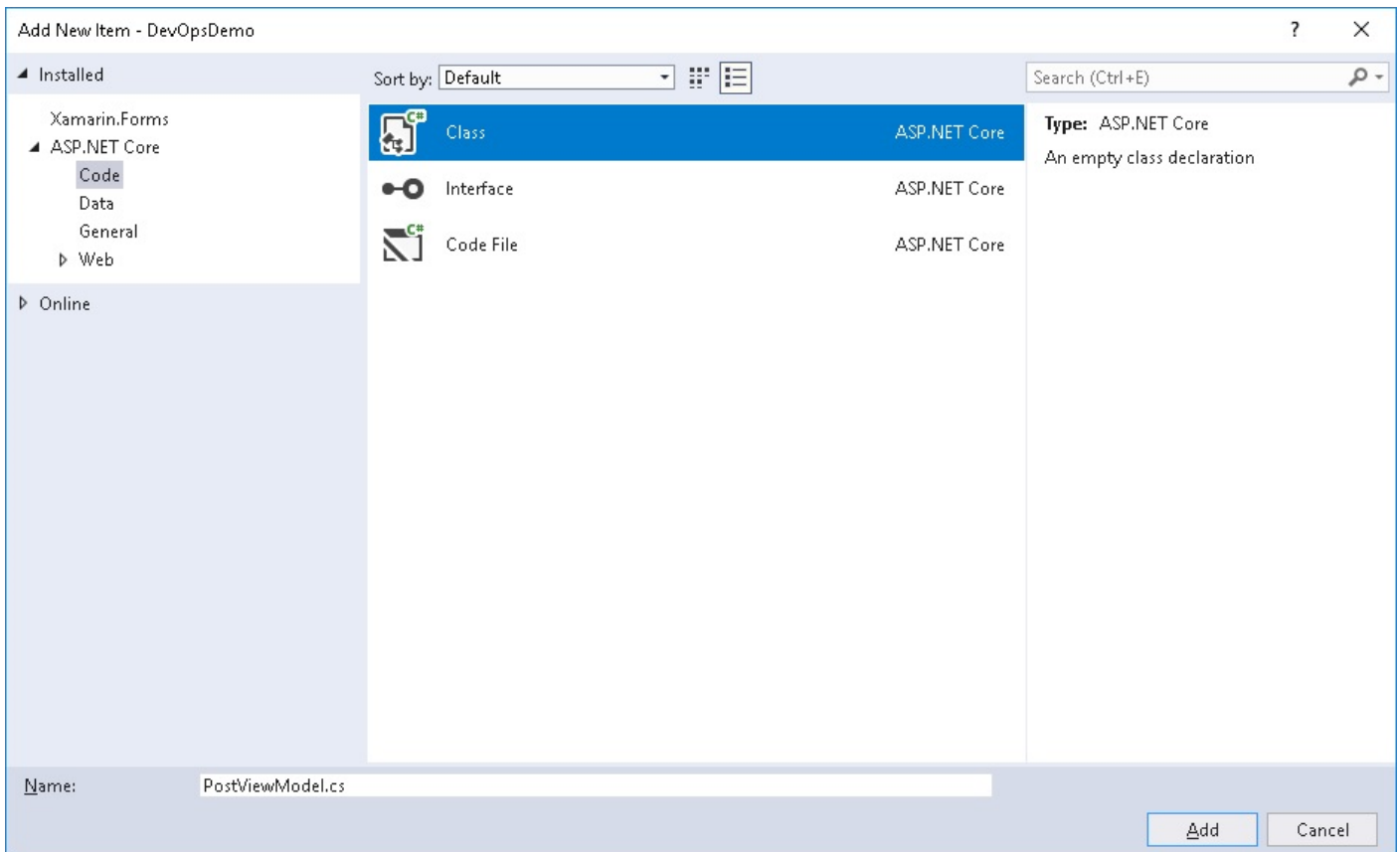


It will take a few moments to create and configure the project and final project will be ready. Once project will be ready, we can see the basic MVC structure application in ASP.NET Core Web Application.

Let add some Model-View-Controller functionality. So, let create a **Model** class for 'Post'. **Right click** on the Model folder from the project and choose **Add** and then choose **Class**.



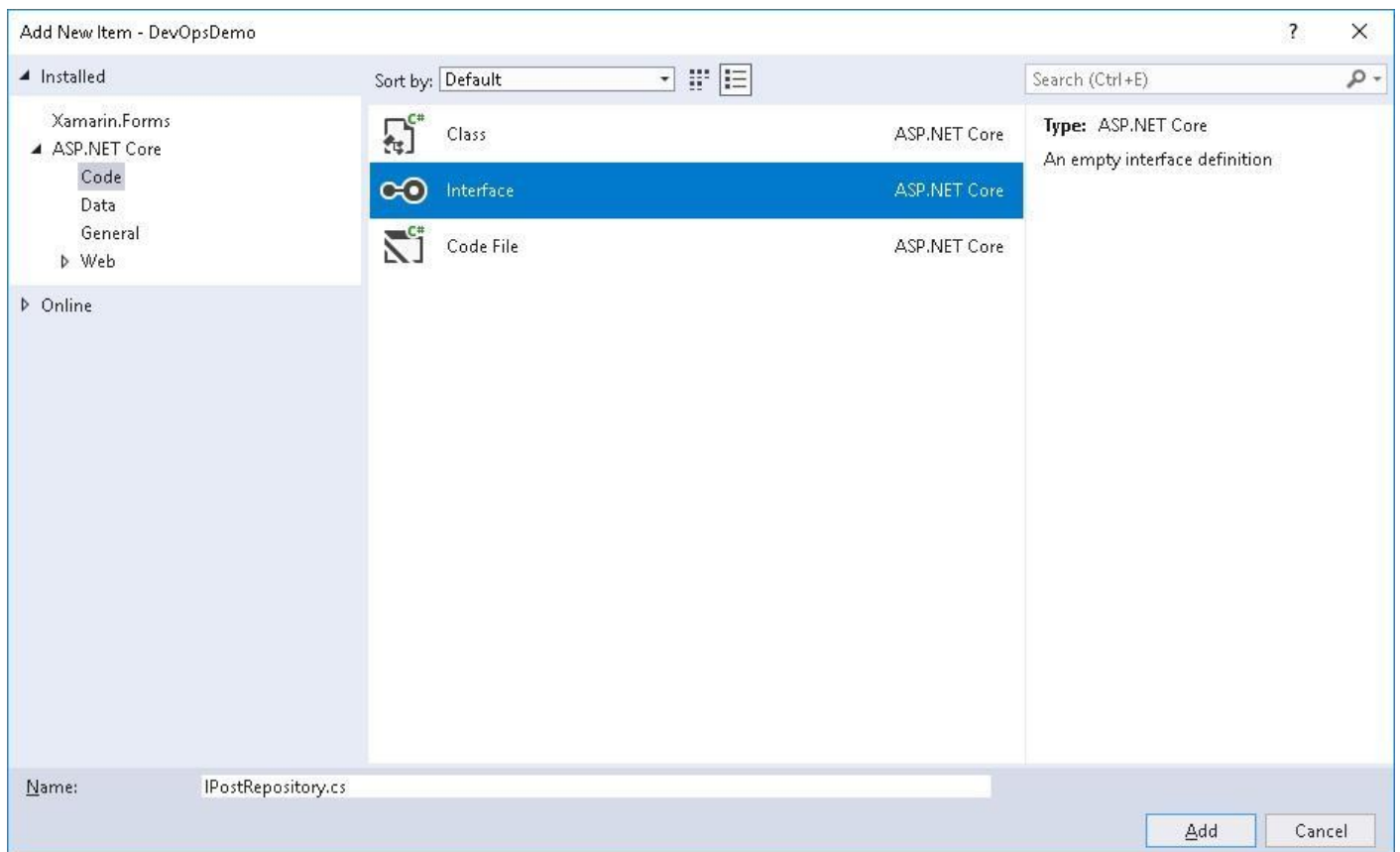
It will open **Add New Item** dialog window, from here we can choose different kinds of Model data file like class, interface etc. So, just choose Class and provide the name for the class as '**PostViewModel.cs**' and click to **Add**.



Open the **PostViewModel** and update the class's code as follows. Here we are adding four properties like **PostId**, **Title**, **Description** and **Author** for PostViewModel class.

```
namespace DevOpsDemo.Models
{
    public partial class PostViewModel
    {
        public int PostId { get; set; }
        public string Title { get; set; }
        public string Description { get; set; }
        public string Author { get; set; }
    }
}
```

Next step to create repository information. So, let create one more folder for repository classes and interfaces as Repository. Once **Repository** folder is created then right click on folder and select **Add > New Item**. It will open **Add New Item** dialog window from where we can select the file type. So, first select an **Interface** and provide the name for interface as **IPostRepository.cs** and click to **Add**.



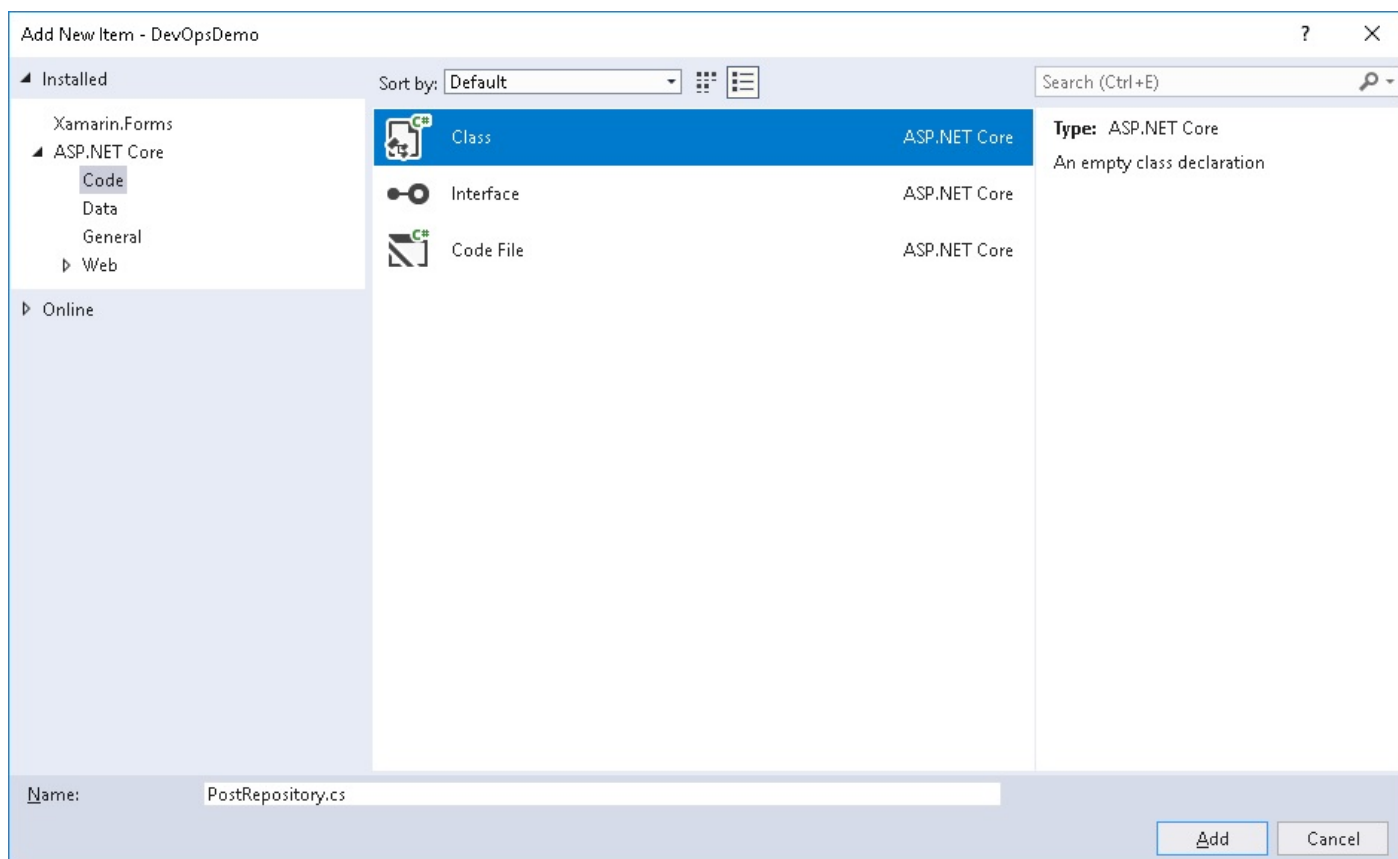
Update the code for **IPostRepository** interface as follows. Here we are adding one method which will return the list of Post.

IPostRepository.cs

```
using DevOpsDemo.Models;
using System.Collections.Generic;

namespace DevOpsDemo.Repository
{
    public interface IPostRepository
    {
        List<PostViewModel> GetPosts();
    }
}
```

Same process needs to follow to add one new class as **PostRepository.cs** in Repository folder as follows.



Here is the **PostRepository** class which implements the **IPostRepository**. We are creating some dummy data for Posts. We are keeping it simple and not implementing the database driven functionality for getting the real time data. It is because this demonstration is only for understanding How DevOps works and How we can implement Continuous Integration and Continuous Delivery.

PostRepository.cs

```
using DevOpsDemo.Models;
using System.Collections.Generic;

namespace DevOpsDemo.Repository
{
    public class PostRepository : IPostRepository
    {
        public List<PostViewModel> GetPosts()
        {
            var posts = new List<PostViewModel> {
                new PostViewModel(){ PostId =101, Title = "DevOps Demo Title 1", Description ="DevOps Demo Description 1", Author="Mukesh Kumar"},
                new PostViewModel(){ PostId =102, Title = "DevOps Demo Title 2", Description ="DevOps Demo Description 2", Author="Banky Chamber"},
                new PostViewModel(){ PostId =103, Title = "DevOps Demo Title 3", Description ="DevOps Demo Description 3", Author="Rahul Rathor"},
            };
        }
    }
}
```

(By: Mukesh Kumar)

```

        return posts;
    }
}

```

Now, it's time to show the data on View which are returning from repository. So, let open the **HomeController** and implement the **constructor dependency injection** for getting the instance of the **PostRepository** class and create a **ActionResult** as **Index**. Here, in the Index method, we will fetch the data using **PostRepository** instance and return the data into the View.

HomeController.cs

```

using DevOpsDemo.Models;
using DevOpsDemo.Repository;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;

namespace DevOpsDemo.Controllers
{
    public class HomeController : Controller
    {
        IPostRepository postRepository;
        public HomeController(IPostRepository _postRepository)
        {
            postRepository = _postRepository;
        }

        public IActionResult Index()
        {
            var model = postRepository.GetPosts();

            return View(model);
        }

        public IActionResult About()
        {
            ViewData["Message"] = "Your application description page.";

            return View();
        }

        public IActionResult Contact()
        {
            ViewData["Message"] = "Your contact page.";

            return View();
        }
    }
}

```

```

        public IActionResult Privacy()
        {
            return View();
        }

        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None,
NoStore = true)]
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id
?? HttpContext.TraceIdentifier });
        }
    }
}

```

We will populate the data on View in tabular format. So, once data will be there, we will iterate on it and display the data as follows.

Index.cshtml

```

@model IList<DevOpsDemo.Models.PostViewModel>

@{
    ViewData["Title"] = "Home Page";
}

<div class="row">
    <h2>Post List</h2>

    <table class="table">
        <thead>
            <tr>
                <th>Post Id</th>
                <th>Title</th>
                <th>Description</th>
                <th>Author</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var item in Model)
            {
                <tr>
                    <td>@Html.DisplayFor(modelItem => item.PostId)</td>
                    <td>@Html.DisplayFor(modelItem => item.Title)</td>
                    <td>@Html.DisplayFor(modelItem => item.Description)</td>
                    <td>@Html.DisplayFor(modelItem => item.Author)</td>
                </tr>
            }
        </tbody>
    </table>

```

(By: Mukesh Kumar)

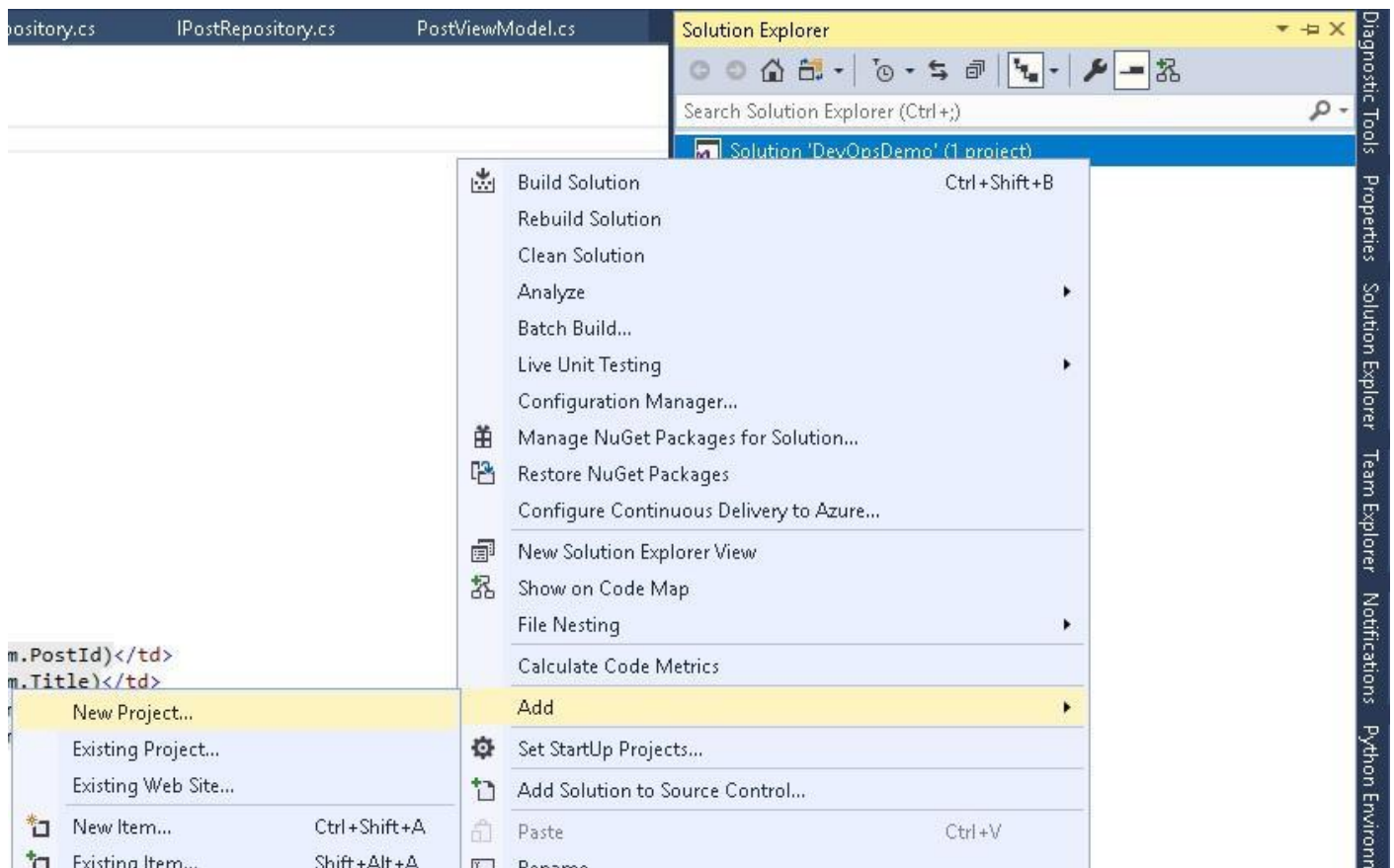
```
</table>
</div>
```

4.2 xUnit Test Project

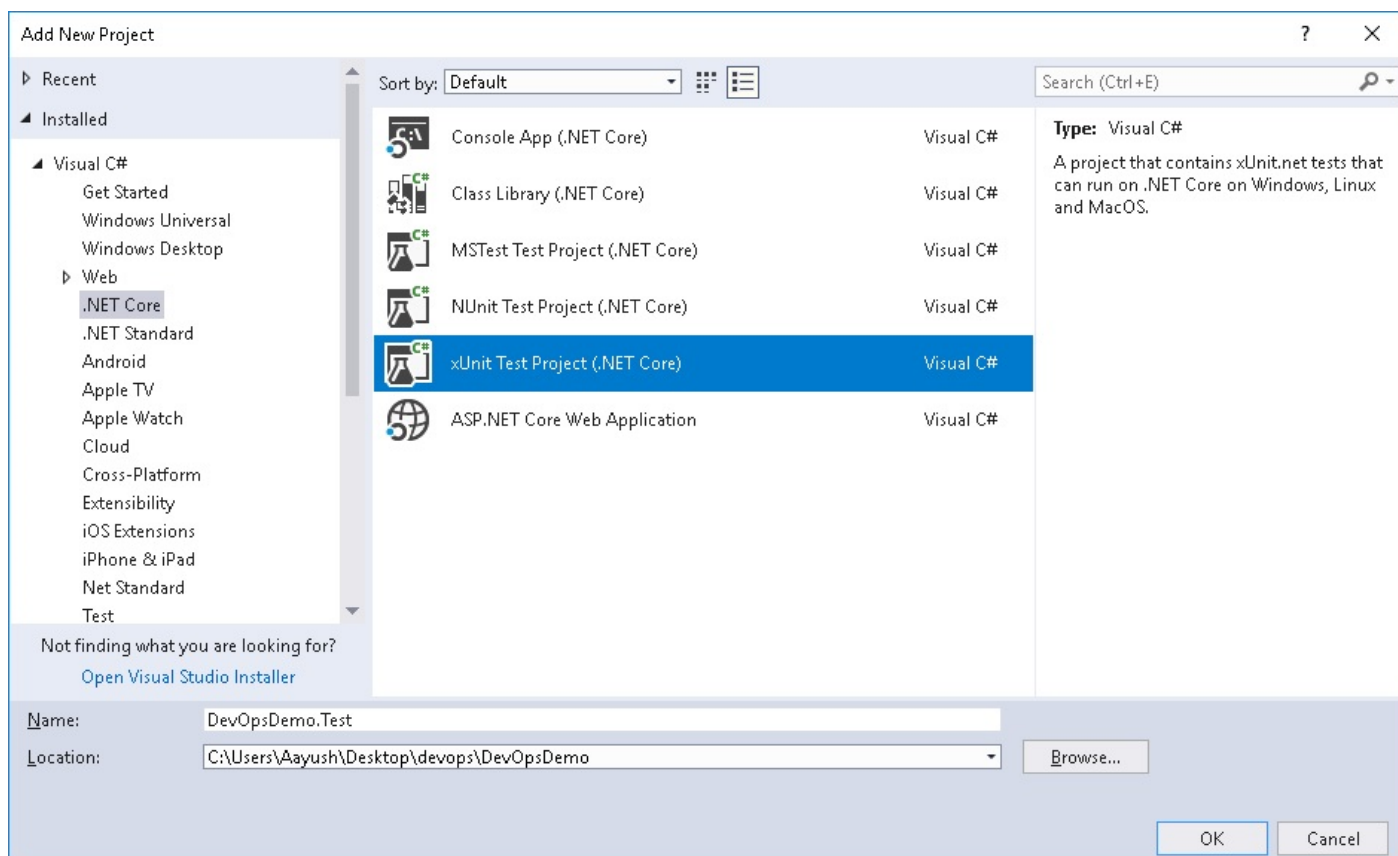
Testing is an important aspects of any product. Without testing, we can not think that product is ready to deliver. We do the testing in many ways but in development phase, while writing the code for specific functionality, we write the test cases against the functionality and check everything is working fine as expected or not.

Unit Test Cases helps us to find the bugs or issues in the code in the earlier stage while building the code. At the time of building the code and creating the artifact, it also executes the test cases and if test cases are being failed, it means, something is wrong and functionality is not as expected.

Here, we will create a separate testing project for ASP.NET Core Web Application. So, let right click on the **DevOpsDemo** solution and select **Add > New Project** as showing in the following image.

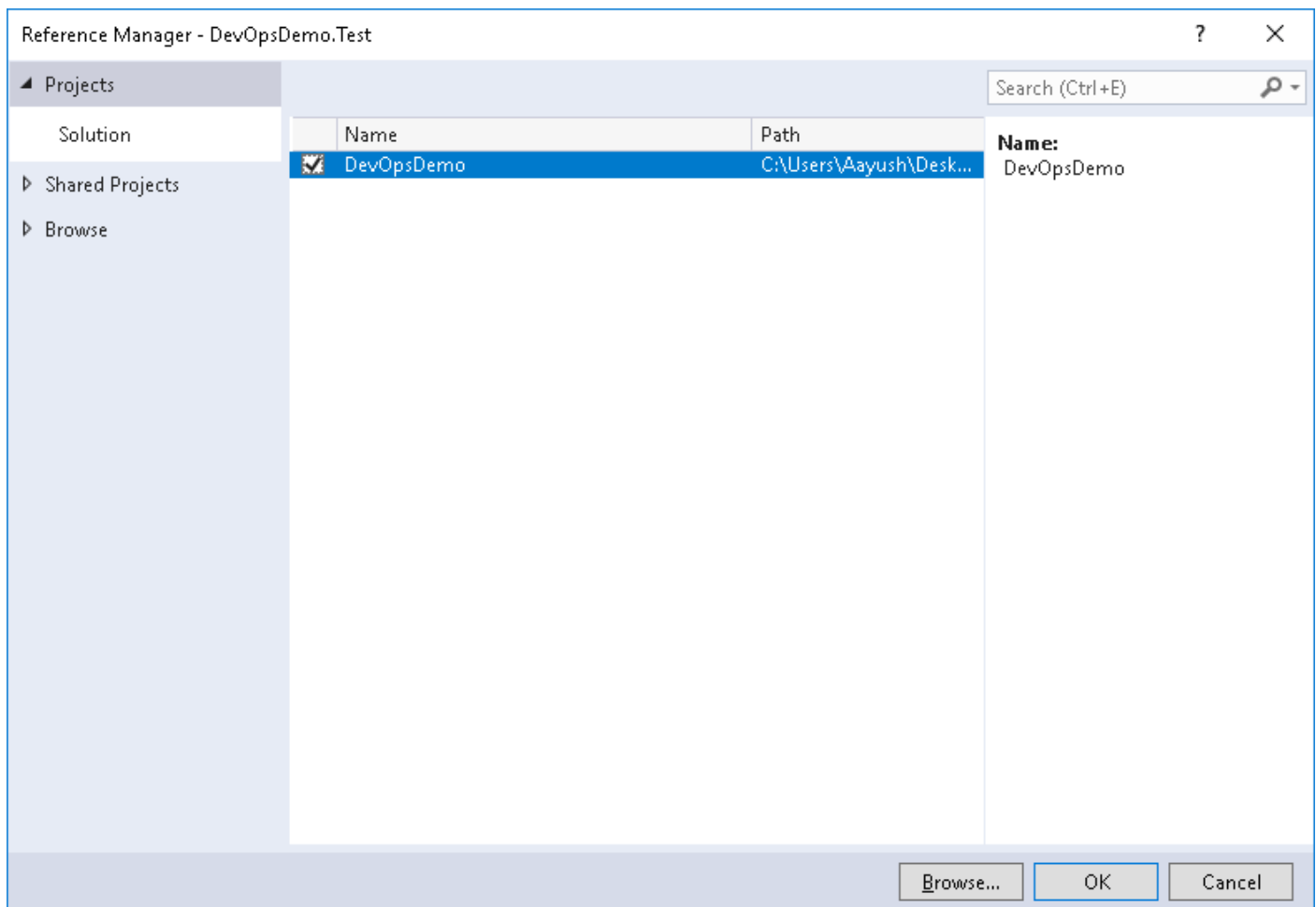


It will open the New Project dialog window, here we have to follow the same process as we have done above for creating the new ASP.NET Core Web Application project. Only we have to change the application template, for a testing project, we will choose the **xUnit Test Project (.Net Core)**. After selecting the project template, provide the name of the testing project as **DevOpsDemo.Test** and click to **OK**.



Within a few seconds, the xUnit testing project will be ready. It will contain one unit test class as **UnitTest1.cs**. Before moving next, just rename **UnitTest1** to **PostTestController**.

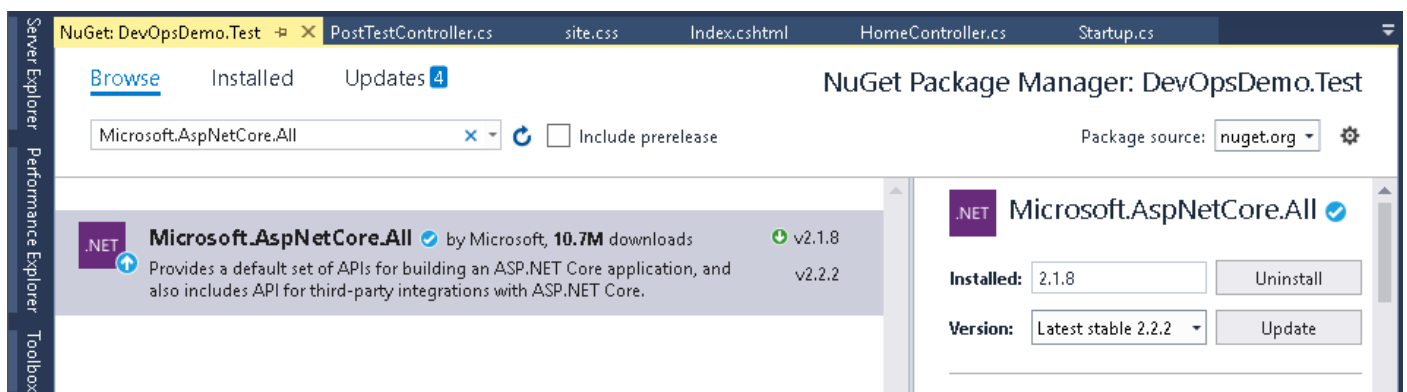
As we will test the main project functionality. So, it is time to add the main project reference in the test project so that we can access the repository and controller for writing the test cases. So, **Right click on the Dependencies in testing project and select Add Reference**. It will open the Reference Manager for **DevOpsDemo.Test** project. From the Projects section in left panel, select the **DevOpsDemo** (mark checked) and click to **OK**.



Install Microsoft.AspNetCore.All from NuGet (version 2.1.8)

As this is the xUnit testing project and by default we can not get all the Asp.Net Core functionality. So, let first add the packages which will provide complete set of APIs for building the Asp.NET Core application. Let **Right click on Dependencies** and select **Manage NuGet Packages**. It will open the **NuGet Package Manager** from where new packages can be searched for installation or see the installed packages or see if any update is available for any package.

So, go to the **Browse** section and search for **Microsoft.AspNetCore.All** in the search section and install it. For this demonstration, we are using version **2.1.8** for **Microsoft.AspNetCore.All**.



(By: Mukesh Kumar)

Open the **PostTestController** and write the few unit test cases for **HomeController** as follows.

PostTestController.cs

```
using DevOpsDemo.Controllers;
using DevOpsDemo.Models;
using DevOpsDemo.Repository;
using Microsoft.AspNetCore.Mvc;
using System;
using System.Collections.Generic;
using Xunit;

namespace DevOpsDemo.Test
{
    public class PostTestController
    {
        private PostRepository repository;

        public PostTestController()
        {
            repository = new PostRepository();
        }

        [Fact]
        public void Test_Index_View_Result()
        {
            //Arrange
            var controller = new HomeController(this.repository);

            //Act
            var result = controller.Index();

            //Assert
            Assert.IsType<ViewResult>(result);
        }

        [Fact]
        public void Test_Index_Return_Result()
        {
            //Arrange
            var controller = new HomeController(this.repository);

            //Act
            var result = controller.Index();

            //Assert
            Assert.NotNull(result);
        }
    }
}
```

(By: Mukesh Kumar)

```

[Fact]
public void Test_Index_GetPosts_MatchData()
{
    //Arrange
    var controller = new HomeController(this.repository);

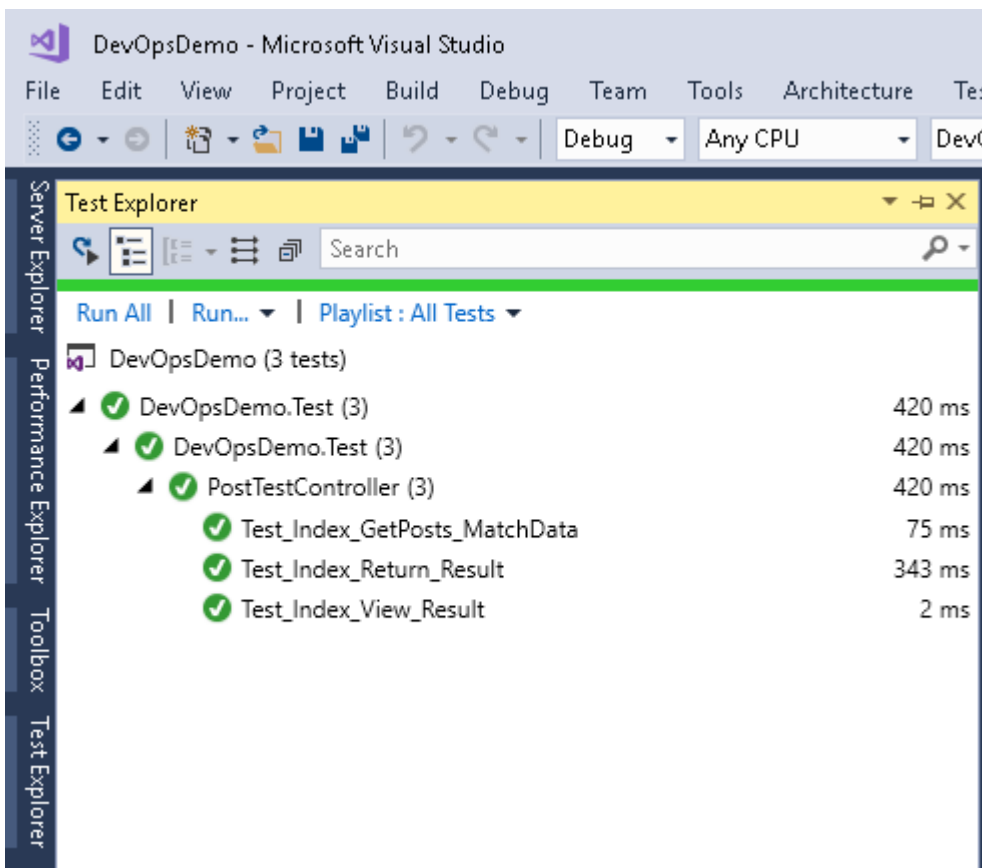
    //Act
    var result = controller.Index();

    //Assert
    var viewResult = Assert.IsType<ViewResult>(result);
    var model
    Assert.IsAssignableFrom<List<PostViewModel>>(viewResult.ViewData.Model);

    Assert.Equal(3, model.Count);
    Assert.Equal(101, model[0].PostId);
    Assert.Equal("DevOps Demo Title 1", model[0].Title);
}
}

```

Let open the **Test Explorer** and click to **Run All** as shown in following image to start executing the unit test cases, which start building the solution and start executing the unit test cases. Once all unit test cases run then you can see the status as green. As per the following image, we can see that all test cases are passed.



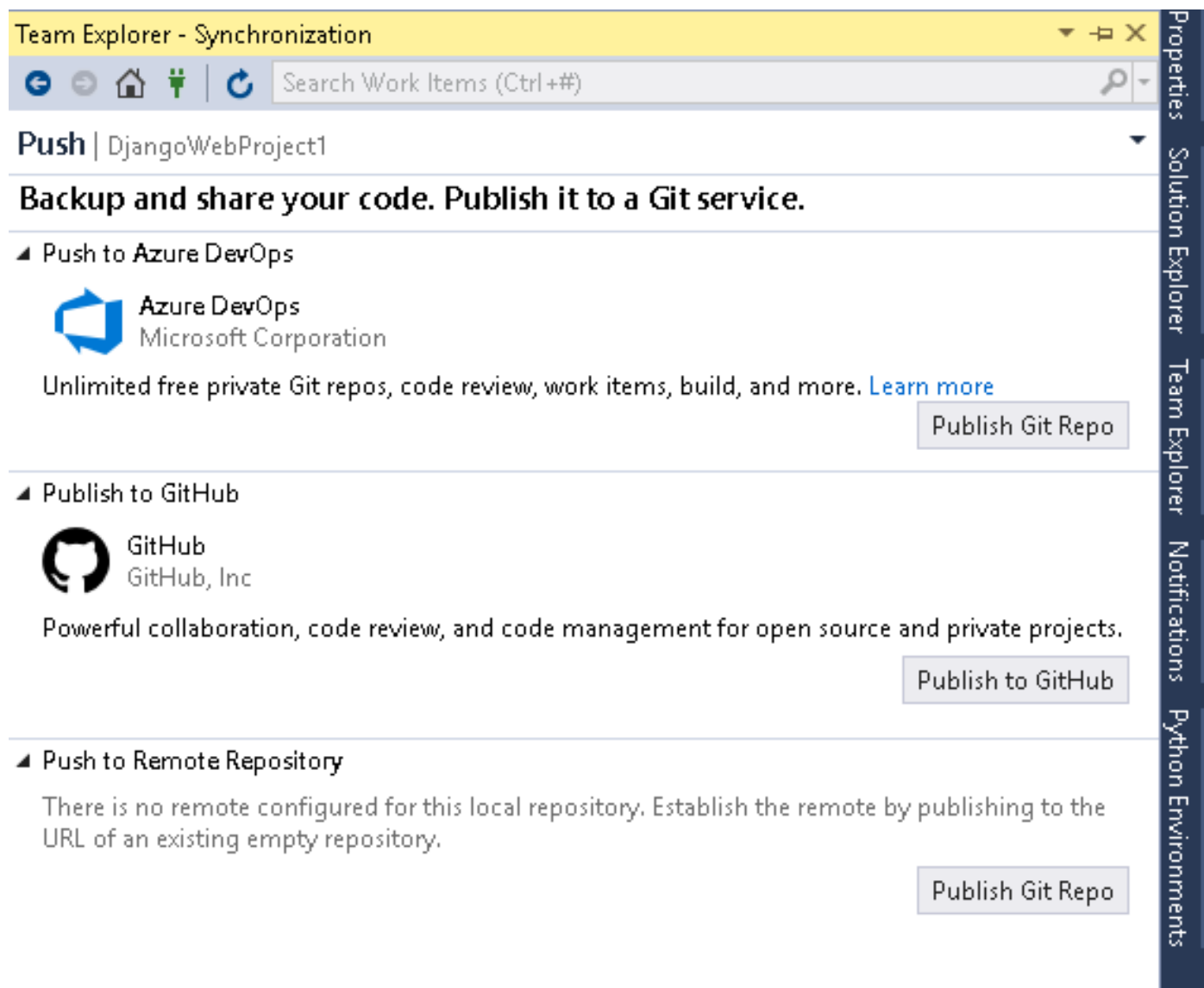
(By: Mukesh Kumar)

4.3 Add Project to GitHub

Now it's time to add this above project to any repository like TFS, GitHub, Bitbucket etc. so that we can access it while setting up Azure DevOps pipeline. Let choose the **GitHub** so that it will be accessible publically. So, right click on the Solution (**DevOpsDemo**) and select **Add Solution to Source Control**. It will add your file in a local repository

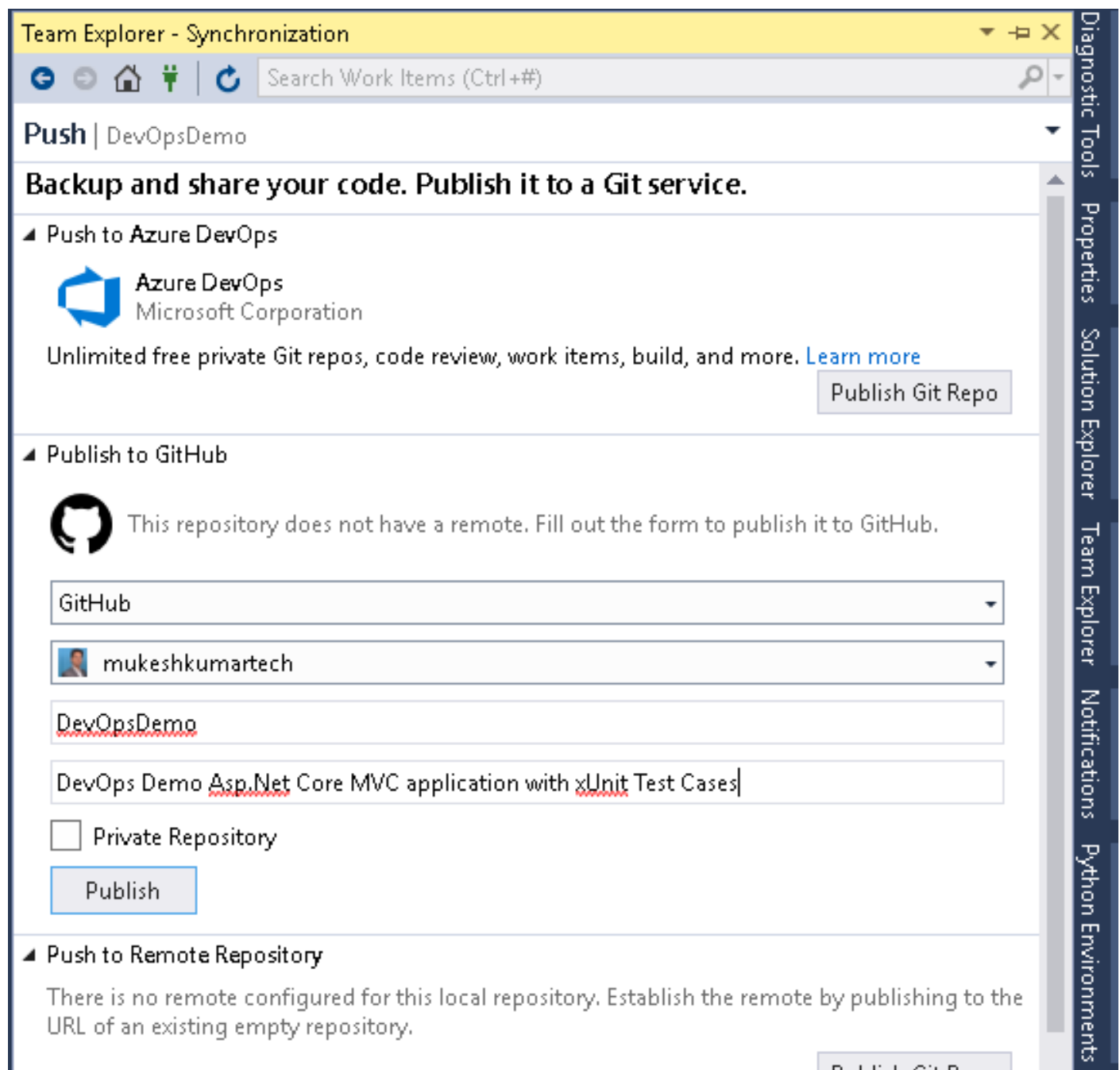
Now, go to Team Explorer, here we can find '**Sync**' option, just click on it. It will open the window from where we can publish code to a particular repository like **GitHub** as follow.

Here, we will choose the **Publish to GitHub**, so that we can publish this code to **GitHub** public repository.



Once we click on the **Publish to GitHub**, it will ask for **Authentication with your GitHub account**. Let provide the credentials to log in with **GitHub** Account. After logging in with **GitHub**, we will get the following screen. Here, we can provide the **name** and **description** of the repository which will create in **GitHub** for adding this project. We can make it private to check the checkbox for Private Repository. For this demonstration, we will keep it public, so let click on the Publish button.

It will take few minutes to create the new repository in **GitHub** with provided name and publish the whole code into this repository.



Now, go to following GitHub URL where we can find the published code for this whole demonstration.

<https://github.com/mukeshkumartech/DevOpsDemo>

The screenshot shows the GitHub interface for the repository 'mukeshkumartech/DevOpsDemo'. The repository is described as 'DevOps Demo Asp.Net Core MVC application with xUnit Test Cases'. It has 20 commits, 1 branch, 0 releases, and 1 contributor. The repository is on the 'master' branch. The commit history is displayed as follows:

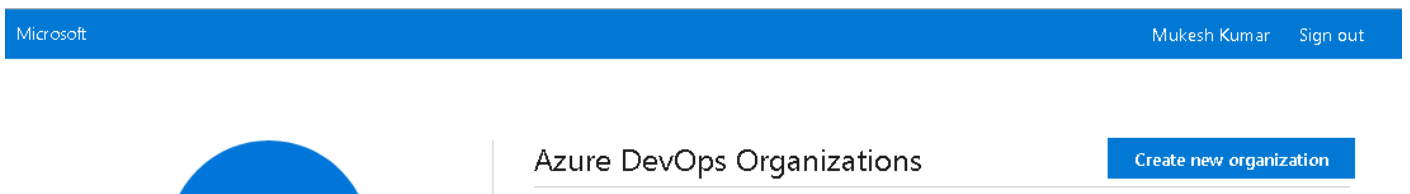
Commit	Message	Time
mukeshkumartech Home Page Release 5.0	Home Page Release 5.0	7 days ago
DevOpsDemo.Test	Add project files.	19 days ago
DevOpsDemo	Home Page Release 5.0	7 days ago
.gitattributes	Add .gitignore and .gitattributes.	19 days ago
.gitignore	Add .gitignore and .gitattributes.	19 days ago
DevOpsDemo.sln	Add project files.	19 days ago
azure-pipelines.yml	Set up CI with Azure Pipelines	19 days ago

5. Create Organization and Project

Let move to <https://visualstudio.microsoft.com> and create new account or **sign in** with existing credentials.



Once we will be logged in successfully, then we have a new screen as follows. From here we can create a new organization on clicking '**Create new organization**' button.



We have another option to create the organization, if we have already an organization and we would like to add a new one then we can create to using **Create Organization** option as follows.



It will take few seconds to configuring Azure DevOps new organization.



Next screen will ask about the name of the **Azure DevOps organization**, here we are giving the name as '**TechHubOrg**', we have to also provide the hosting location as '**South India**' for our organization. At the time of creating the new organization, we can also provide the name of the project. This project will auto create inside this new organization. Now, fill the security question and click to **Continue** button.



Almost done...

Name your Azure DevOps organization *

dev.azure.com/ TechHubOrg

We'll host your projects in *

South India

Name your project

DevOpsDemo

Enter the characters you see

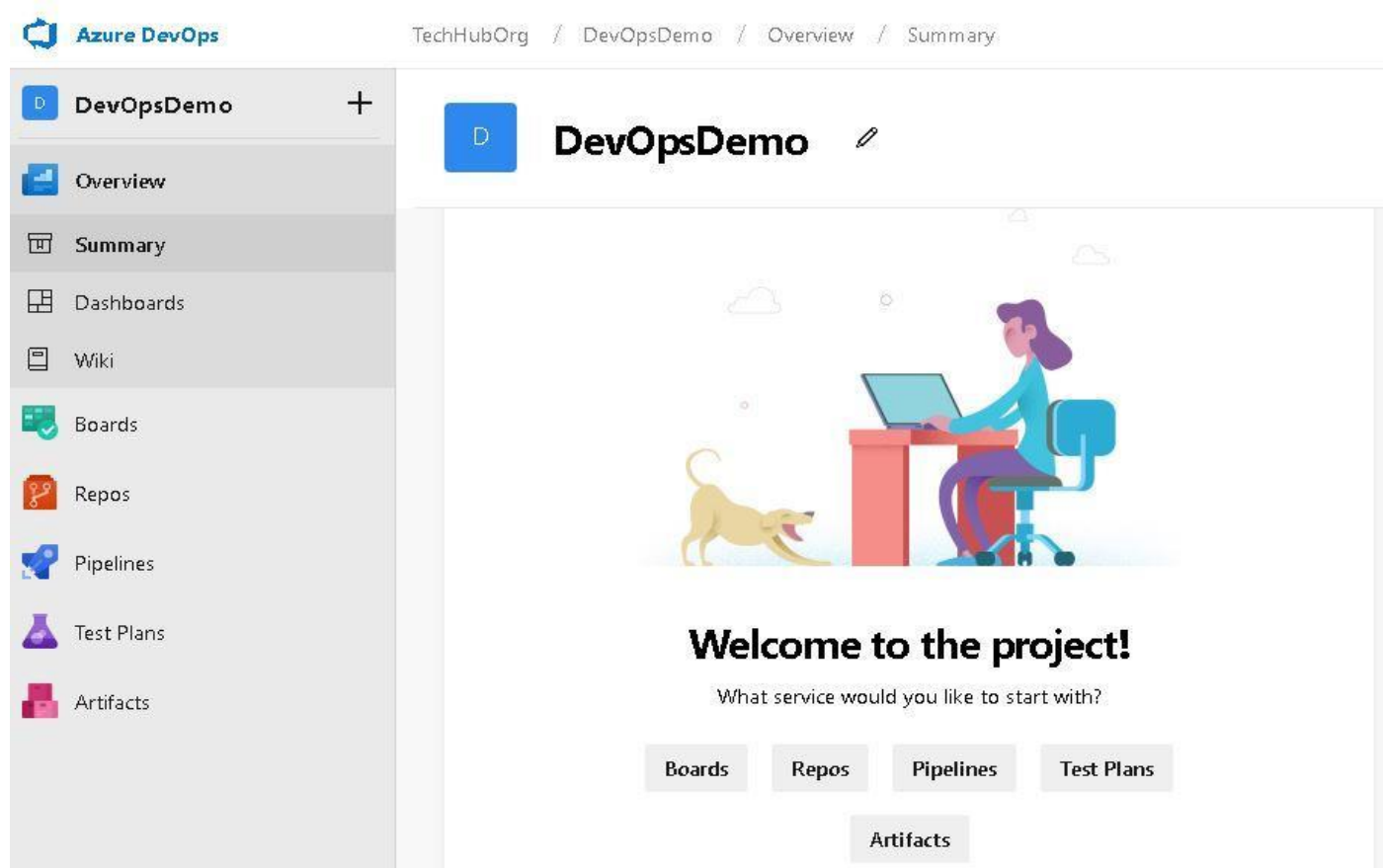
[New](#) | [Audio](#)



5WKQVK54Y

Continue

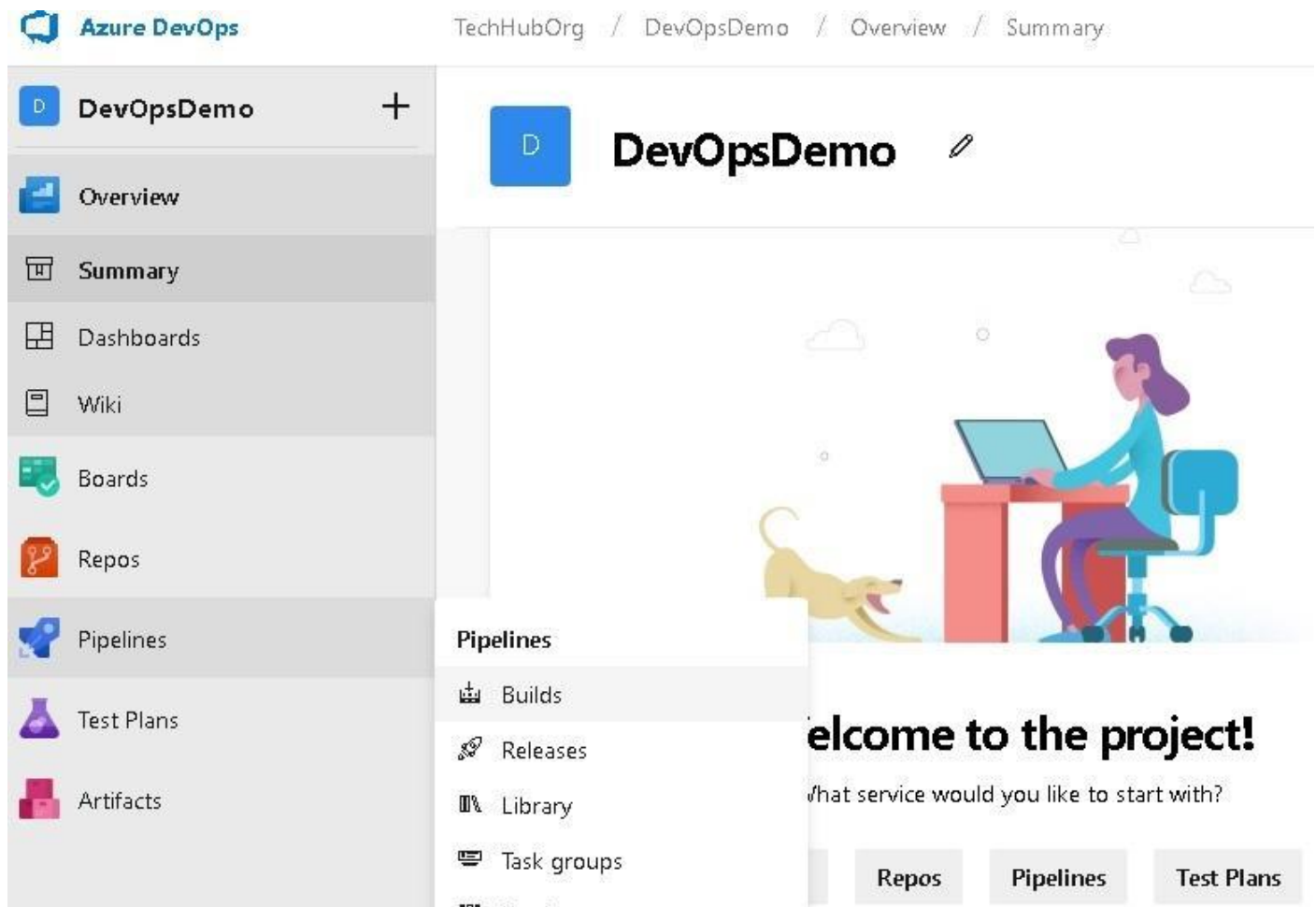
Next screen will open the project which has created recently at the time of creating the organization. Here, we have created **DevOpsDemo** project inside the **TechHubOrg** organization. Following is the welcome screen for the DevOpsDemo project. From here, we can manage all the things like Boards, Repos, Pipeline and Test Plans etc. which will be project specific.



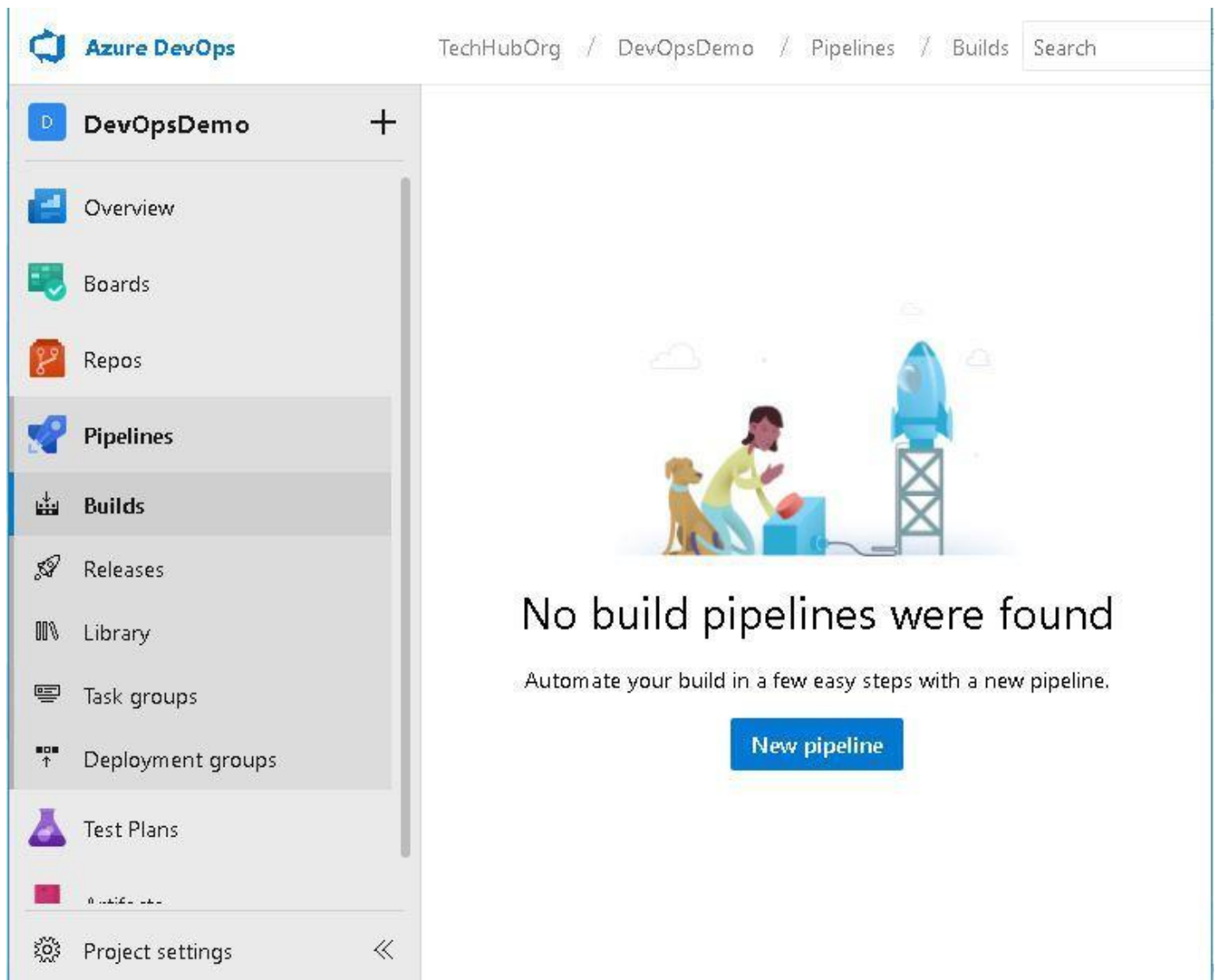
6. Continuous Integration

Let create the Azure Build pipeline. Build pipeline is basically responsible for building the code and testing the corresponding Unit Test Cases once the developer checks in the code into the repository. If everything will fine, then it will create the Artifact which can be used for deployment.

For creating the build pipeline in **Azure DevOps**, click on **Pipeline** and select **Build** as shown in the following images.



Next screen says that we don't have any build pipeline setup yet and it has one button as new pipeline for creating new one. So, let click on **New Pipeline**. It will only create the build pipeline.



After clicking on **New Pipeline**, it will start the wizard for setting up the build pipeline. It will basically ask for where code repository file and do some configuration.

Here, we will not go with Wizard process, we will use the **Visual Designer** for creating and configuring the Azure DevOps Build pipeline. So, let click on the link '**Use the Visual Designer**' as shown in below image.

As we select the Visual Designer, it will ask to choose the repository. All available repositories are part of the Azure DevOps like Azure Repo Git, GitHub, Bitbucket etc. We have already pushed our project code which we have already created above in **GitHub**. So, select the **GitHub** and provide the connection name, which will use further. Now, click to **Authorize using OAuth**. It will open a dialog window for login to **GitHub**. Here we have to provide the GitHub credentials for Authorization with **GitHub**.

Azure DevOps

TechHubOrg / DevOpsDemo / Pipelines

Search

DevOpsDemo

Overview

Boards

Repos

Pipelines

Builds

Releases

Library

Task groups

Deployment groups

Test Plans

Artifacts

Select your repository

Tell us where your sources are.
You can customize how to get these sources from the repository later.

Select a source

Azure Repos Git

GitHub

GitHub Enterprise

Subversion

Bitbucket Cloud

External Git

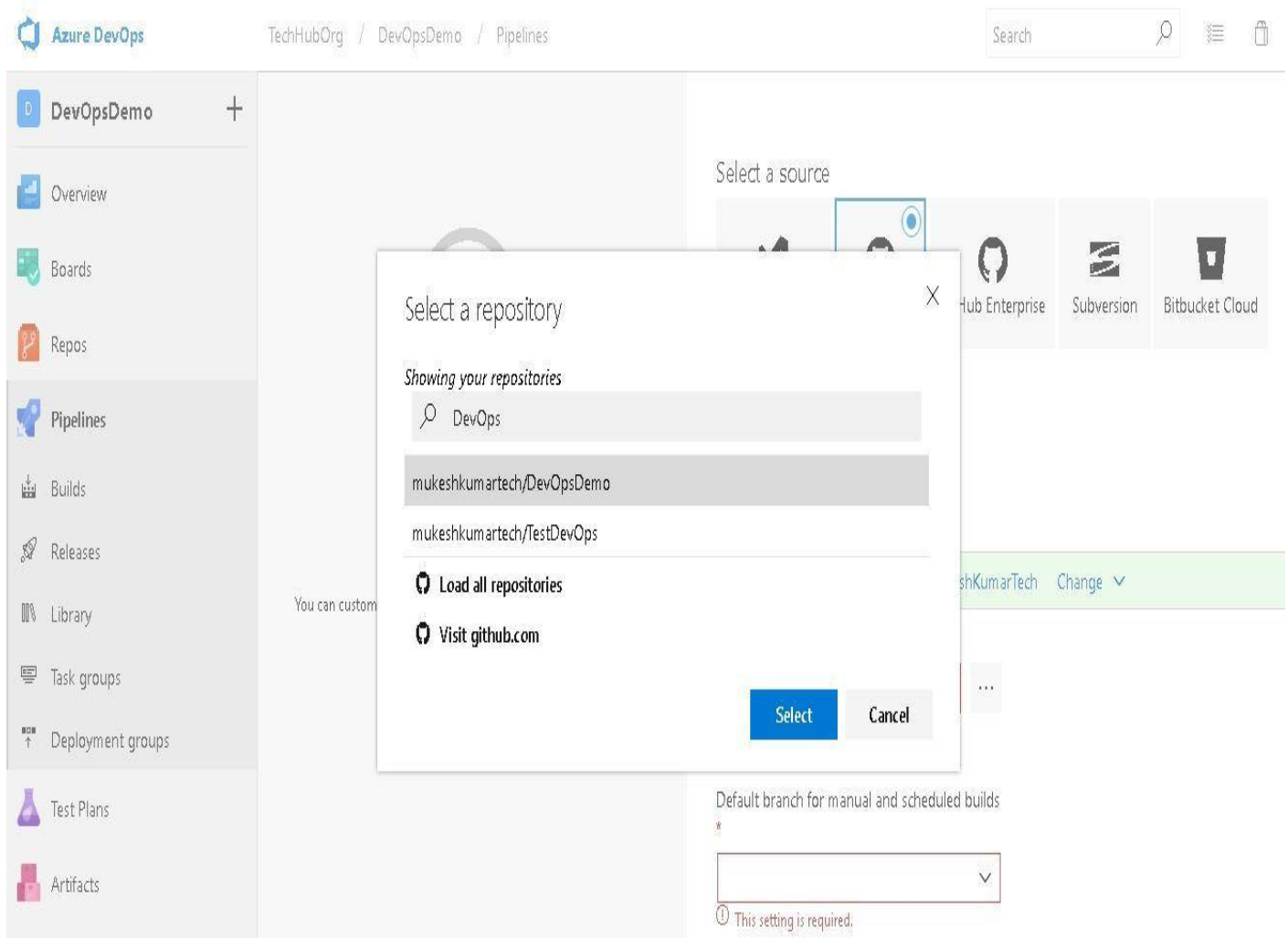
We need your authorization to access your repositories

Connection name *

MukeshKumarTech

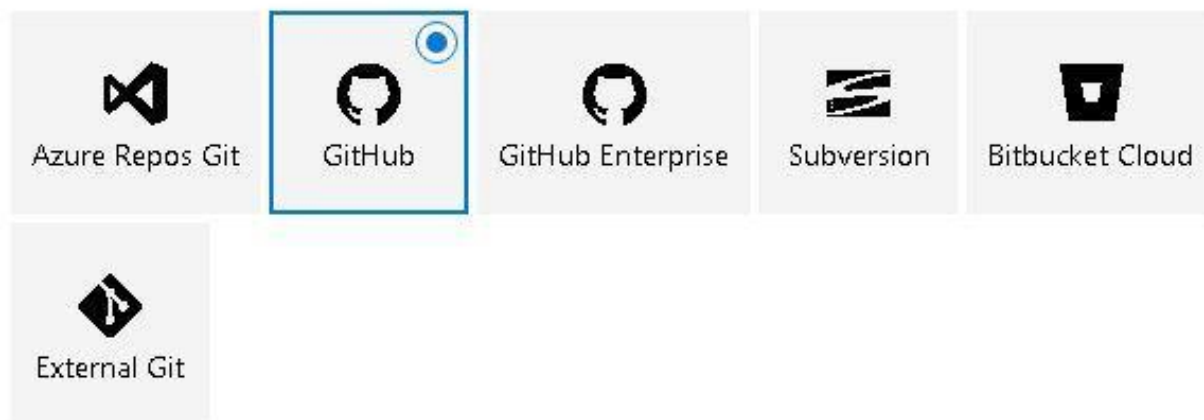
Authorize using OAuth Or Authorize with a GitHub personal access token

Once we will authorize with GitHub account then we can see all **GitHub** repository. Here we have to select **DevOpsDemo** because we have created and added our project code into DevOpsDemo repository.



Let select the repository and then the branch, by default it will be **master** [Other branch can also be selected] and click to **Continue**.

Select a source



✓ Authorized using connection: [MukeshKumarTech](#) [Change](#) ▼

Repository * | [Manage on GitHub](#) [🔗](#)

mukeshkumartech/DevOpsDemo

...

Default branch for manual and scheduled builds *

master ▼

Continue

Next screen will ask about selecting the project template, as we have created the **Asp.Net Core application**. So, here we will choose the **Asp.Net Core** and click to **Apply**.

Select a template

Or start with an  **Empty job**



Python package

Create and test a Python package on multiple Python versions.



Xcode

Build, test, archive, or package an Xcode workspace on macOS.

Others



Ant

Build and test a Java project with Apache Ant.



ASP.NET Core

Build and test an ASP.NET Core web application.

Apply



ASP.NET Core (.NET Framework)

Build an ASP.NET Core web application that targets the full .NET Framework.

Next screen will all about **Azure DevOps Build pipeline configuration** like name of the build pipeline, restoring the code from repository, building the code, testing the unit test cases etc.

By default, **Tasks** tab will be selected. Here we can provide the **name of build pipeline** as we have given it as **'DevOpsDemo-CI'**.

We have two options for project, first as 'Projects to restore and build'. For our case, we don't need to do anything. Let keep the default one. Second, 'Project to test', here, our testing project name was **DevOpsDemo.Test**. So, we will mention it inside **'Project to Test'**.

By default, we have available jobs like **Restore, Build, Test and Publish**. If anything else requires and we would like to perform in between in build process, we can add new job using **+ sign**.

(By: Mukesh Kumar)

The screenshot shows the 'Build pipeline' configuration page in Azure DevOps. The left sidebar contains navigation options: Overview, Boards, Repos, Pipelines (selected), Builds, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main area is titled 'DevOpsDemo-CI' and has tabs for Tasks, Variables, Triggers, Options, Retention, and History. The 'Tasks' tab is active, showing a list of tasks: Get sources (mukeshkumartech/DevOpsDemo, master), Agent job 1 (Run on agent), Restore (.NET Core), Build (.NET Core), Test (.NET Core), Publish (.NET Core), and Publish Artifact (Publish Build Artifacts). On the right, the configuration details for the selected task are shown: Name is 'DevOpsDemo-CI', Agent pool is 'Hosted Ubuntu 1604', Parameters are set to 'Project(s) to restore and build' with the value '**/*.csproj', and 'Project(s) to test' with the value '**/DevOpsDemo.Test.csproj'.

Let move to **Variables** tab, here we can configure the build setting like **Build Configuration**, **Build Platform**.

The screenshot shows the 'Variables' tab in the Azure DevOps Pipelines configuration page. The left sidebar is the same as the previous screenshot. The main area is titled 'DevOpsDemo-CI' and has tabs for Tasks, Variables (selected), Triggers, Options, Retention, and History. The 'Variables' tab is active, showing a list of pipeline variables. The variables are organized into 'Variable groups' and 'Predefined variables'. The predefined variables are listed in a table:

Name ↑	Value
BuildConfiguration	Release
BuildPlatform	any cpu
system.collectionId	7cbeca5f-c199-4268-a329-5644fbde5dd2
system.debug	false
system.definitionId	< No pipeline id yet >
system.teamProject	DevOpsDemo

Below the table, there is a '+ Add' button to add new variables.

Move to next tab '**Triggers**'. It is very important tab from where we can enable the **Continuous Integration**. So, just check the checkbox for **Enable Continuous Integration**. It means, as we will check in our code into repository, it will auto start the building code and creating the build artifact which will be deployed in release cycle.

(By: Mukesh Kumar)

If we have multiple branch and we would like to filter any specific branch then we can define it into **Branch Filters** section.

Let move to **Options** tab, here we get the options to define the build version number using '**Build number format**' section. This one is by default format, but we can modify it as per our requirement.

Create work item on failure is also a great feature which is used to create new work item once build becomes fail. In a Build Job section, we can define the build job timeout in minutes.

Name	Condition	Value
+ Add		

Now, time to move on **Retention** tab. Here we can define that for how many days, we would like to keep the build information. How many good builds we would like to keep for those days. Everything can be set up here. Apart from these, we can also define, what information should be deleted and what are not at the clearing the build information.

The screenshot shows the Azure DevOps interface for the 'DevOpsDemo-CI' pipeline. The 'Retention' tab is active, displaying a list of policies under the heading 'Policies'. The policies are:

- Keep for 20 days, 1 good build (+refs/heads/*)
- Keep for 30 days, 10 good builds (Maximum)

On the right, the 'Settings' panel is visible, showing options for 'Branch filters' and 'Days to keep'.

Branch filters:

- Type: Include
- Branch specification: refs/heads/*

Days to keep:

- Days to keep: 20
- Minimum to keep: 1

When cleaning up builds, delete the following:

- ☒ Build record
- ☐ Source label
- ☒ File share
- ☒ Symbols
- ☒ Automated test results

Let move to **History** tab, here we are doing setup the **Azure DevOps build cycle**, so we don't have any history information. But in future, when any build cycle performs then we can see all the history here.

We have done configuring the Azure DevOps Build Pipeline, now let click to Save and Queue.

The screenshot shows the Azure DevOps interface for the 'DevOpsDemo-CI' pipeline. The 'History' tab is active, displaying a table with the following columns:

Changed By	Change Type	Changed Date	Comment

Once we will click to **Save and Queue** then it will ask for confirmation. So, just click to **Save and Queue** button once more as follows.

×

Save build pipeline and queue

Agent pool

Hosted Ubuntu 1604

Branch

master

Commit

Variables

Demands

BuildConfiguration

Release

BuildPlatform

any cpu

system.debug

false

+ Add

Save & queue

Cancel

After clicking on Save and Queue button. First it will save the configuration for build pipeline and start queuing a build. As per the following image, we can see a message '**Build #20190216.1 has been queued**'.

TechHubOrg / DevOpsDemo / Pipelines

Search

DevOpsDemo

Overview

Boards

Repos

Pipelines

Builds

Build #20190216.1 has been queued.

Tasks Variables Triggers Options Retention History Save & queue Discard Summary Queue

Changed By	Change Type	Changed Date	Comment
Mukesh Kumar	Add	16/02/2019 18:35	

Let click on the **build # number** and we can see the log for jobs which are performing. As per the following images, build has already processed the Initialize Job, Checkout Job, Restoring the code Job and now has been moved on Build. It is performing the Build.

DevOpsDemo

Overview

Boards

Repos

Pipelines

Builds

Releases

Library

Task groups

Deployment groups

Test Plans

Artifacts

#20190216.1: Set up CI with Azure Pipelines

Manually run just now by Mukesh Kumar mukeshkumartech/DevOpsDemo master 731d165

Cancel build

Logs Summary Tests

Agent job 1 Job

Pool: Hosted Ubuntu 1604 · Agent: Hosted Agent

Started: 2/17/2019, 12:09:33 AM 1m 38s

Initialize Agent	succeeded	<1s
Initialize job	succeeded	<1s
Checkout	succeeded	7s
Restore	succeeded	1m 12s
Build		15s

```

Starting: Build
Task : .NET Core

```

After build completes, it will move to Test the Unit Test Cases, which we have already defined at the time of creating the Build Pipeline.

Repos

Pipelines

Builds

Releases

Library

Task groups

Deployment groups

Test Plans

Artifacts

Project settings

Agent job 1 Job

Pool: Hosted Ubuntu 1604 · Agent: Hosted Agent

Started: 2/17/2019, 12:09:33 AM 2m 26s

Initialize Agent	succeeded	<1s
Initialize job	succeeded	<1s
Checkout	succeeded	7s
Restore	succeeded	1m 12s
Build	succeeded	32s
Test		31s

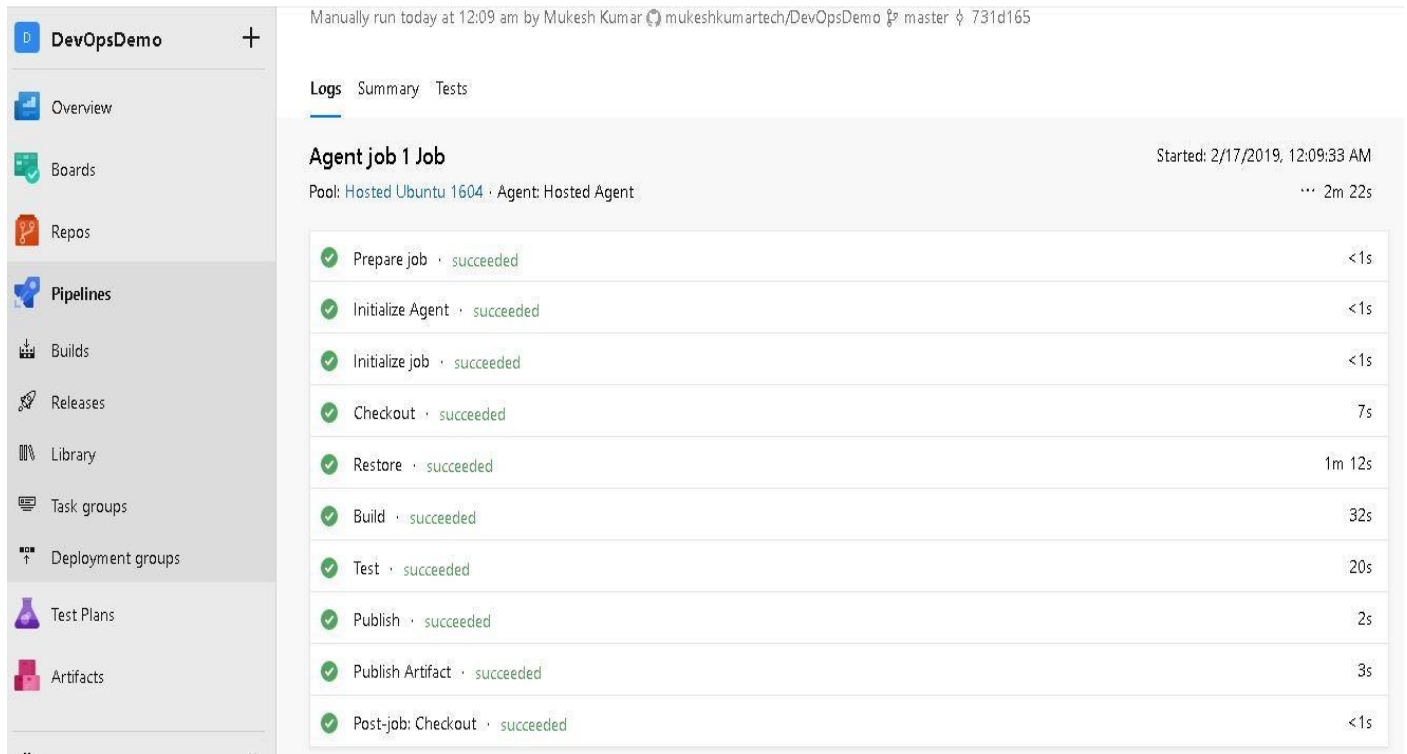
```

/usr/share/dotnet/sdk/2.2.103/Sdks/Microsoft.NET.Sdk/targets/Microsoft.NET.Sdk.DefaultItems.targets(153,5): warning NETSDK1071: A PackageReference to 'Microsoft.AspNetCore.All' specified a Version of '2.1.8'. Specifying the version of this package is not recommended. For more information, see https://aka.ms/sdkimplicitrefs [/home/vsts/work/1/s/DevOpsDemo.Test/DevOpsDemo.Test.csproj]
Build completed.
Test run for /home/vsts/work/1/s/DevOpsDemo.Test/bin/Release/netcoreapp2.1/DevOpsDemo.Test.dll (.NETCoreApp, Version=v2.1)

```

(By: Mukesh Kumar)

After few minutes, we can see that all jobs complete successfully as follows with **succeeded message**. If something will wrong with our code then it will fail with proper information.



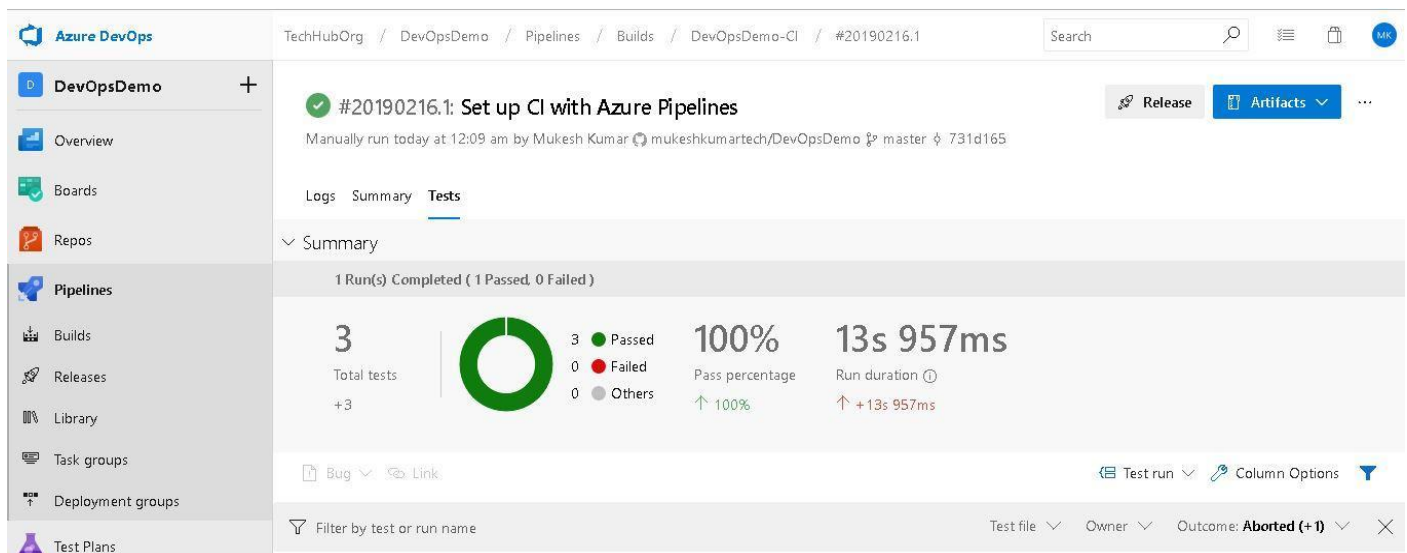
Manually run today at 12:09 am by Mukesh Kumar mukeshkumartech/DevOpsDemo master 731d165

Logs Summary Tests

Agent job 1 Job Started: 2/17/2019, 12:09:33 AM
Pool: Hosted Ubuntu 1604 · Agent: Hosted Agent ... 2m 22s

✓ Prepare job · succeeded	<1s
✓ Initialize Agent · succeeded	<1s
✓ Initialize job · succeeded	<1s
✓ Checkout · succeeded	7s
✓ Restore · succeeded	1m 12s
✓ Build · succeeded	32s
✓ Test · succeeded	20s
✓ Publish · succeeded	2s
✓ Publish Artifact · succeeded	3s
✓ Post-job: Checkout · succeeded	<1s

Here, we have to confirm that **our Unit Test Cases** has executed and passed successfully or not. So, let move to Tests tab, here we can the summary of executed Unit Test Cases. We had created 3 Unit Test Cases and all have been passed.



TechHubOrg / DevOpsDemo / Pipelines / Builds / DevOpsDemo-CI / #20190216.1 Search

✓ #20190216.1: Set up CI with Azure Pipelines Release Artifacts ...

Manually run today at 12:09 am by Mukesh Kumar mukeshkumartech/DevOpsDemo master 731d165

Logs Summary Tests

Summary

1 Run(s) Completed (1 Passed, 0 Failed)

3 Total tests +3

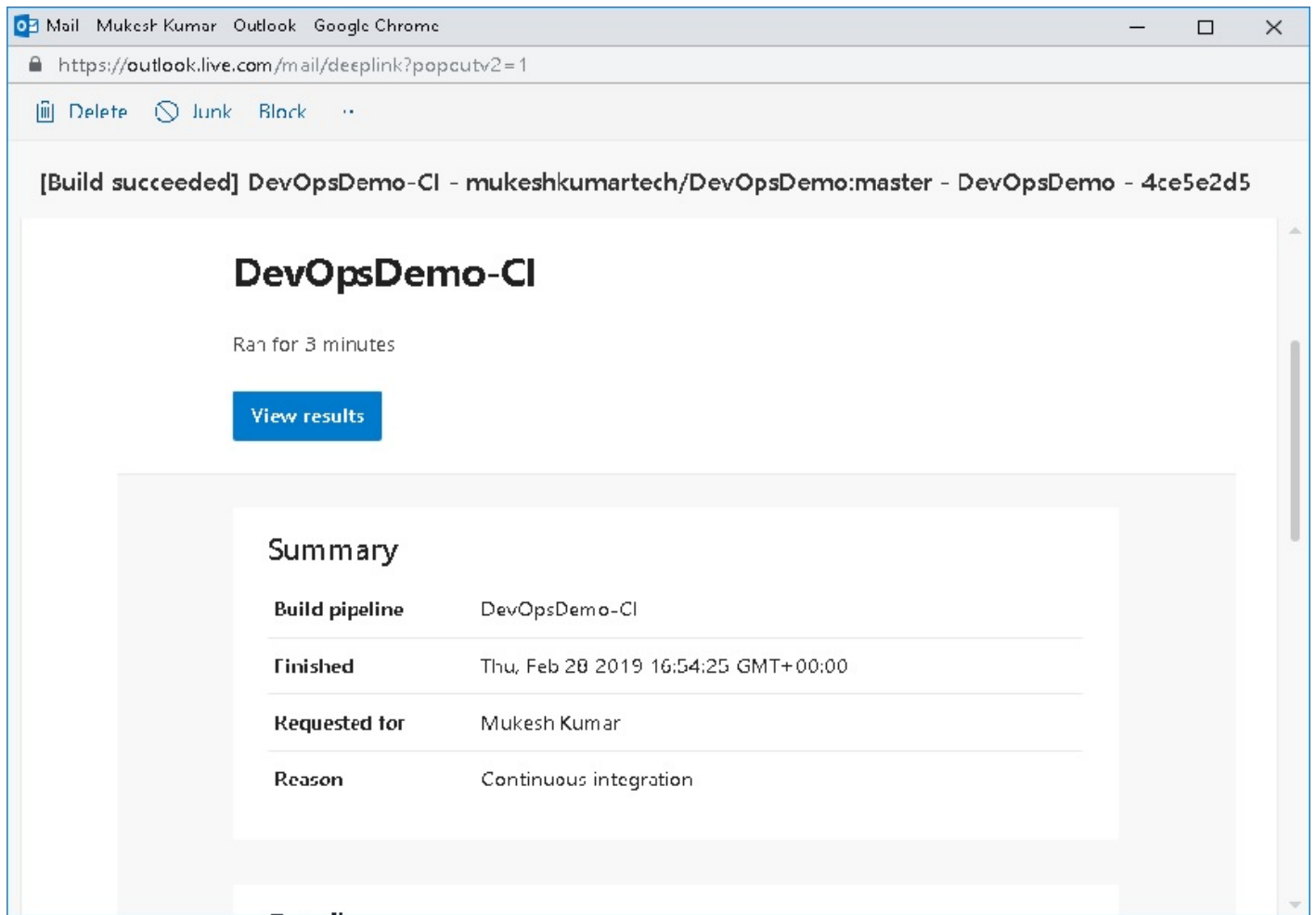
100% Pass percentage ↑ 100%

13s 957ms Run duration ⓘ ↑ +13s 957ms

3 ● Passed
0 ● Failed
0 ● Others

Filter by test or run name Test file Owner Outcome: Aborted (+1) X

We will also inform about any build process completes in the Azure DevOps Build pipeline through Email as follows.



So, as we have performed several things as follows.

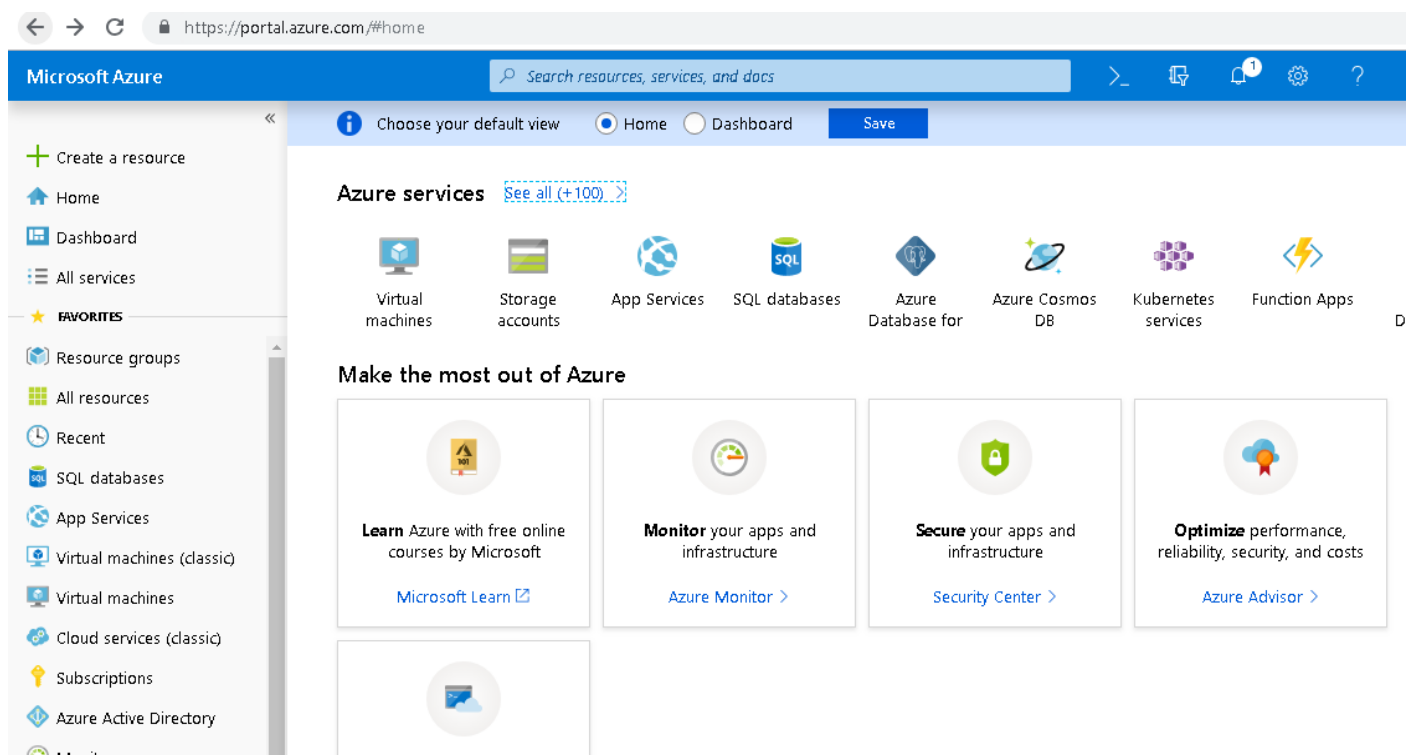
- ✓ Created the Azure DevOps Build Pipeline.
- ✓ Configured the Build Pipeline and Enable the Continues Integration.
- ✓ Save the Build and Queue the Default Build.
- ✓ Azure DevOps Build executed successfully.
- ✓ Unit Test Cases passed successfully.

7. Create Azure App Services

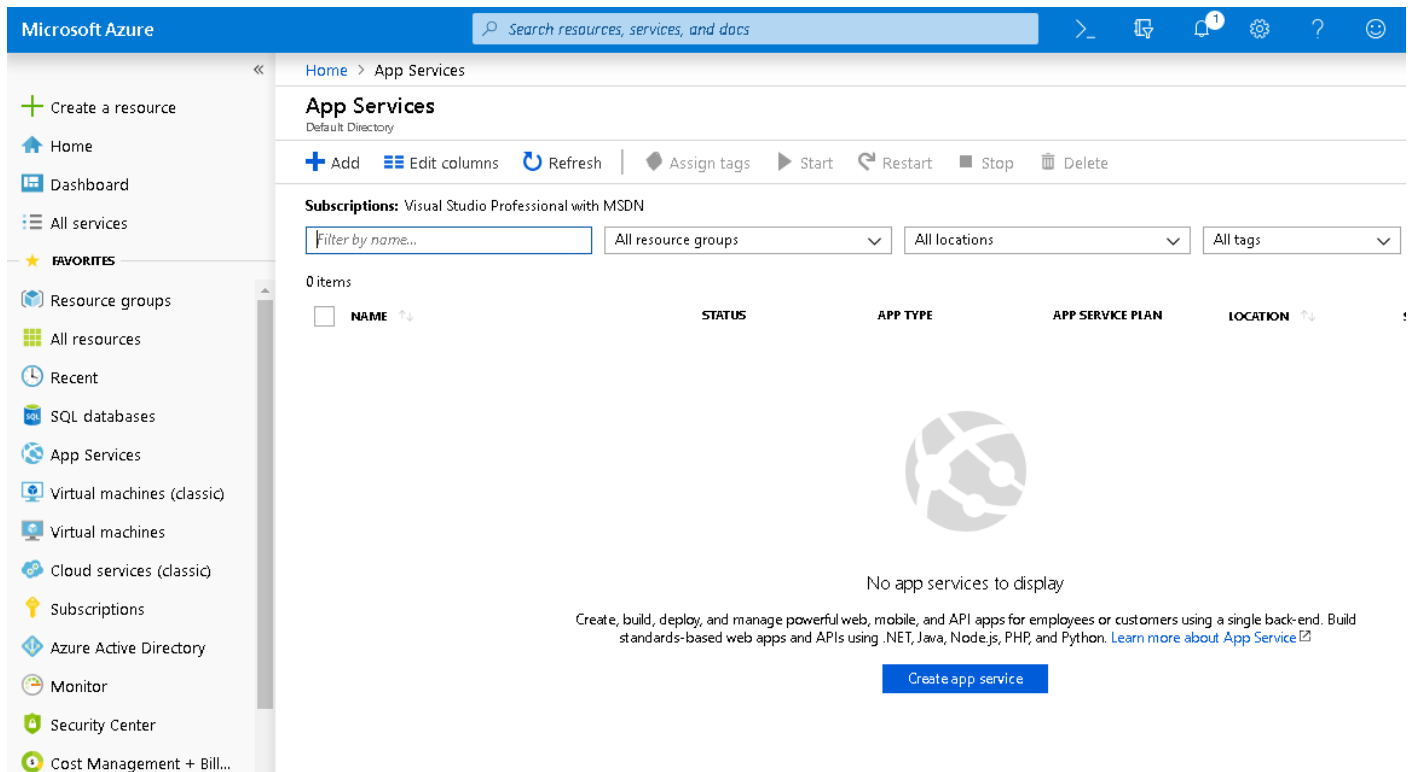
As we have done with creating the Build Artifact in Azure DevOps Build Pipeline which can be deployed. So, we have to see how we can deploy it to web app. But before moving to Release lifecycle. First we will create the 3 Web Apps (**DEV, QA and PROD**) where we can deploy our artifact in different stages.

Before moving to next, as we have defined above that we will require the Azure Subscription. So, open <https://portal.azure.com> and log in with your credentials. Once we will login, we can see the default Dashboard for Azure Portal as follows. Here we have different options available to create the VM, Web App, Storage etc.

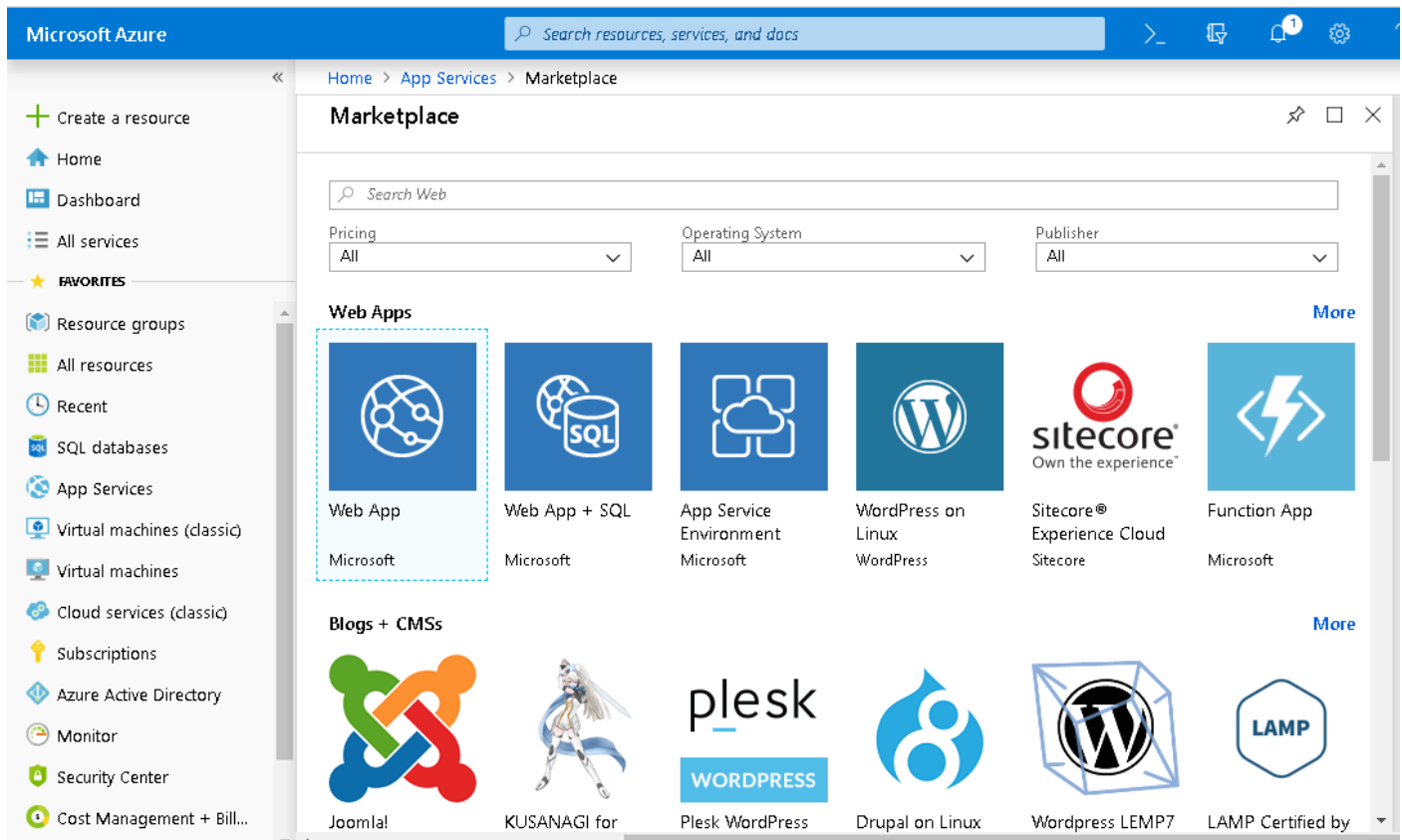
Azure App Service is a fast and simple way to create web apps using Java, Node, PHP or ASP.NET, as well as supporting custom language runtimes using Docker. A continuous integration and continuous deployment (CI/CD) pipeline that pushes each of your changes automatically to Azure app services allows you to deliver value to your customers faster.



We are here to create 3 Web App. So for doing that let click on '**App Services**' from the left panel just after SQL Database [See the above image]. It will open the App Services page from where we can create new App Services. We don't have any App Services in our bucket. So, let create new App Service to click on button '**Create App Service**'.



Next screen will provide us different kinds of templates available for creating the App Services. Here we will create simple Web App. So, click on **Web App**.



From the **Web App** section, we have to click on **Create**.

Web App

Microsoft

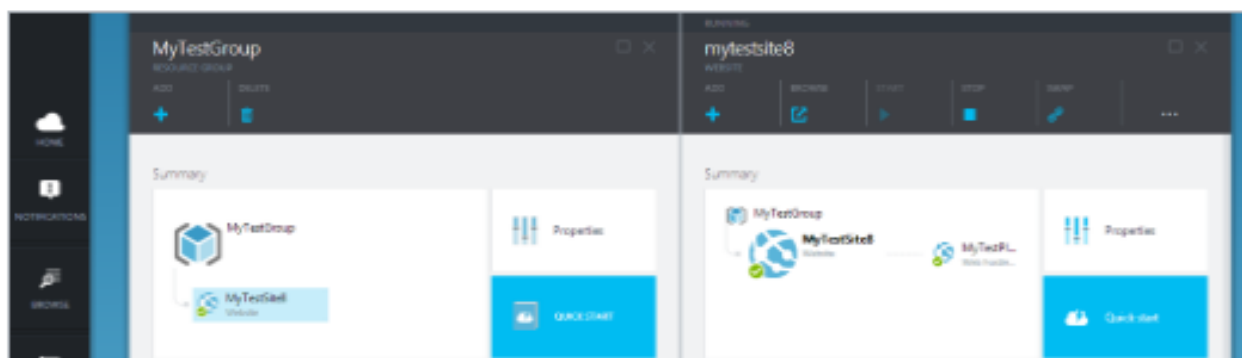


Create and deploy web sites in seconds, as powerful as you need them

Leverage your existing tools to create and deploy applications without the hassle of managing infrastructure. Microsoft Azure Web Sites offers secure and flexible development, deployment, and scaling options for any sized web application. Use frameworks and templates to create web sites in seconds. Choose from source control options like TFS, GitHub, and BitBucket. Use any tool or OS to develop your site with .NET, PHP, Node.js or Python.

- Fastest way to build for the cloud
- Provision and deploy fast
- Secure platform that scales automatically
- Great experience for Visual Studio developers
- Open and flexible for everyone
- Monitor, alert, and auto scale (preview)

♥ Save for later



Create

Next screen will ask a few information before creating the Web App. So, let provide the name of a web app as **'TestDEV-100'**. Name should be common through out Azure Portal. Next, we have to provide the **Azure Subscription**. Next, we have to provide the **Resource Group**, we can reuse if we have already created but for this demonstration we are creating the new.

Next option is **App Service Plan and Location**. It is used to track the what we have used and how much we have to pay as per used resource. Rest of the options just keep the default and click to Create button.

(By: Mukesh Kumar)

Microsoft Azure

Search resources, services,...

Home > Web > Web App > Web App

Web App

Create

* App name
TestDEV-100 ✓
.azurewebsites.net

* Subscription
Visual Studio Professional with MSDN

* Resource Group ⓘ
☒ Create new ☐ Use existing
TestDEV-100 ✓

* OS
Windows Linux

* Publish
Code Docker Image

* App Service plan/Location
[Empty field] >

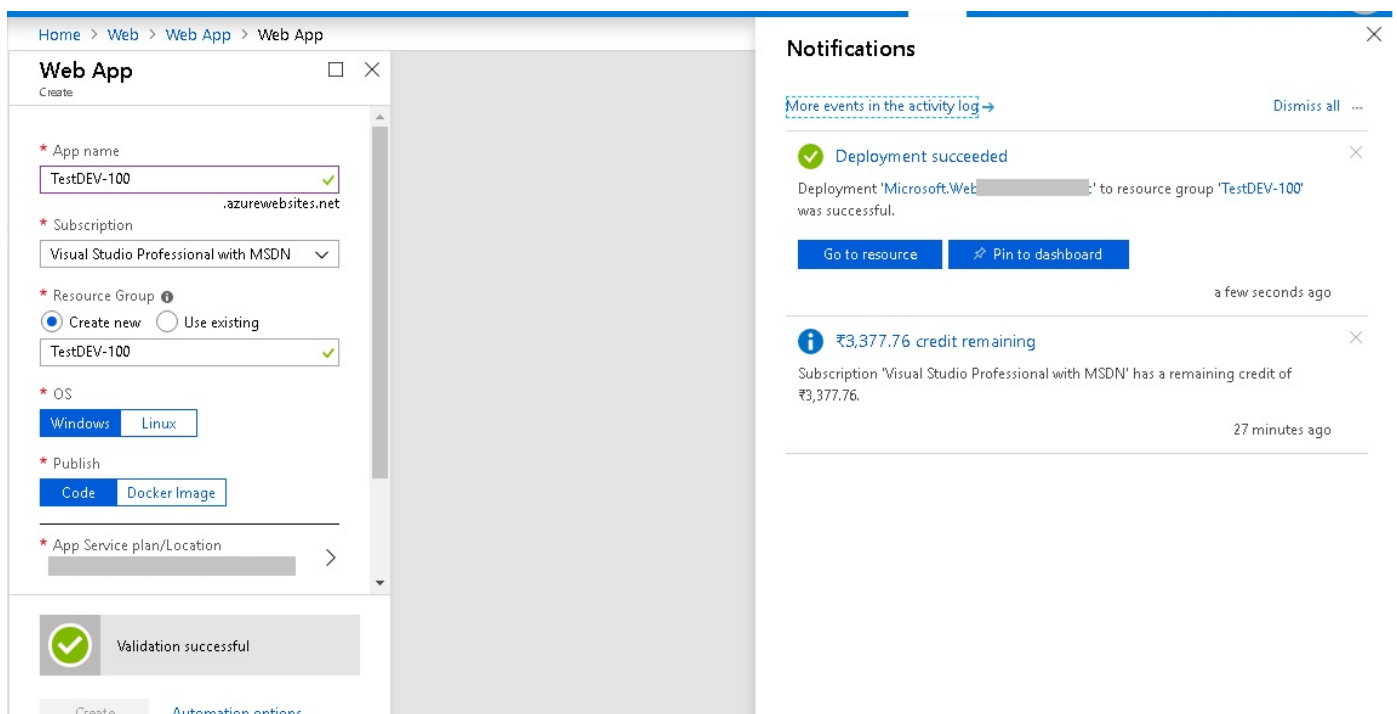
Application Insights
Disabled >

Create Automation options

It will start the validating the information which we have provided. Once validation will succeed. It will start creating the Web App Service. It might take time to create the Web App Service as per our network speed. We can see the progress of creating the Web App Service in the Notification bar.

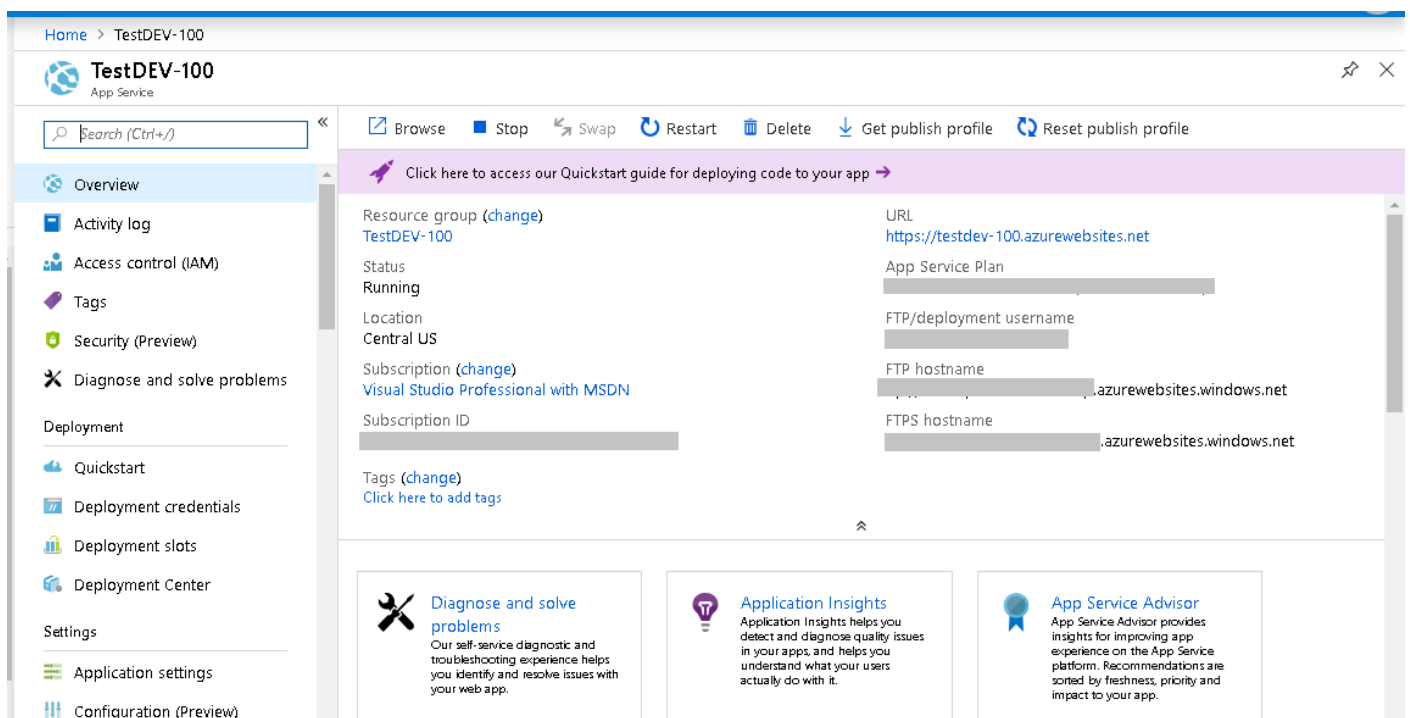
Just click on the **Notification icon** and it will open the windows something like below. Here we will get notify that our resource (**Web App**) has created and we have an option to **Go to Resource**.

(By: Mukesh Kumar)



Let click on **Go to Resource**. It will open the page where we can get all the information for newly created web app (**TestDEV-100**). Here we can see location of the resource where it is created, URL of web app for accessing it and many further options for deployment like **user name and password**.

At the top, we have couple of more option for this Web App, like we can **stop** the Web App if it is running. We can **restart** it , we can **delete** it. If we would like to get he **publish profile** which could be used for publishing the artifact on this web app.



Let move **App Services** and follows the same process as we have followed for creating the **TestDEV-100** web app and create two more Web App like **TestQA-100** and **TestPROD-100**.

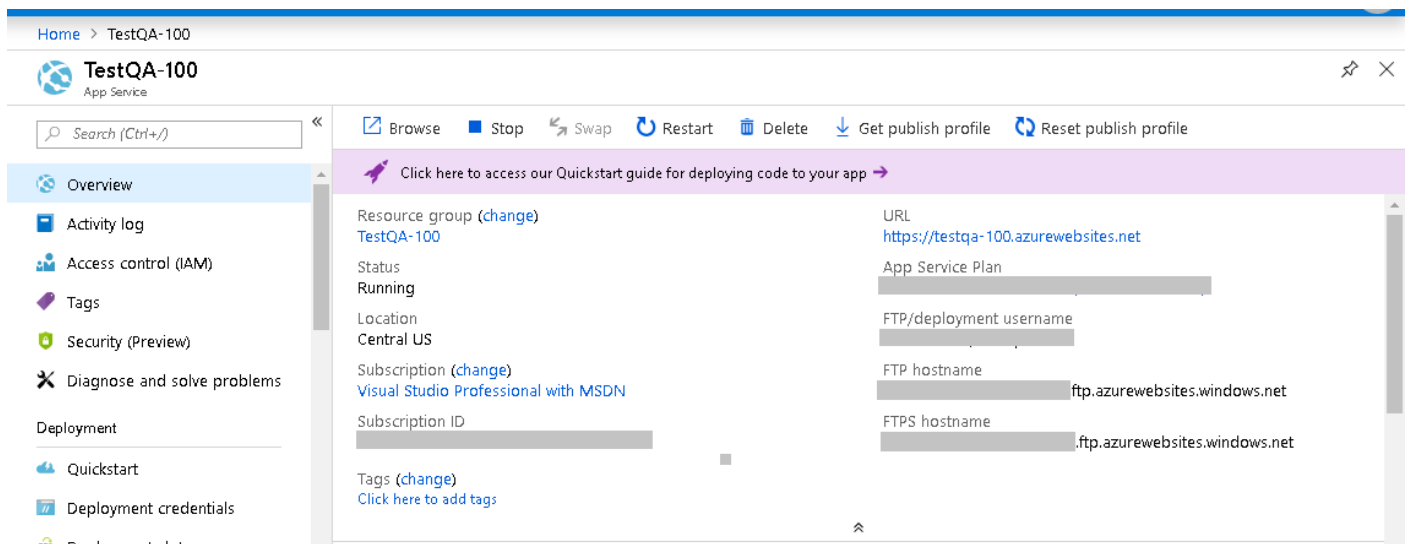
For the QA environment, we will create the **TestQA-100** Web app.

The screenshot shows the Azure Portal interface for creating a new Web App. The left-hand navigation pane includes options like 'Create a resource', 'Home', 'Dashboard', 'All services', and a 'FAVORITES' section with links to Resource groups, All resources, Recent, SQL databases, App Services, Virtual machines (classic), Virtual machines, Cloud services (classic), Subscriptions, Azure Active Directory, Monitor, Security Center, and Cost Management + Bill... The main content area is titled 'Web App' and shows the 'Create' wizard. The wizard includes the following fields and options:

- App name:** TestQA-100 (with a green checkmark icon).
- Subscription:** Visual Studio Professional with MSDN (dropdown menu).
- Resource Group:** Create new (selected) / Use existing. The dropdown shows 'TestQA-100' (with a green checkmark icon).
- OS:** Windows (selected) / Linux.
- Publish:** Code (selected) / Docker Image.
- App Service plan/Location:** A dropdown menu with a right arrow.
- Application Insights:** Disabled (with a right arrow).

At the bottom of the wizard, there is a blue 'Create' button and a link for 'Automation options'.

Once **TestQA-100** will create successfully, we will get the information something like as follows.



The screenshot displays the Azure Portal interface for an App Service named "TestQA-100". The left-hand navigation pane includes sections for "Overview", "Activity log", "Access control (IAM)", "Tags", "Security (Preview)", "Diagnose and solve problems", and "Deployment". The "Overview" section is currently selected. The main content area shows the app's status as "Running" and its location as "Central US". It also provides links to "Browse", "Stop", "Swap", "Restart", "Delete", "Get publish profile", and "Reset publish profile". A purple banner at the top of the main area offers a "Quickstart guide for deploying code to your app". Below this, the "Resource group" is "TestQA-100" and the "URL" is "https://testqa-100.azurewebsites.net". The "App Service Plan" is listed as "App Service Plan". The "FTP/deployment username" is "FTP/deployment username". The "FTP hostname" is "ftp.azurewebsites.windows.net". The "FTPS hostname" is ".ftp.azurewebsites.windows.net".

For the **PRDO** environment, we will create the **TestPROD-100** Web app.

Home > All resources > New > Web App

Web App

Create

* App name
TestPROD-100 ✓
.azurewebsites.net

* Subscription
Visual Studio Professional with MSDN ▼

* Resource Group ⓘ
☒ Create new ☐ Use existing
TestPROD-100 ✓

* OS
Windows Linux

* Publish
Code Docker Image

* App Service plan/Location
[Empty field] >

Application Insights
Disabled >

Create Automation options

Once **TestPROD-100** web app will be created successfully. We will get all information about this resource as follows.

Home > TestPROD-100

TestPROD-100
App Service

Search (Ctrl+ /)

Overview

- Activity log
- Access control (IAM)
- Tags
- Security (Preview)
- Diagnose and solve problems

Deployment

- Quickstart
- Deployment credentials
- Deployment slots

Click here to access our Quickstart guide for deploying code to your app

Resource group (change)
TestPROD-100

Status
Running

Location
Central US

Subscription (change)
Visual Studio Professional with MSDN

Subscription ID

Tags (change)
Click here to add tags

URL
<https://testprod-100.azurewebsites.net>

App Service Plan

FTP/deployment username

FTP hostname
.azurewebsites.windows.net

FTPS hostname
.azurewebsites.windows.net

Now, let move to **Resource page**, here we can find all newly create App Services. We have 3 Web App for **DEV, QA and PROD environment** respectively. Apart from we have one **App Service Plan**, under which all App Service will run.

Home > All resources

All resources
Default Directory

+ Add | Edit columns | Refresh | Assign tags | Delete | Export to CSV | Try preview

Subscriptions: Visual Studio Professional with MSDN

Filter by name... | All resource groups | All types | All locations | All tags | No grouping

4 items | Show hidden types

NAME	TYPE	RESOURCE GROUP	LOCATION	SUBSCRIPTION
[App Service Plan Icon] [Name]	App Service plan	TestDEV-100	Central US	Visual Studio Profession...
[App Service Icon] TestDEV-100	App Service	TestDEV-100	Central US	Visual Studio Profession...
[App Service Icon] TestPROD-100	App Service	TestPROD-100	Central US	Visual Studio Profession...
[App Service Icon] TestQA-100	App Service	TestQA-100	Central US	Visual Studio Profession...

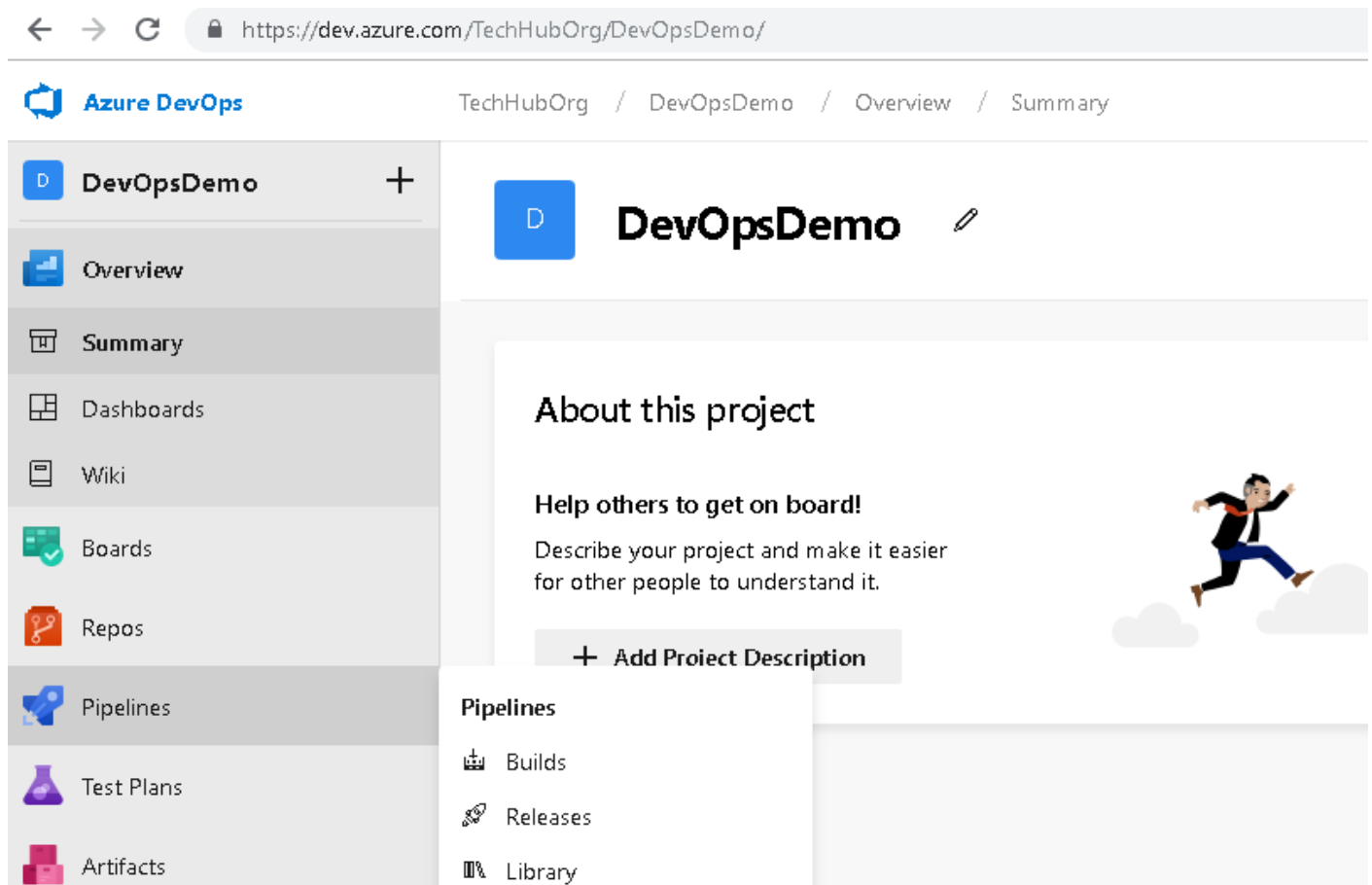
Till yet, we have created the **Azure DevOps Build Pipeline** successfully and able to create the build artifact. Apart from this, we have created 3 Web App for different environments like DEV, QA and PROD. These app services will be used in Azure DevOps Release Pipeline for deployments.

8. Continuous Delivery

Let move to **DevOpsDemo** project in Azure DevOps using following URL.

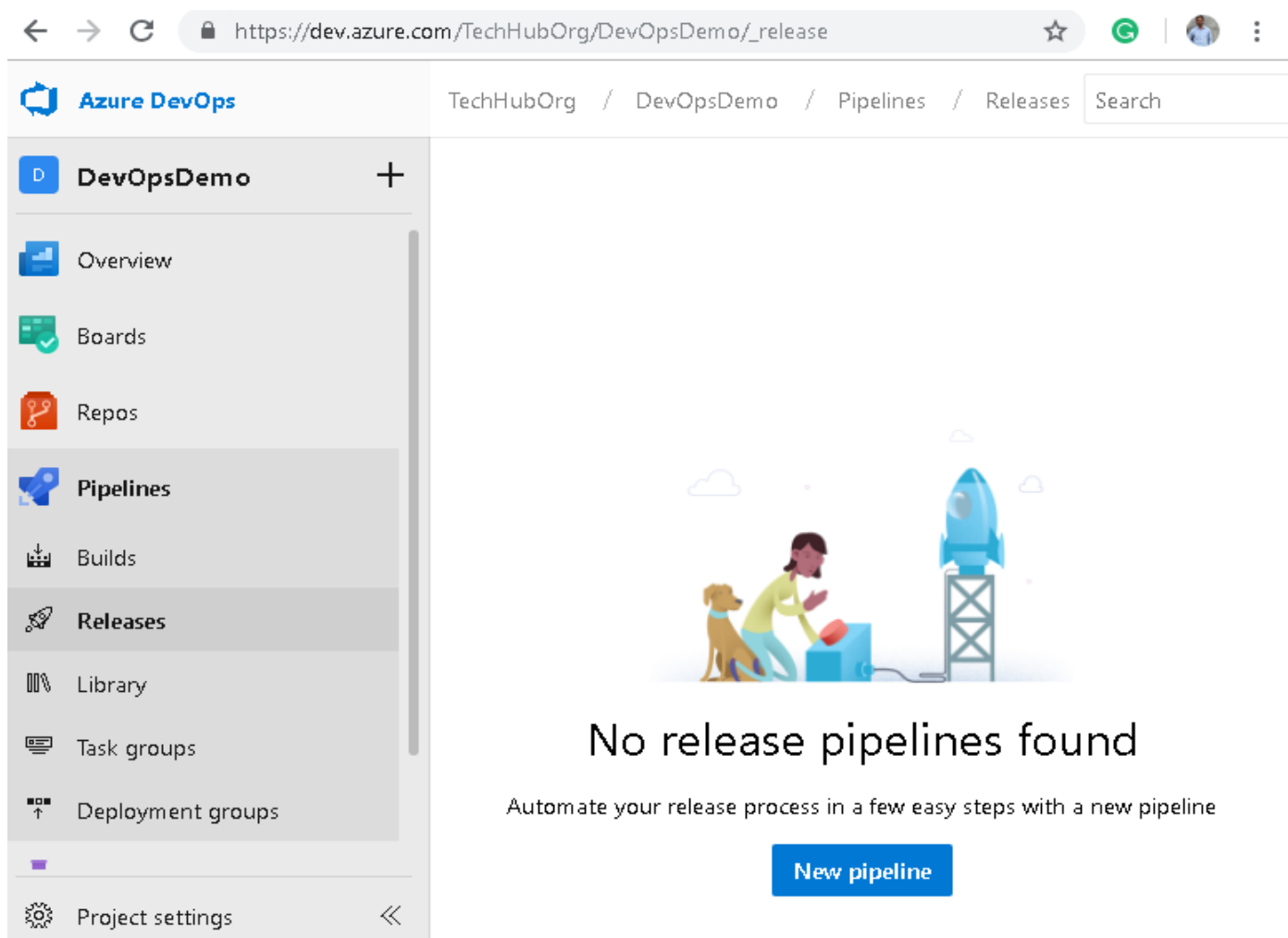
<https://dev.azure.com/TechHubOrg/DevOpsDemo/>.

Click to **Pipeline** and choose **Releases** for creating new **Azure DevOps Release Pipeline**.



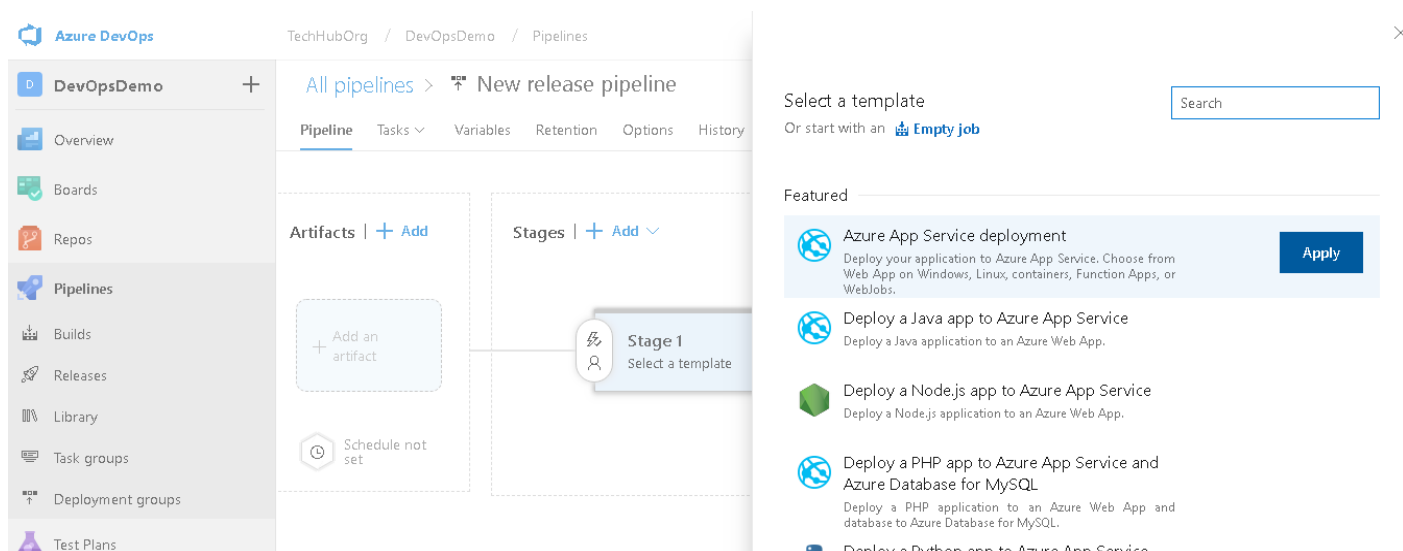
Next screen will show all the release pipeline if we have any. But we don't have created any for now. So, let create new to click on **New Pipeline**.

8.1 Create Dev Stage



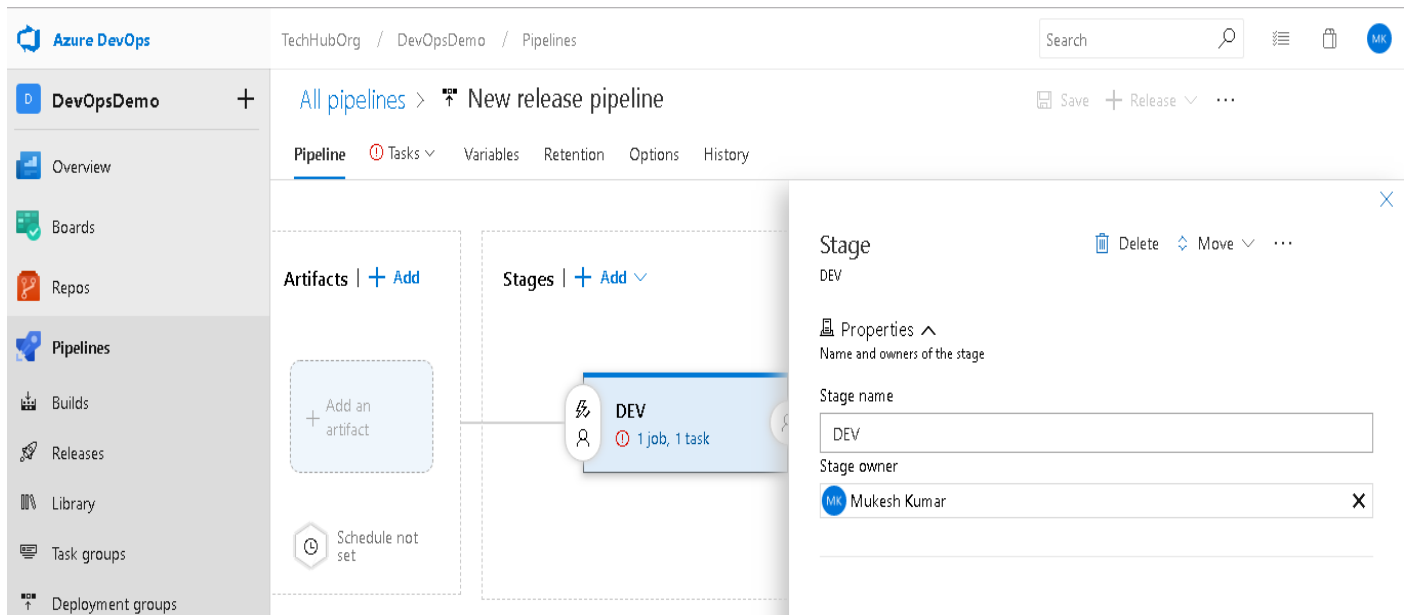
Next screen will ask to configure the **Release Pipeline** like what service we will use, which Artifact will use for deployment, what template will use for deployment etc. So, let select a template as **Azure App Service Deployment** and click to **Apply**. As we have already created 3 App Services above for deployment for different environments like DEV, QA and PRDO. We will choose TestDEV-100 for this DEV stage.

By default the name of the Release Pipeline is '**New Release Pipeline**'. We will change it later.



(By: Mukesh Kumar)

After selecting the template as **Azure App Service Deployment**, it will ask to provide the **name** for this stage. This is our first stage, so let provide the Stage Name as '**DEV**' and click to close (X) button in right top corner (**Only for closing this dialog, don't close the page itself.**).



Once we close the stage dialog then we will get the screen something like below. Here, we have two section for now. First one is the **Artifact** section. Here we will **add the Artifact** which is created in **Azure DevOps Build Pipeline**. Another section is **Stage** section. We could have multiple stage like DEV, QA, PRDO etc or many more as per the requirement.

The screenshot displays the Azure DevOps web interface for creating a new release pipeline. On the left, the navigation pane shows the 'DevOpsDemo' project and the 'Pipelines' section. The main content area is titled 'All pipelines > New release pipeline'. Below the title, there are tabs for 'Pipeline', 'Tasks', 'Variables', 'Retention', 'Options', and 'History'. The 'Pipeline' tab is active, showing a visual representation of the pipeline. It includes an 'Artifacts' section with an 'Add an artifact' button and a 'Stages' section with an 'Add' button. A stage named 'DEV' is already added, containing 1 job and 1 task. A warning icon indicates that the 'Schedule not set'.

Let click on the **Add an Artifact**. It will open a popup a dialog window in right side as follows. Here we will configure the Artifact. So, first select **the Source Type as Build**, because we will take the Artifact for deployment from the Build Pipeline. Next select the **name of the project as DevOpsDemo**. Next we have to select the **Source (Build Pipeline)**. As we have Build Pipeline '**DevOpsDemo-CI**', so select it. Rest of the options keep the default and click to **Add**.

TechnicalHubOrg / DevOpsDemo / Pipelines

All pipelines > New release pipeline

Pipeline Tasks Variables Retention Options History

Artifacts | + Add

Stages | + Add

DEV 1 job, 1 task

Schedule not set

Add an artifact

Source type

Build Azure Repos ... GitHub TFVC

4 more artifact types

Project * DevOpsDemo

Source (build pipeline) * DevOpsDemo-CI

Default version * Specify at the time of release creation

Source alias * _DevOpsDemo-CI

The artifacts published by each version will be available for deployment in release pipelines. Select the version when you create a release. For automatically triggered releases, the latest version will be chosen.

Add

Once we click to **Add**, it will add the '**_DevOpsDemo-CI**' artifact. Now, we have available the **Artifact** which can be deployed to any environment. So, let enable the **Continuous Deployment**. Click on the **Continuous Deployment Trigger** as we can see in below image. It will open the Continuous Deployment Trigger dialog window in the right. Let **Enabled the Continuous Deployment Trigger** which will create a new release every time a new build is available. Let keep all other options are default.

TechnicalHubOrg / DevOpsDemo / Pipelines

All pipelines > New release pipeline

Pipeline Tasks Variables Retention Options History

Artifacts | + Add

Stages | + Add

DEV 1 job, 1 task

Schedule not set

Continuous deployment trigger

_DevOpsDemo-CI

Continuous deployment trigger

Build: _DevOpsDemo-CI

Enabled

Creates a release every time a new build is available.

Build branch filters

No filters added.

+ Add

Pull request trigger

Build: _DevOpsDemo-CI

Disabled

Enabling this will create a release every time a selected artifact is available as part of a pull request workflow

Let close the **Continuous Deployment Trigger** dialog window. So we have done with setup of Continuous Deployment Trigger, if new build will be there then it will deploy to respective environment.

Now let move to Stages section and click to '**1 Job, 1 Task**' in **DEV** stage for configuring the **DEV** environment so that if new build will be there it auto deploys on DEV environment.

Each stage in Release pipeline has its own configuration. We have to move on **Tasks** tab, here we can provide or modify information like **stage name**, **Azure Subscription** etc. The most important configuration is **App Service name**. We will define the **TestDEV-100 web app in App Service Name** so that after building the application, it can deploy on DEV server first.

The screenshot shows the 'New release pipeline' configuration page in Azure DevOps. The left sidebar shows the 'DEV' stage with a task 'Deploy Azure App Service'. The main panel is the 'Tasks' tab, showing configuration for the 'DEV' stage. The 'Stage name' is 'DEV'. The 'Parameters' section shows 'Azure subscription' set to 'Visual Studio Professional with MSDN' and 'App type' set to 'Web App on Windows'. The 'App service name' is 'TestDEV-100'.

We don't have to do in **Variables** tab section, let it be default value and move to **Retention** tab. Here we can do the setting for **DEV** environment like how many days we would like to retain the release information and how many releases information, we want to keep.

The screenshot shows the 'New release pipeline' configuration page in Azure DevOps, specifically the 'Retention' tab. The left sidebar shows the 'DEV' stage with a note 'Keep for 30 days, 3 good releases and keep artifacts'. The main panel is the 'Retention' tab, showing settings for the 'DEV' stage. The 'Days to retain a release' is set to 30, and the 'Minimum releases to keep' is set to 3. The 'Retain associated artifacts' checkbox is checked. A link 'View or manage retention policy defaults' is visible at the bottom.

Now let move to **Options** tab, here we can provide the information about this Release like **description of the Release** and what will be **format of Release Name**. By default, it will create default format for release name but it can be modify.

TechHubOrg / DevOpsDemo / Pipelines

Search

All pipelines > New release pipeline

Save + Release View release

Pipeline Tasks Variables Retention **Options** History

General

Integrations

Description ⓘ

Release name format ⓘ

Release-\${rev:r}

Next tab is **History** tab. Here for now, we will not find anything. It will be empty because we haven't run any release cycle for now.

After configuring all tabs as per the requirements, its time to **save** the information. So, click on **Save**.

TechHubOrg / DevOpsDemo / Pipelines

Search

All pipelines > New release pipeline

Save + Release View releases

Pipeline Tasks Variables Retention Options **History**

Changed By	Change Type	Changed Date	Comment
------------	-------------	--------------	---------

After creating and saving the release pipeline, we can see our configured Azure DevOps Release pipeline as follows.

We can see Release pipeline anytime from **Pipeline>Releases**. Here we can see all the available Release pipeline. If we would like to **Edit** and then we can **edit** also.

Azure DevOps

TechHubOrg / DevOpsDemo / Pipelines / Releases

Search

DevOpsDemo +

Overview

Boards

Repos

Search all pipelines

+ New

New release pipeline

No deployments found

New release pipeline

Releases Deployments Analytics*

Edit Create a release

All releases

Let click on the **Edit** button (As showing in above image) for **editing the Release Pipeline**. Once we click to edit the release pipeline, it will open the same page from where we can define the artifact and other stages information.

(By: Mukesh Kumar)

So, let first modify the name of the Release Pipeline. Click on the **'New release pipeline'** text just after All pipelines and rename it with **'Test100-Release'**. So, we have changed the name of the **Azure DevOps Release Pipeline** name.

The screenshot shows the Azure DevOps web interface. On the left, the 'DevOpsDemo' project is selected, and the 'Pipelines' section is active. The main view shows the 'Test100-Release' pipeline. Under the 'Artifacts' section, there is an artifact named '_DevOpsDemo-CI'. Under the 'Stages' section, there is a stage named 'DEV' which contains 1 job and 1 task. The 'DEV' stage is highlighted with a blue bar. A 'Schedule not set' icon is visible at the bottom left of the pipeline view.

8.2 Create QA Stage

We have already setup the **DEV** environment. If new build will be available it will deploy to **Dev (TestDEV-100 Web App)** environment. But let add one more stage for **QA (TestQA-100)**.

For **adding new stage**, we can directly click to **(+ Add)** icon just after Stages or mouse hover just below to DEV stage, here we will get the icon (+ Add). Just click on it.

The screenshot shows the Azure DevOps interface for the 'Test100-Release' pipeline. The left sidebar displays the 'DevOpsDemo' project with a '+' icon. The main area shows the pipeline structure with 'Artifacts' and 'Stages'. The 'Artifacts' section contains '_DevOpsDemo-CI' with a 'Schedule not set' icon. The 'Stages' section contains a 'DEV' stage with '1 job, 1 task'. Below the 'DEV' stage are '+ Add' and 'Add a stage' buttons.

So, it has added one more stage in **Azure DevOps Release Pipeline**. Now, it is asking for selecting the template. We have to follow the same process which we have already followed for **DEV (TestDEV-100)** environment setup. So, let select the template as '**Azure App Service Deployment**' and click to **Apply** button.

The screenshot shows the Azure DevOps interface for the 'Test100-Release' pipeline. The left sidebar displays the 'DevOpsDemo' project with a '+' icon. The main area shows the pipeline structure with 'Artifacts' and 'Stages'. The 'Artifacts' section contains '_DevOpsDemo-CI' with a 'Schedule not set' icon. The 'Stages' section contains a 'DEV' stage with '1 job, 1 task'. Below the 'DEV' stage are '+ Add' and 'Add a stage' buttons. A 'Select Template' dialog box is open on the right, showing a search bar and a list of featured templates. The 'Azure App Service deployment' template is highlighted with an 'Apply' button.

Close the '**Select Template**' dialog window and we will see that one more stage (**Stage 1 – in above image**) has added just after DEV stage. Now, click on the **Stage 1** and change the stage name to **QA** as showing in following image.

The screenshot shows the Azure DevOps Pipelines interface for a pipeline named 'Test100-Release'. The 'DEV' stage is selected, and the 'QA' stage is highlighted. The 'QA' stage has a red status icon and '1 job, 1 task'. A 'Stage' dialog box is open on the right, showing the 'QA' stage name and owner 'Mukesh Kumar'.

Now close the the Stage name dialog using close (X) icon in the right top corner. Its time to configure the **QA (TestQA-100)** environment. So, click on the '1 Job, 1 Task' in QA stage.

The screenshot shows the Azure DevOps Pipelines interface for a pipeline named 'Test100-Release'. The 'QA' stage is selected, and the '1 Job, 1 Task' link is highlighted. A 'View stage tasks' dialog box is open, showing the 'QA' stage name and owner 'Mukesh Kumar'.

Here, we have to configure the **QA stage**. First tab is Tasks tab. Here we will do the configuration as we have already done for DEV stage. So, let provide the Azure Subscription and most important the App Service Name as '**TestQA-100**'.

For the rest of the tabs like **Variables**, **Retention**, **Options** and **History**, we are not going to do anything. We will keep the default setting for these tabs. But we can modify it as per our requirements. Now click to **Save**.

All pipelines > Test100-Release

Save + Release View releases

Pipeline Tasks Variables Retention Options History

QA
Deployment process

Run on agent
Run on agent

Deploy Azure App Service
Azure App Service Deploy

Stage name
QA

Parameters | Unlink all

Azure subscription * | Manage

Visual Studio Professional with MSDN

Scoped to subscription 'Visual Studio Professional with MSDN'

App type
Web App on Windows

App service name *
TestQA-100

This time, we have two stages **DEV** and **QA**. What we have configured till yet that if new build be available then it automatically will deploy on DEV stage. But what about QA. How we will deploy on QA.

Actually for build deployment, it will deploy automatically to all stages one by one once after build is available. But here we would like to some confirmation before deploying on the QA stage.

So, let configure the '**Pre-Deployment Conditions**'. Click on the '**Pre-Deployment Conditions**' from the **QA stage** as showing in below image.

TechHubOrg / DevOpsDemo / Pipelines

All pipelines > Test100-Release

Save + Release

Pipeline Tasks Variables Retention Options History

Artifacts | + Add

_DevOpsDemo-CI

Schedule not set

Stages | + Add

Pre-deployment conditions

DEV
1 job, 1 task

QA
1 job, 1 task

Enable the Pre-Deployment Approvals and provide the **name of the approver** in Approvers section. For this demonstration, we are providing the current user name '**Mukesh Kumar**'. So, now after DEV deployment, the Artifact will not auto deploy to QA. It will first wait for the approval by Mukesh Kumar.

(By: Mukesh Kumar)

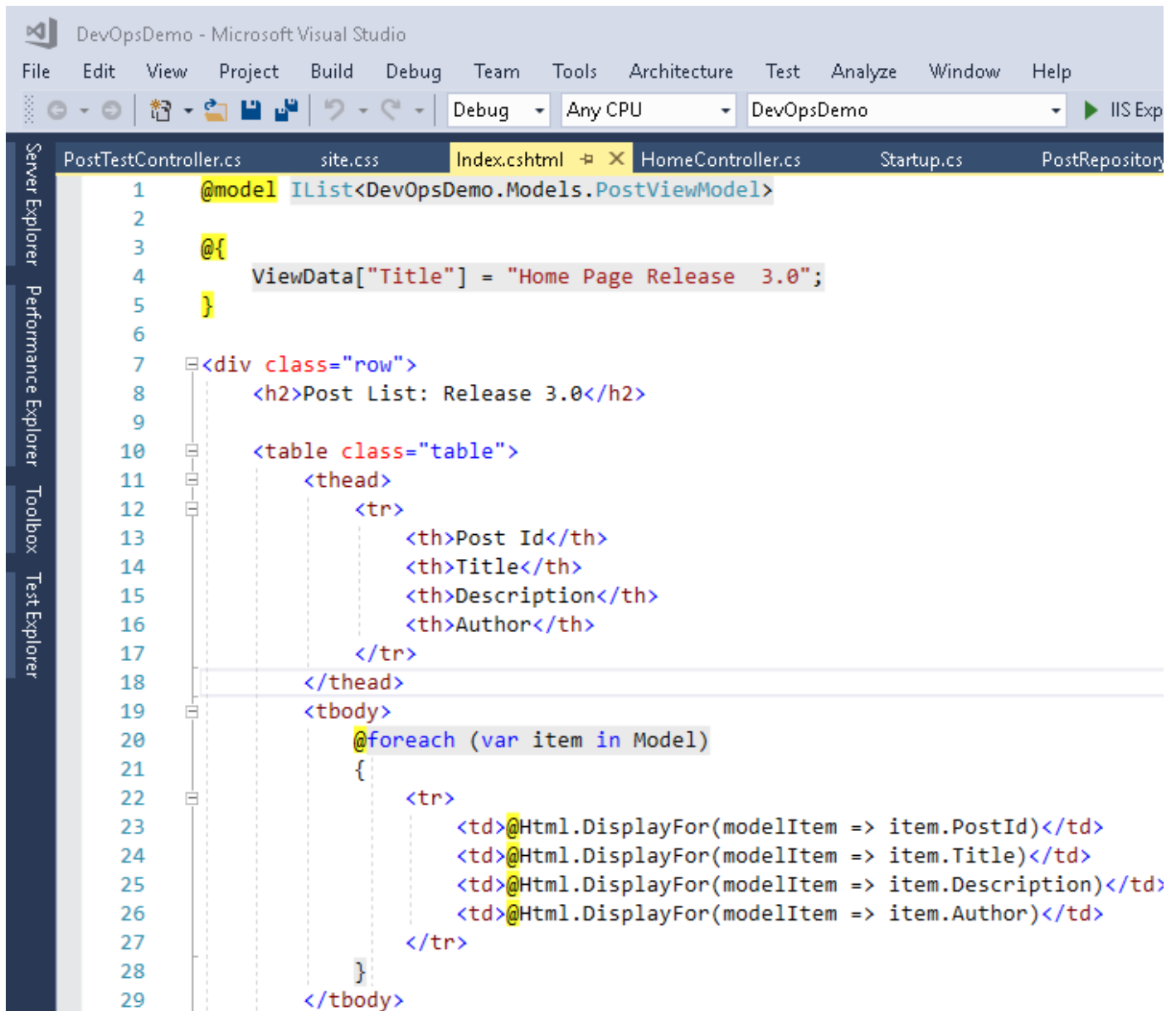
The screenshot displays the Azure DevOps Pipelines interface for a pipeline named 'Test100-Release'. The left pane shows the pipeline graph with two stages: 'DEV' and 'QA'. The 'QA' stage is highlighted with a blue arrow. The right pane shows the configuration for the 'QA' stage. The 'Triggers' section is expanded, showing 'Pre-deployment approvals' which are 'Enabled'. The 'Approvers' section lists 'Mukesh Kumar' as an approver. The 'Timeout' is set to '30' days. The 'Approval policies' section has two options: 'The user requesting a release or deployment should not approve it' (unchecked) and 'Skip approval if the same approver approved the previous stage' (unchecked). The 'Gates' section is 'Disabled'. The 'Deployment queue settings' section is also visible.

We have done with **QA environment setup**. Let click to **SAVE** for saving the **Azure DevOps Release Pipeline with new stage as QA**.

So, what we have achieved so far. We have configured the Azure DevOps Build Pipeline as well as Release pipeline. In Release pipeline, we have created two stages for two different environments like DEV and QA. So, if new build (Artifact) will be available then it first will deploy to DEV (TestDEV-100) environment and will ask for Approval before deploying it to QA (TestQA-100).

Now, its time to see the real time deployment. How build deploy on the different stages. So, let open the Visual Studio 2017 or higher version. And opent the project '**DevOpsDemo**', which we have created in previous section.

Open the **Index.cshtml** file in main project and replace the text for title from '**Home Page**' to '**Home Page Release 3.0**'. And similarly change the text '**Post List**' with '**Post List Release 3.0**'. Here we will only change the version number and see the output after auto deployment.



```

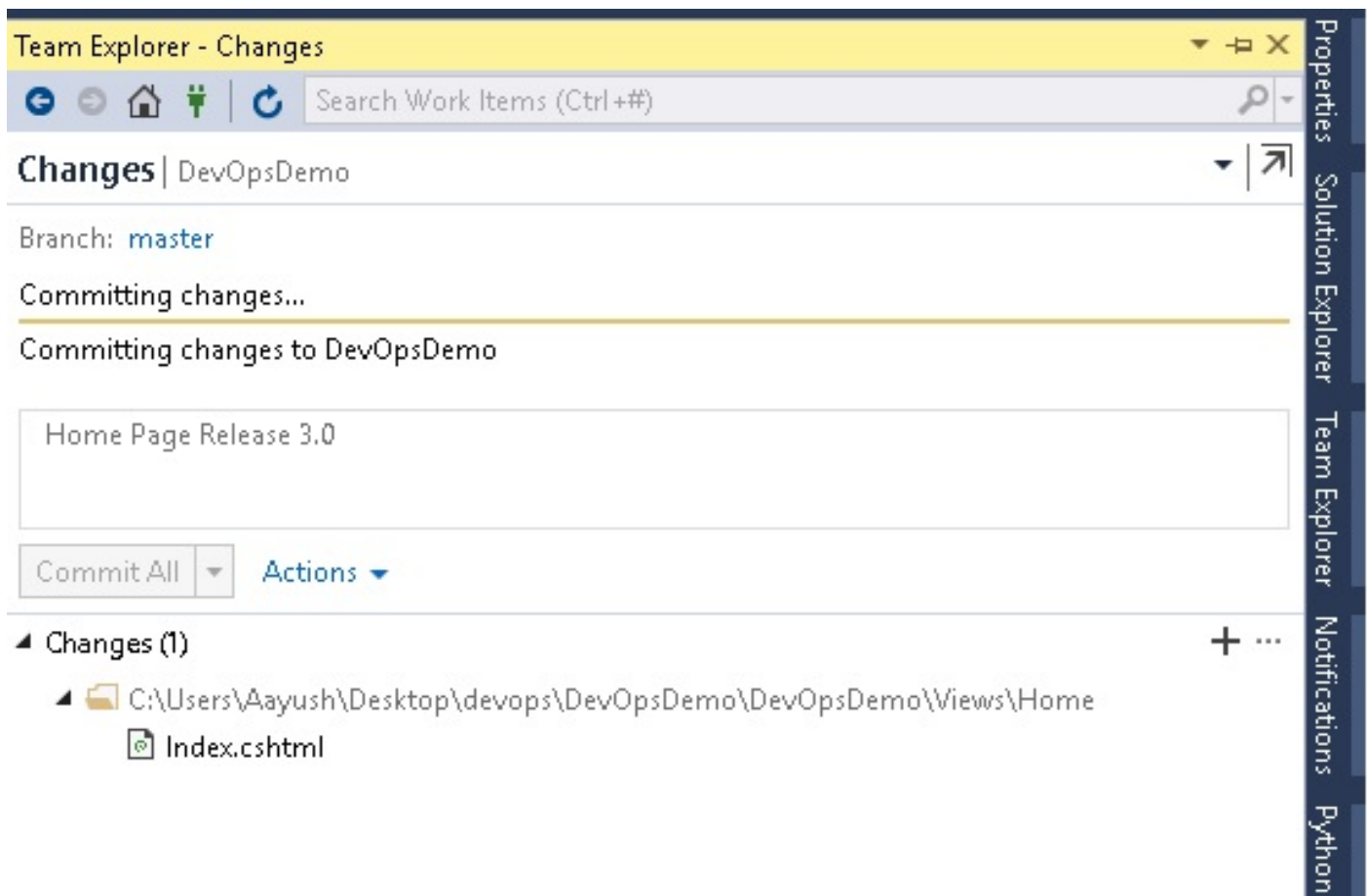
DevOpsDemo - Microsoft Visual Studio
File Edit View Project Build Debug Team Tools Architecture Test Analyze Window Help
Debug Any CPU DevOpsDemo IIS Exp

PostTestController.cs site.css Index.cshtml HomeController.cs Startup.cs PostRepository

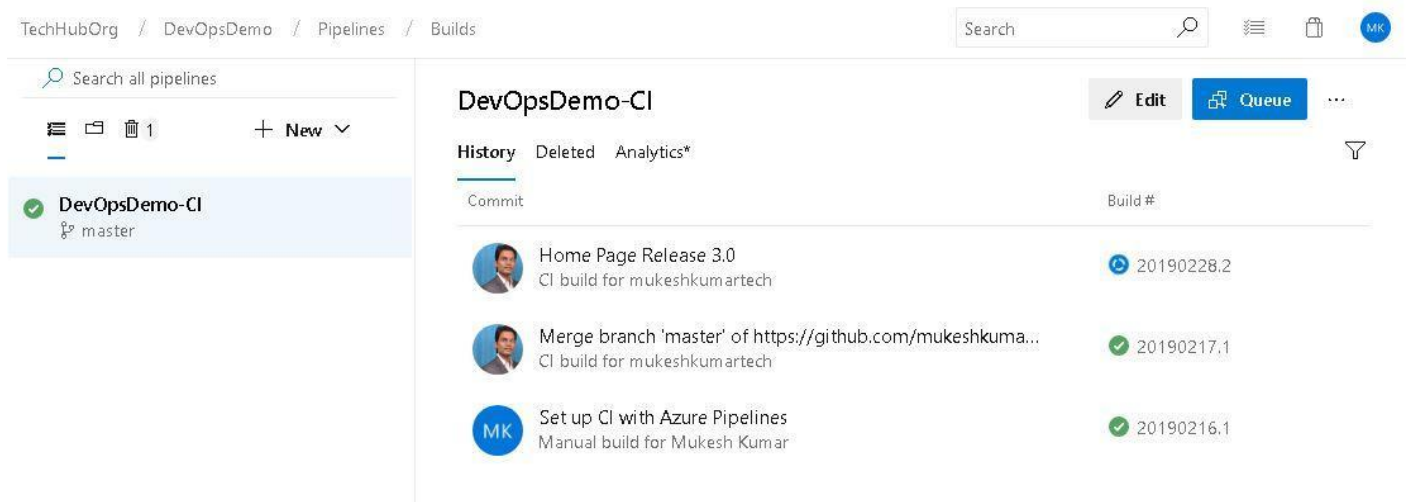
1 @model IList<DevOpsDemo.Models.PostViewModel>
2
3 @{
4     ViewData["Title"] = "Home Page Release 3.0";
5 }
6
7 <div class="row">
8     <h2>Post List: Release 3.0</h2>
9
10    <table class="table">
11        <thead>
12            <tr>
13                <th>Post Id</th>
14                <th>Title</th>
15                <th>Description</th>
16                <th>Author</th>
17            </tr>
18        </thead>
19        <tbody>
20            @foreach (var item in Model)
21            {
22                <tr>
23                    <td>@Html.DisplayFor(modelItem => item.PostId)</td>
24                    <td>@Html.DisplayFor(modelItem => item.Title)</td>
25                    <td>@Html.DisplayFor(modelItem => item.Description)</td>
26                    <td>@Html.DisplayFor(modelItem => item.Author)</td>
27                </tr>
28            }
29        </tbody>

```

Its time to check in the code in **GitHub** repository. So, Go to **Team Explorer > Changes > provide the comment as 'Home Page Release 3.0'** and click to **'commit all and sync'**.



When we open the **Build Pipeline**, in the **history** tab, we can see that a new **Continuous Integration** build is started with same comment which we have provided at the time of check in the code as '**Home Page Release 3.0**'.



Click on new started build and we will see that all the Jobs are executing one by one as follow.

✓ Initialize job · succeeded	<1s
✓ Checkout · succeeded	8s
✓ Restore · succeeded	1m 1s
⚙ Build	20s

```

*****
Starting: Build
*****
Task       : .NET Core
Description: Build, test, package, or publish a dotnet application, or run a custom dotnet command. For package commands, supports NuGet.org and
authenticated feeds like Package Management and MyGet.
Version    : 2.147.2
Author     : Microsoft Corporation
Help       : [More Information](https://go.microsoft.com/fwlink/?linkid=832194)
*****
[command]/usr/bin/dotnet build /home/vsts/work/1/s/DevOpsDemo.Test/DevOpsDemo.Test.csproj --configuration Release
Microsoft (R) Build Engine version 15.9.20+e88f5fadfbc for .NET Core

```

Once all jobs execute successfully then we can see the a 'succeeded' message in green color for each Job.

Logs	Summary	Tests
Agent job 1 Job		
Pool: Hosted Ubuntu 1604 · Agent: Hosted Agent		Started: 2/28/2019, 10:21:55 PM
		... 2m 28s

✓ Initialize Agent · succeeded	<1s
✓ Prepare job · succeeded	<1s
✓ Initialize job · succeeded	1s
✓ Checkout · succeeded	10s
✓ Restore · succeeded	1m 6s
✓ Build · succeeded	33s
✓ Test · succeeded	28s
✓ Publish · succeeded	2s
✓ Publish Artifact · succeeded	3s
✓ Post-job: Checkout · succeeded	<1s
✓ Finalize Job · succeeded	<1s

Let move to **Azure DevOps Release Pipeline** and open **Test100-Release**. Here we can see that one release is initiated as 'Release-1' and DEV is processing this.

The screenshot shows the 'Test100-Release' page in Azure DevOps. The breadcrumb navigation is 'TechHubOrg / DevOpsDemo / Pipelines / Releases'. On the left, there's a sidebar with 'Search all pipelines' and a list of releases, including 'Test100-Release' with a 'DEV' tag. The main area shows the 'Releases' tab for 'Test100-Release'. It includes a table with columns 'Releases', 'Created', and 'Stages'. The first row shows 'Release-1' with a build icon, version '20190228.2', source 'master', and a creation time of '2019-02-28 22:24'. Below the table, there are buttons for 'DEV' (selected) and 'QA'.

Now, click on the **Release-1** for detailed information. As per the following image, we can see that we have artifact **(build)** is available and this is performing the **Continuous Deployment on DEV (TestDEV-100)** environment. Yet it is in progress.

The screenshot shows the detailed view of 'Release-1' in the 'Test100-Release' pipeline. The breadcrumb navigation is 'TechHubOrg / DevOpsDemo / Pipelines'. The top navigation bar includes 'Pipeline', 'Variables', 'History', and buttons for '+ Deploy', 'Cancel', 'Refresh', 'Release (old view)', and 'Edit release'. The main content area is divided into two sections: 'Release' and 'Stages'. The 'Release' section shows 'Continuous deployment for Mukesh Kumar' on '2/28/2019, 10:24 PM' and lists an artifact '_DevOpsDemo-CI' with version '20190228.2' from the 'master' branch. The 'Stages' section shows a deployment pipeline with two stages: 'DEV' (labeled 'In progress' with '1/1 tasks') and 'QA' (labeled 'Not deployed').

After few minutes, we will see that **Continuous Deployment** has done on **DEV**. But for **QA**, it is **pending for approval**. Once a current user (**Mukesh Kumar**), as we have defined earlier, will approve then Continuous Deployment will start on QA.

So, let check first what has been deployed on **DEV** environment. So, open the **TestDEV-100 App service**. Here we can find the URL for accessing the TestDEV-100 app service and it is <https://testdev-100.azurewebsites.net>. Let open this in **browser** and here we go.

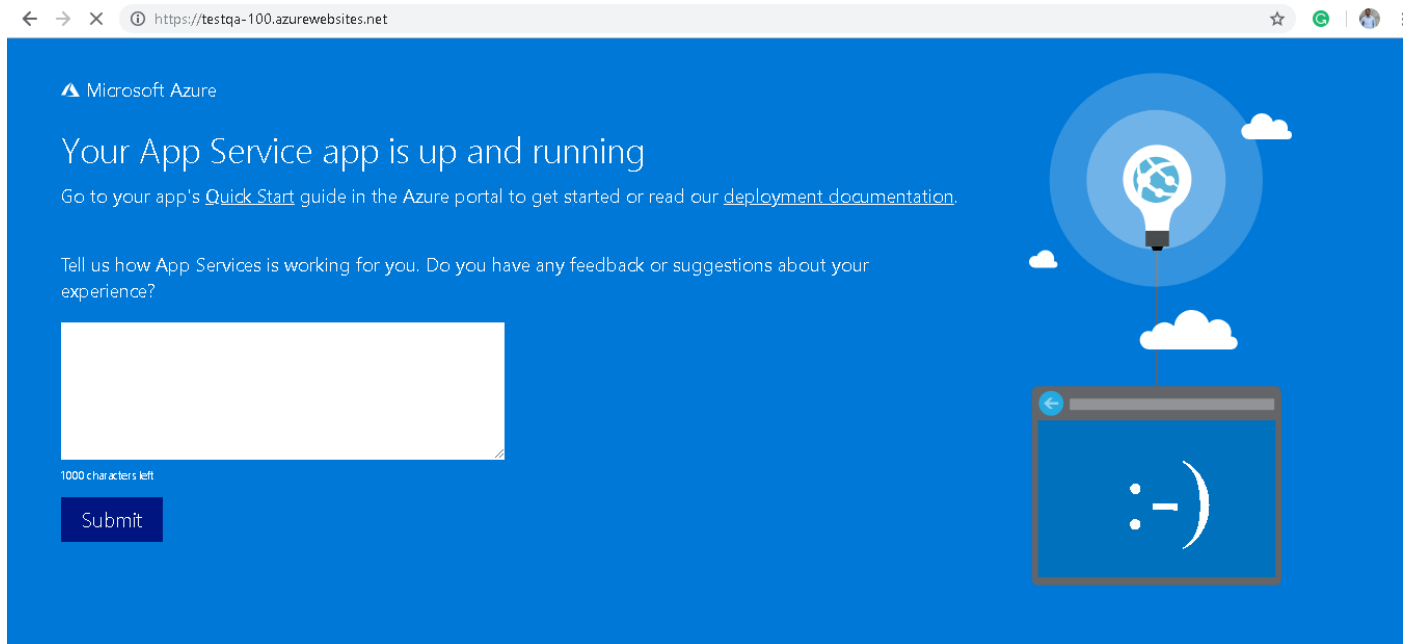
Great, as we can see with below image. It has been successfully deployed the **Release 3.0 on DEV (TestDEV-100)** environment. We can see both title and post list text are showing **Release 3.0**.

Post List: Release 3.0

Post Id	Title	Description
101	DevOps Demo Title 1	DevOps Demo Description 1
102	DevOps Demo Title 2	DevOps Demo Description 2
103	DevOps Demo Title 3	DevOps Demo Description 3

Now, let move to **QA** environment and open **TestQA-100** app service using URL <https://testqa-100.azurewebsites.net>. And we will see that nothing has deployed yet on QA environment. And its showing the default page for Azure App Service.

(By: Mukesh Kumar)



We will also be informed via email that approval is pending for QA deployment.

[Pre-deployment approval pending] Test100-Release > Release-1 : QA

Deployment to QA is waiting for your approval

🏰 DevOpsDemo-CI / 20190228.2 🐙 master

Requested for Mukesh Kumar

[View approval](#)

Summary

Release description	Triggered by DevOpsDemo-CI 20190228.2.
Deployment trigger	automated: after successful deployment to DEV
Requested for	Mukesh Kumar
Attempt	1

(By: Mukesh Kumar)

Now, let current user (**Mukesh Kumar**) to approve the pending approval for QA environment. When we click on **Approve** button from QA stage, it will open the dialog where we can put some comment before approving it. So, let put the comment as '**Deploy Home Page 3.0 to QA**' and click to **Approve** button.

The screenshot shows the Azure DevOps Release view for 'Test100-Release > Release-1'. The left pane shows the release pipeline with the DEV stage 'Succeeded' and the QA stage 'Pending'. The right pane shows the 'QA' stage details with 'Pre-deployment conditions' set to 'Pending approval'. The 'Approvers' tab is active, showing 'Mukesh Kumar' as the approver. A comment box contains the text 'Deploy Home Page 3.0 to QA.' and the 'Approve' button is highlighted.

As pending approval will approve, **Continuous Deployment** has initiated and will start deploying the Artifact (Build) on **QA (TestQA-100)** environment.

The screenshot shows the Azure DevOps Release view for 'Test100-Release > Release-1'. The left pane shows the 'Release' section with 'Continuous deployment...' for 'Mukesh Kumar' on '2/28/2019, 10:24 PM'. The right pane shows the 'Stages' section with the DEV stage 'Succeeded' and the QA stage 'In progress...'. The QA stage details show 'Deploy Azure App Service' with '4/4 tasks' completed and a timer at '00:03'.

After few minutes, the deployment will done on QA and we can see **succeeded** message on **QA stage**.

Test100-Release > Release-1

Pipeline Variables History + Deploy Cancel Refresh Release (old view) Edit release ...

Release

Continuous deployment...
for Mukesh Kumar
2/28/2019, 10:24 PM

Artifacts

_DevOpsDemo-CI
20190228.2
master

Stages

DEV

Succeeded

on 2/28/2019, 10:25 PM

QA

Succeeded

on 2/28/2019, 10:49 PM

Now, once more let open the **QA (TestQA-100 App Service)** URL as <https://testqa-100.azurewebsites.net>. And here we go, we have **successfully deployed to QA of release 3.0**.

Home Page Release 3.0 - DevOps x +

https://testqa-100.azurewebsites.net

Use this space to summarize your privacy and cookie use policy. Lea

Post List: Release 3.0

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

So far, so good. We have seen, committing the code into the GitHub repository, building the code in Build pipeline, executing the unit test cases in Build pipeline, deployment on the DEV and QA environment.



8.3 Create Prod Stage

Now, its time to add one more stage as **PROD**. So, open the **Pipeline > Release > Test100-Release**. From here just click on **Edit** button for editing the **Azure DevOps Release Pipeline**.

The screenshot shows the Azure DevOps interface for the 'Test100-Release' pipeline. The left sidebar contains the navigation menu with 'Releases' selected. The main area displays the 'Test100-Release' pipeline with a table of releases. The 'QA' stage is highlighted.

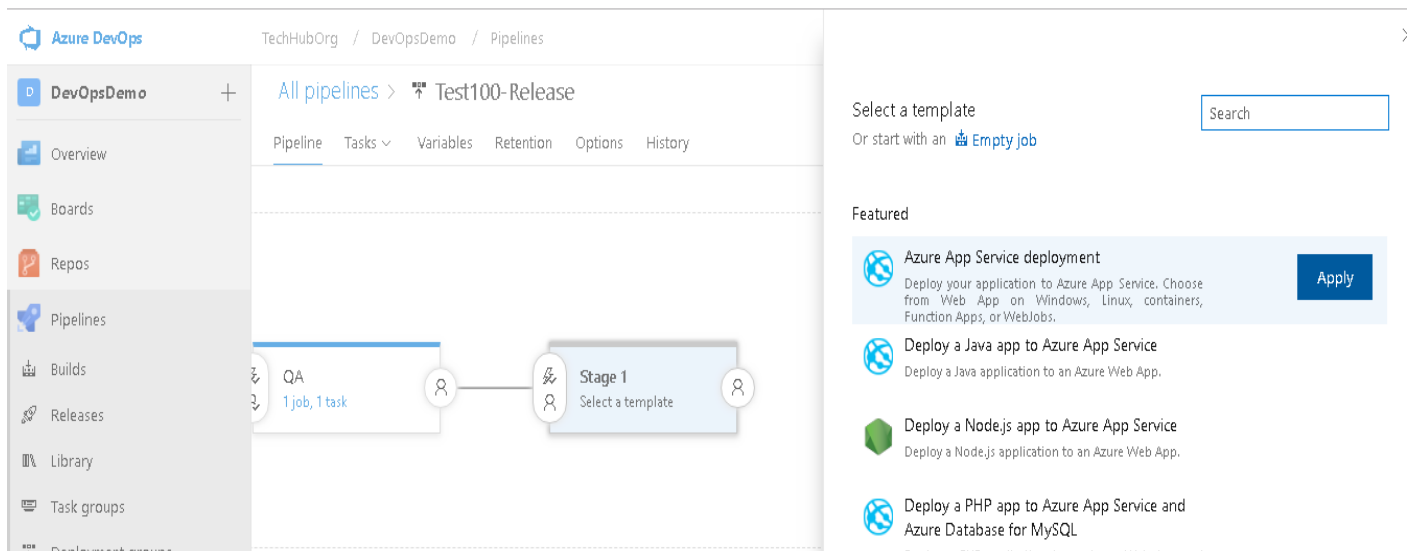
Releases	Created	Stages
Release-1 20190228.2 mastx	2019-02-28 22:24	DEV QA

Follow the same steps which we have already followed while adding the **QA stage**. So, just mouse hover just below to QA stage and click to **(+ Add)** icon for adding the new stage.

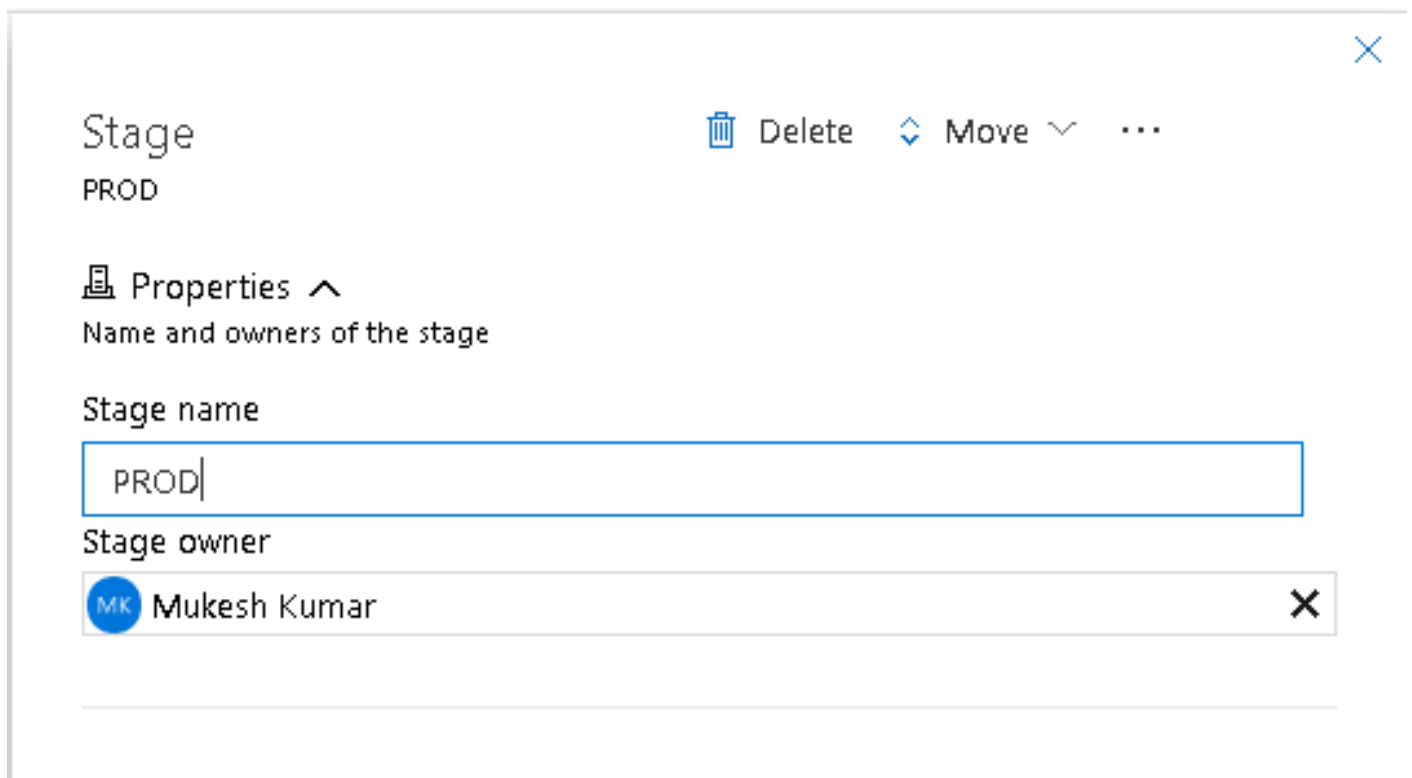
The screenshot shows the Azure DevOps interface for the 'Test100-Release' pipeline in edit mode. The left sidebar contains the navigation menu with 'Pipelines' selected. The main area displays the 'Test100-Release' pipeline with a table of releases. The 'QA' stage is highlighted, and the '+ Add' icon is visible below it.

Artifacts + Add	Stages + Add
_DevOpsDemo-CI Schedule not set	DEV 1 job, 1 task QA 1 job, 1 task + Add Add a stage

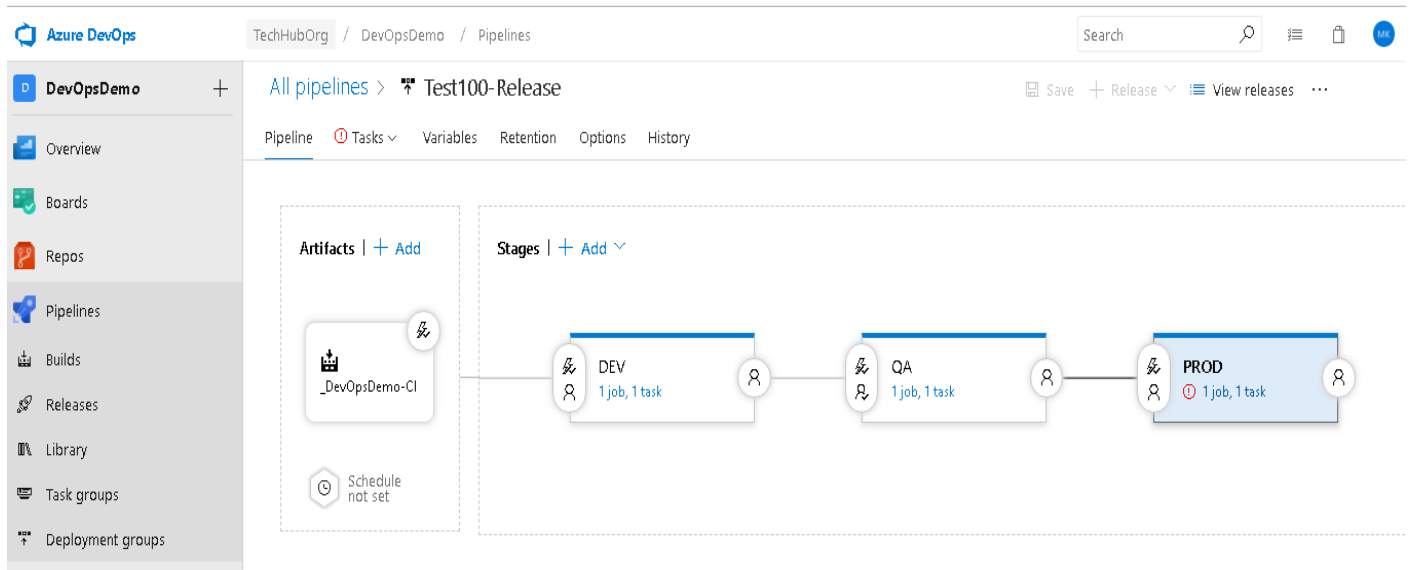
As generally, it will ask to select the template. So, select the template as **'Azure App Service Deployment'** and click to **Apply**. After applying the template just close this dialog window.



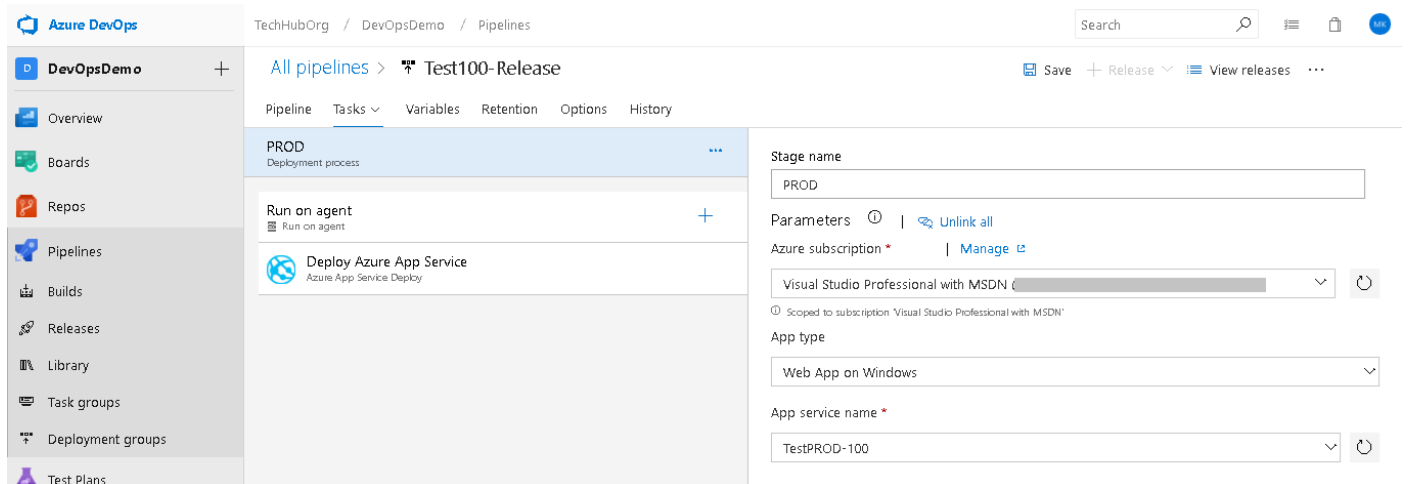
Now, we can see that just after the **QA** stage, one more stage has added as '**Stage 1**'. Just click on it and it will open the stage name dialog popup. From here we can change the name of the stage. So, just add the name as '**PROD**'. And close the Stage dialog popup using close icon (**X**) at the right top corner.



Here with following image, we can clearly see that we have now 3 different environment for **DEV**, **QA** and **PROD** respectively. But configuration is pending for **PROD (TestPROD-100)** environment. So, let complete it first. Click on the **(1 Job, 1 Task)**.

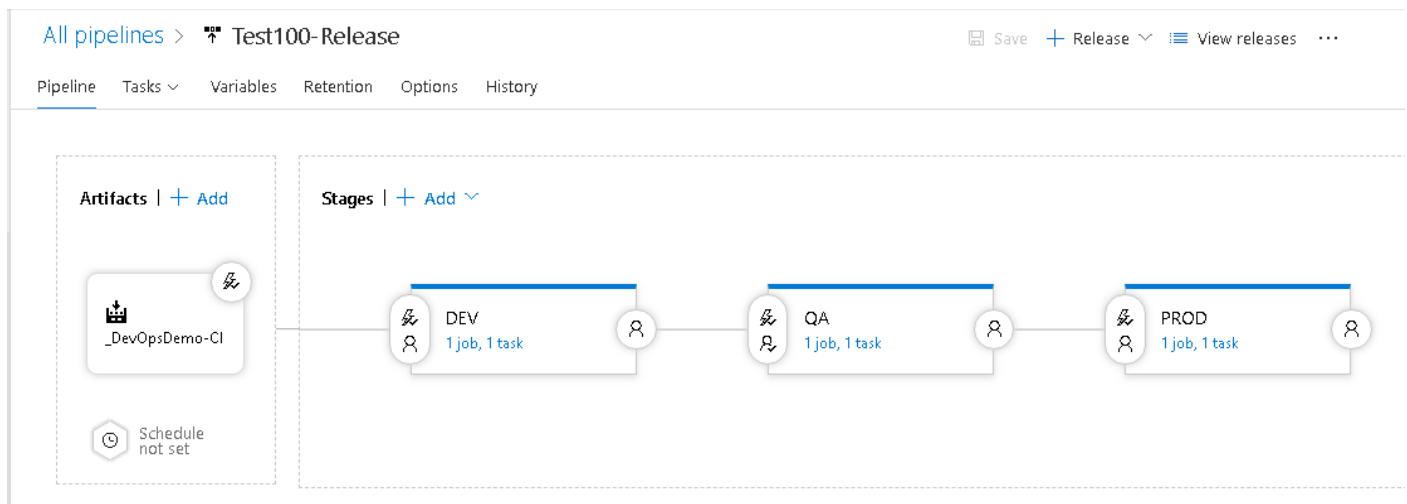


Go to **Tasks** tab, Provide the **Azure Subscription**, App type as we have done previously. But be carefully while selecting the App Service Name. This time, we will choose the **TestPRDO-100** as an App Service Name.



Nothing we have to do for **variable, retention, options and history**. Keep all tabs with default values and click to **SAVE**.

So, we have ready all 3 environments as follows. We are configuring any approval for **PROD** deployment. After QA deployment, PROD will start deploying.



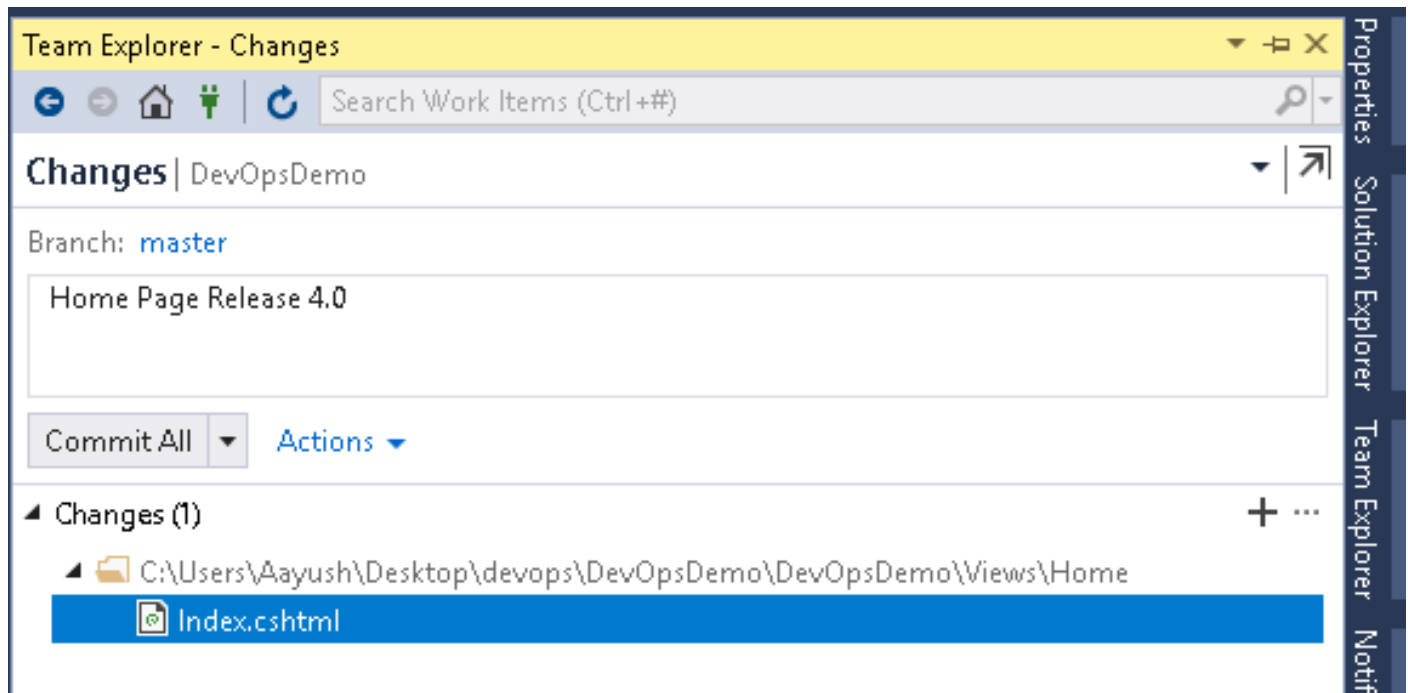
Let move to **Visual Studio 2017 or higher version** and make again changes in **Index.cshtml** file. This time we will only change the version number from **'3.0' to '4.0'**. Rest of the code will be same.

```

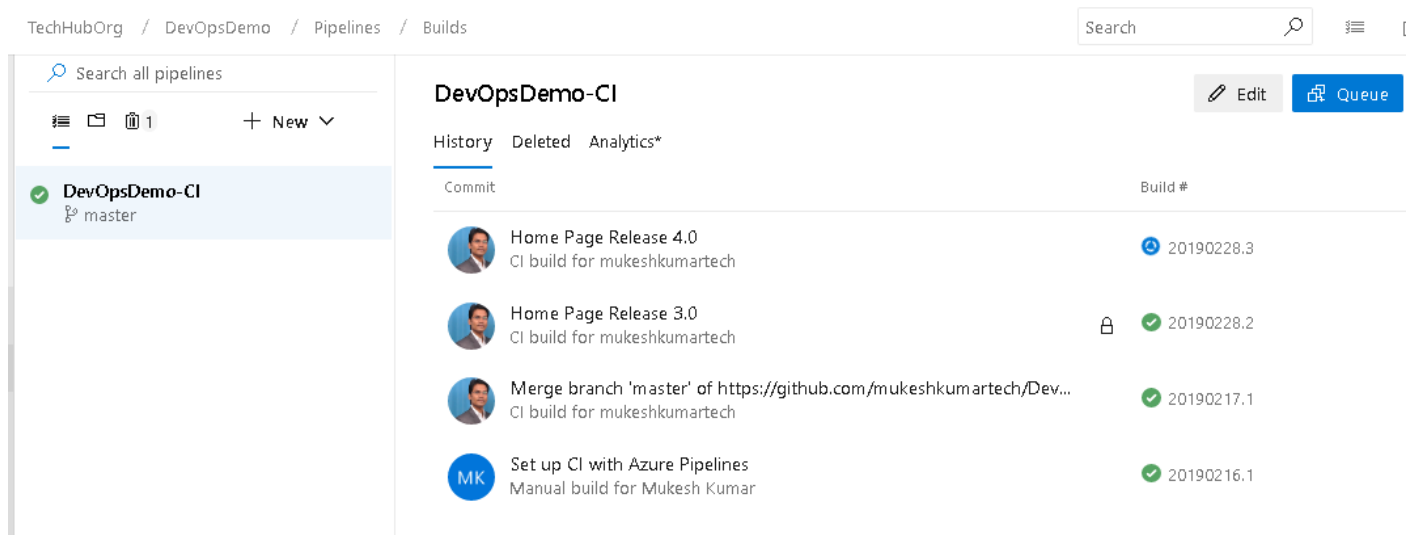
Server Explorer  Performance Explorer  Toolbox  Test Explorer
PostTestController.cs  site.css  Index.cshtml  HomeController.cs  Startup.cs  PostRepository.cs
1  @model IList<DevOpsDemo.Models.PostViewModel>
2
3  @{
4      ViewData["Title"] = "Home Page Release 4.0";
5  }
6
7  <div class="row">
8      <h2>Post List: Release 4.0</h2>
9
10     <table class="table">
11         <thead>
12             <tr>
13                 <th>Post Id</th>
14                 <th>Title</th>
15                 <th>Description</th>
16                 <th>Author</th>
17             </tr>
18         </thead>
19         <tbody>
20             @foreach (var item in Model)
21             {
22                 <tr>
23                     <td>@Html.DisplayFor(modelItem => item.PostId)</td>
24                     <td>@Html.DisplayFor(modelItem => item.Title)</td>
25                     <td>@Html.DisplayFor(modelItem => item.Description)</td>
26                     <td>@Html.DisplayFor(modelItem => item.Author)</td>
27                 </tr>
28             }
29         </tbody>
30     </table>

```

After making the changes in **Index.cshtml** as above, let check in the code. First provide the valuable comment so that it can be tracked and **Commit All and Sync**.



After successfully checked in the code. Let move to build pipeline (**TechHubOrg > DevOpsDemo > Pipeline > Builds**). Here we can see that new build as initiats as **'Home Page Release 4.0'**.



Let opent the new build **'Home Page Release 4.0'** as follows. Here it will perform all jobs before completing the Build.

#20190228.3: Home Page Release 4.0 Cancel build ...

Triggered just now for mukeshkumartech mukeshkumartech/DevOpsDemo master 0cbe1a9

Logs Summary Tests

Agent job 1 Job Started: 2/28/2019, 11:09:58 PM
 Pool: [Hosted Ubuntu 1604](#) · Agent: Hosted Agent 1m 37s

Initialize Agent · succeeded	< 1s
Initialize job · succeeded	1s
Checkout · succeeded	8s
Restore · succeeded	1m 6s
Build	20s

```

*****
Starting: Build
*****
Task       : .NET Core
Description : Build, test, package, or publish a dotnet application, or run a custom dotnet command. For package commands, supports NuGet.org and
authenticated feeds like Package Management and MyGet.
Version    : 2.147.2
Author     : Microsoft Corporation
Help       : [More Information](https://go.microsoft.com/fwlink/?linkid=832194)
*****
[command]/usr/bin/dotnet build /home/vsts/work/1/s/DevOpsDemo.Test/DevOpsDemo.Test.csproj --configuration Release
Microsoft (R) Build Engine version 15.9.20+88f5fadf6e for .NET Core
  
```

Let wait for completing the Build and withing few minutes we will see that Build has completed as follows.

#20190228.3: Home Page Release 4.0 Release Artifacts ...

Triggered today at 11:09 pm for mukeshkumartech mukeshkumartech/DevOpsDemo master 0cbe1a9 Retained by release

Logs Summary Tests

Agent job 1 Job Started: 2/28/2019, 11:09:58 PM
 Pool: [Hosted Ubuntu 1604](#) · Agent: Hosted Agent 2m 24s

Initialize Agent · succeeded	< 1s
Prepare job · succeeded	< 1s
Initialize job · succeeded	1s
Checkout · succeeded	8s
Restore · succeeded	1m 6s
Build · succeeded	33s
Test · succeeded	27s
Publish · succeeded	2s
Publish Artifact · succeeded	3s
Post-job: Checkout · succeeded	< 1s
Finalize Job · succeeded	< 1s

Let move to **Release Pipeline** for this (**TechHubOrg > DevOpsDemo > Pipeline > Releases > Test100-Release**). Here we will find new release has initiated as **Release 2** and **Continuous Deployment** is started on **DEV**.

(By: Mukesh Kumar)

The screenshot shows the 'Test100-Release' page in Azure DevOps. On the left, there's a sidebar with a search bar and a list of releases. The main area shows a table of releases:

Releases	Created	Stages
Release-2 20190228.3 master	2019-02-28 23:12	☒ DEV ☐ QA ☐ PROD
Release-1 20190228.2 master	2019-02-28 22:24	☒ DEV ☒ QA

Let open the **Release-2** and we will find that **Continuous Deployment** is in progress on **DEV** environment. Let wait for completing it.

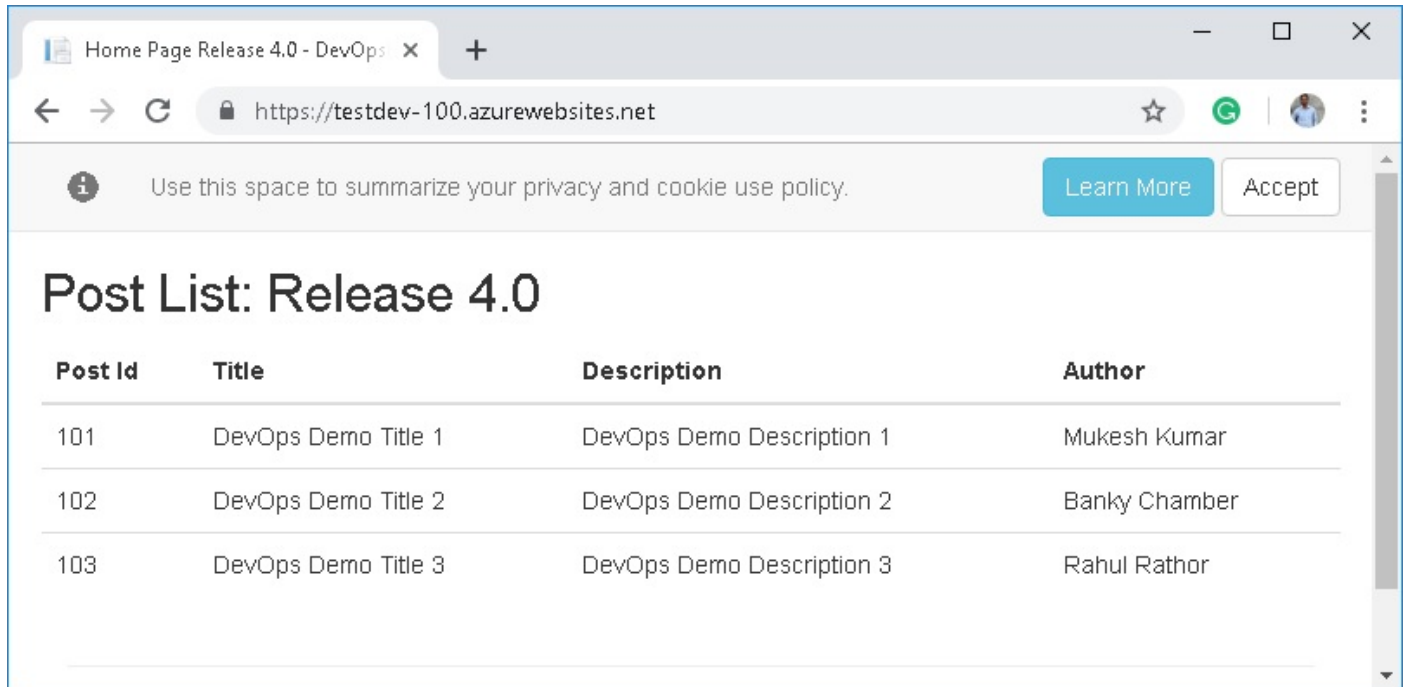
The screenshot shows the 'Test100-Release > Release-2' page. The 'Release' section on the left shows the deployment details for 'Continuous deployment...' for 'Mukesh Kumar' on '2/28/2019, 11:12 PM'. The 'Stages' section shows a sequence of stages: 'DEV' (In progress... 5/5 tasks), 'QA' (Pending approval), and 'PROD' (Not deployed).

After few minutes, it will complete the deployment on **DEV** and move to **QA**. But as we have configured that approval is required before deployment on QA. It will ask for **Approval**.

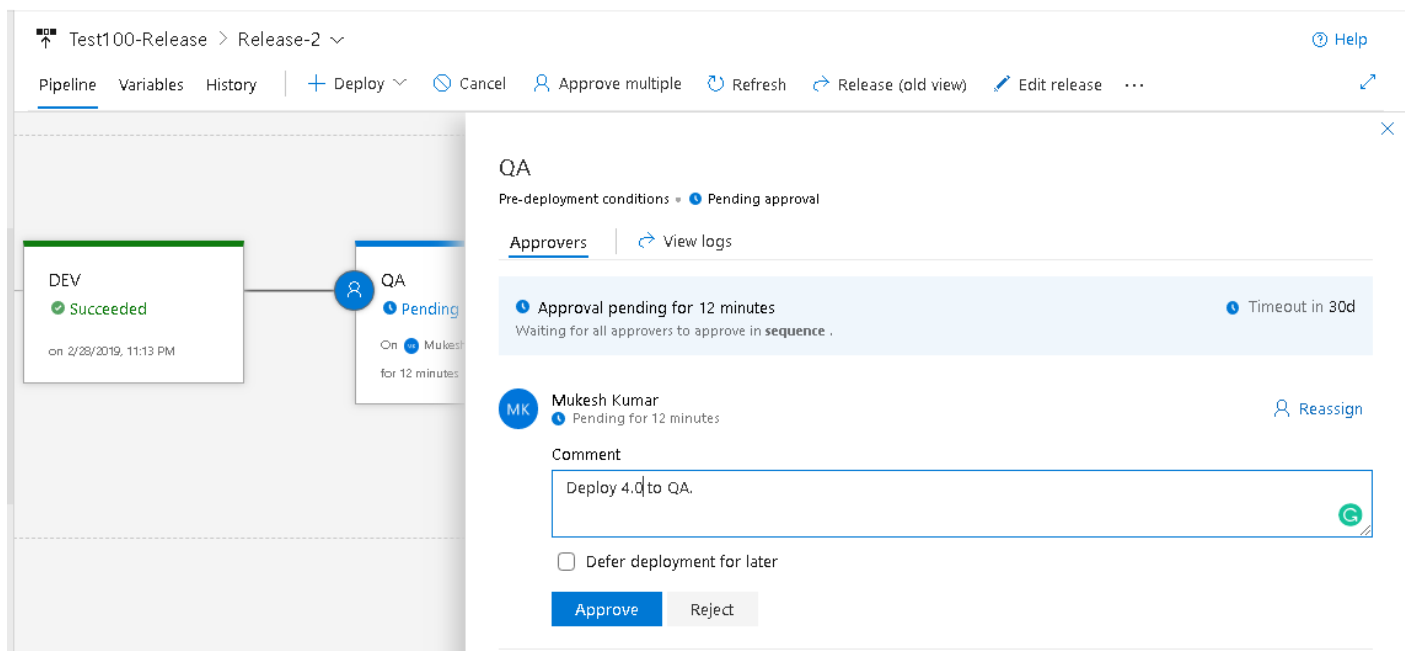
The screenshot shows the 'Test100-Release > Release-2' page after some time. The 'DEV' stage is now 'Succeeded' (green checkmark) and 'on 2/28/2019, 11:13 PM'. The 'QA' stage is 'Pending approval' (blue circle with a person icon) and 'On Mukesh Kumar for 8 minutes'. The 'PROD' stage remains 'Not deployed'. There is an 'Approve' button below the QA stage.

(By: Mukesh Kumar)

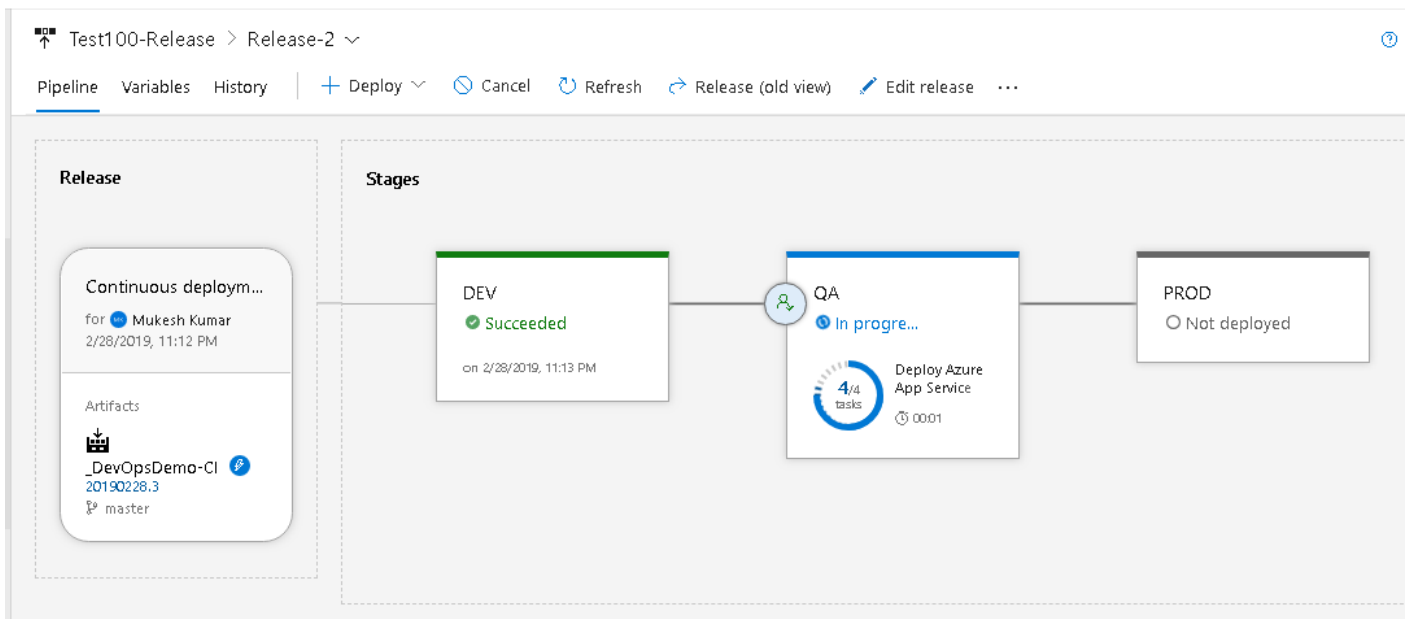
Before approving for QA. Let check what changes has deployed on **DEV (TestDEV-100 App Service)**. So, open the URL for DEV server as <https://testdev-100.azurewebsites.net> and here we go. Good, we have deployed **Release 4.0** to **DEV** server successfully.



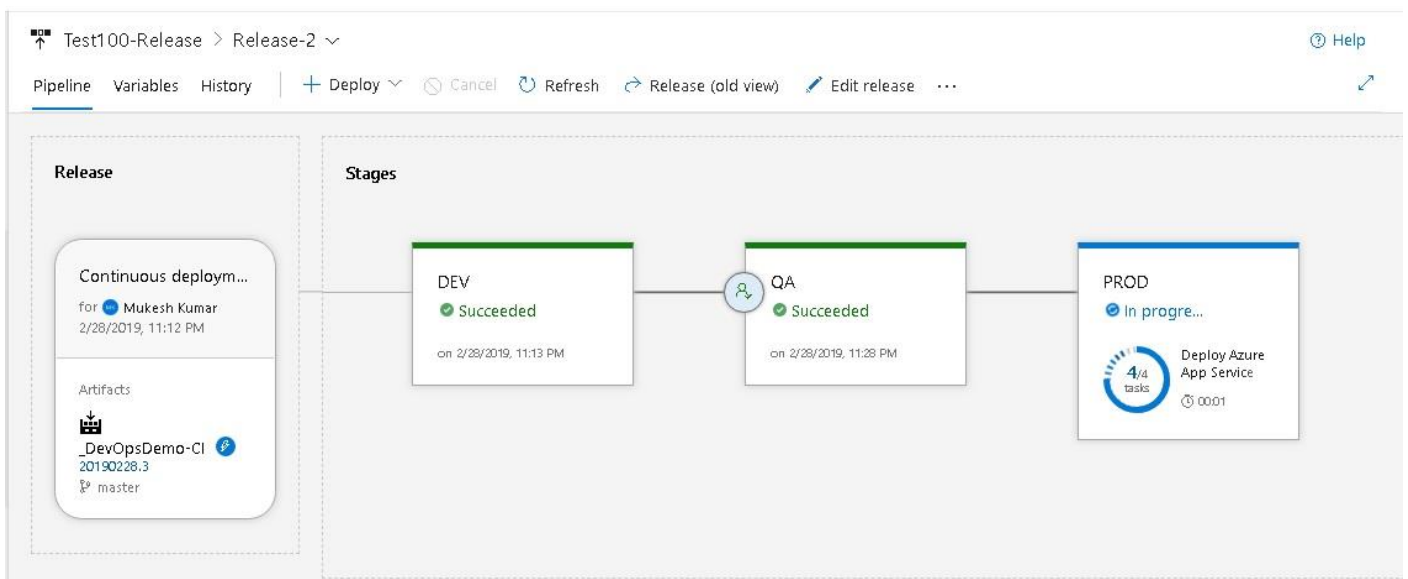
Now, its time to approve the pending approval for **QA**. So, click on Approve. It will ask for comment. Provide the comment like '**Deploy 4.0 to QA**' and click to **Approve** button.



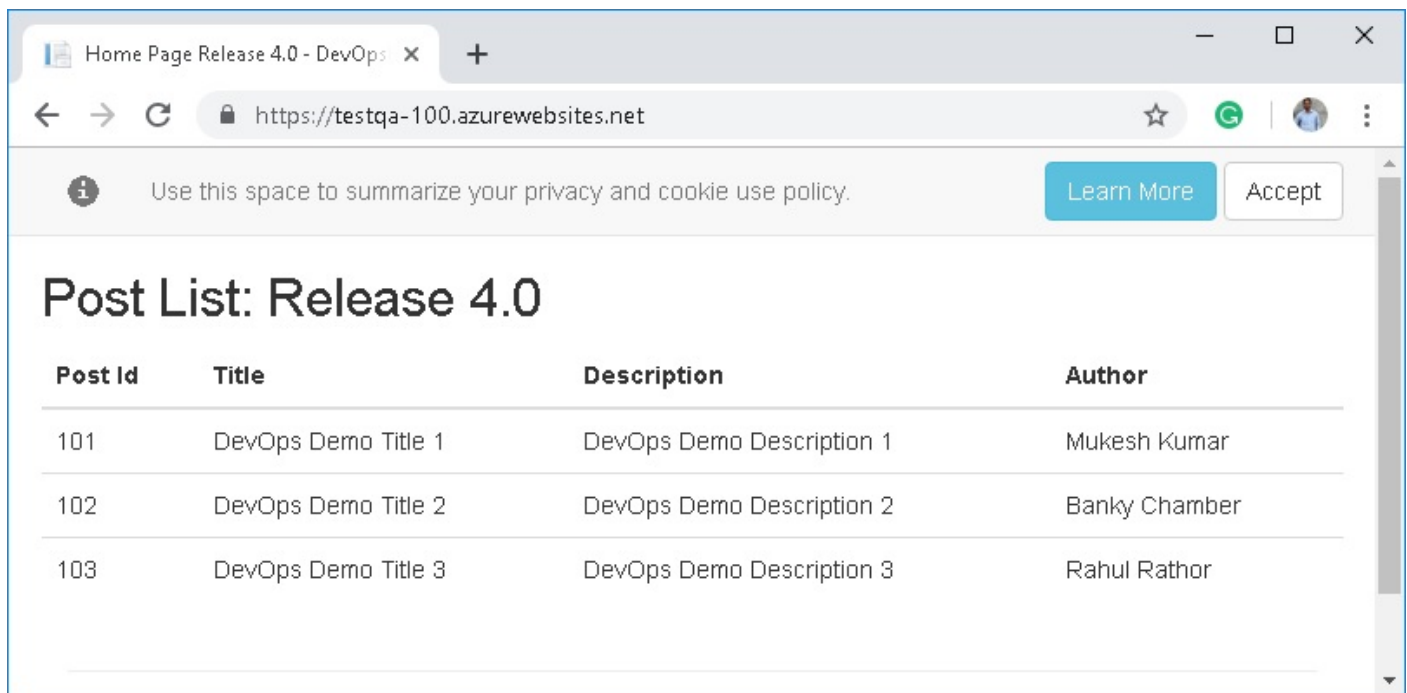
As we have approved, it will start the deployment on QA server. As we can see with following image. Continuous Deployment on QA is in progress.



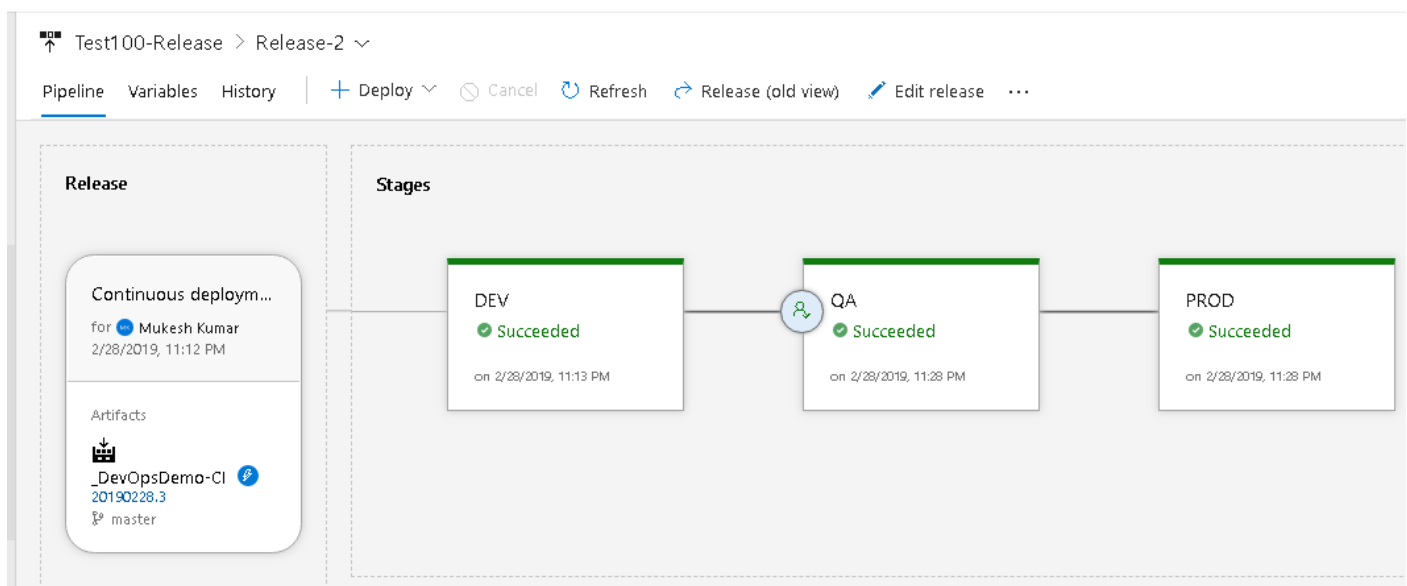
After few minutes, it will done with **Continoouse Deployment on QA**.



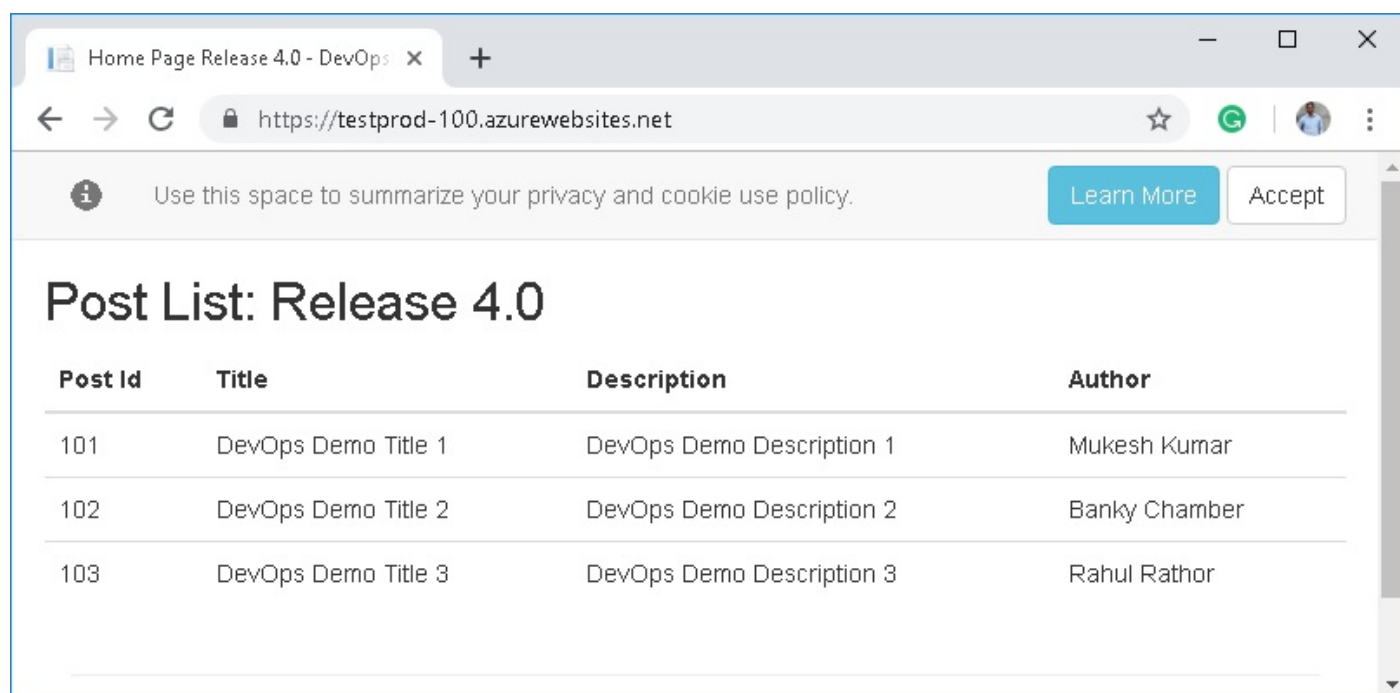
Let open the QA environment (**TestQA-100 App Service**) URL <https://testqa-100.azurewebsites.net> in browser and here we go. Great, **Release 4.0** is also deployed on **QA**.



As we didn't configure any approval before **PROD** deployment. So, after deployment on QA, it will auto deploy on PROD. So, **Release 4.0 has deployed to DEV, QA and PROD as well.**



Let open the PROD server and see that release 4.0 has deployed or not. Open the **PROD** environment URL as <https://testprod-100.azurewebsites.net>. Great, we have also deployed **release 4.0 on PRDO** as well.



The screenshot shows a web browser window with a single tab titled "Home Page Release 4.0 - DevOps". The address bar displays the URL "https://testprod-100.azurewebsites.net". A privacy notice banner is visible, stating "Use this space to summarize your privacy and cookie use policy." with "Learn More" and "Accept" buttons. The main content area features the heading "Post List: Release 4.0" and a table with four columns: Post Id, Title, Description, and Author. The table contains three rows of demo data.

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

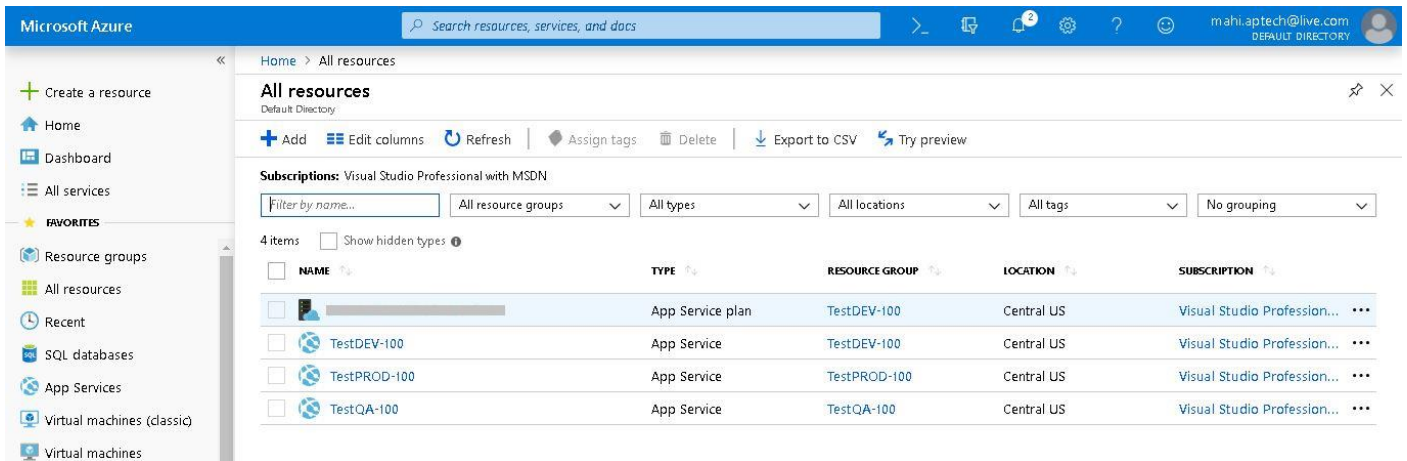
9. Add Slot

Go to <https://portal.azure.com> and **All Resources** page, here we can see listed all the available resource which we had created till now. Here we would like to add one more stage between QA and PROD. Actually, the reason for using the **Staging** environment is that we don't want directly publish the changes from QA to PROD. First we will verify the changes and functionality and if everything is working as expected then we will move the changes on PROD using **SWAP**.

Slots allow you to deploy your application to a separate live app service, warm it up and make sure it's ready for use in production, and then swap the slots to provide seamless traffic redirection. You can slot swap manually (in the portal or command line) or you can automate the slot swap with Auto-swap or in a script.

SWAP is basically is a feature of Azure DevOps where we use two different environments for deployments where one is slot, using SWAP, we can swap the production environment with some staging (slot) environment with 0 downtimes. So, it is a very great feature for PROD deployment.

As we are going to create one more staging environment just before PROD. So, let open the PROD app service from the **All Resources** page.



NAME	TYPE	RESOURCE GROUP	LOCATION	SUBSCRIPTION
[Icon] [Name]	App Service plan	TestDEV-100	Central US	Visual Studio Profession...
[Icon] TestDEV-100	App Service	TestDEV-100	Central US	Visual Studio Profession...
[Icon] TestPROD-100	App Service	TestPROD-100	Central US	Visual Studio Profession...
[Icon] TestQA-100	App Service	TestQA-100	Central US	Visual Studio Profession...

Once open the **PROD (TestPROD-100)** environment, here we can see all configuration related to this App Service. Apart from this, we have an option to create **Slot** in Deployment section as '**Deployment Slots**'. Using Deployment Slots option, we can create new slot (**staging environment**). So, let click on the **Deployment Slots**.

The screenshot shows the Azure portal interface for the 'TestPROD-100' App Service. The left sidebar contains navigation options like 'Create a resource', 'Home', 'Dashboard', 'All services', and 'FAVORITES'. The main content area shows the 'Overview' tab with details such as Resource group (TestPROD-100), Status (Running), Location (Central US), and Subscription (Visual Studio Professional with MSDN). A table at the bottom lists deployment slots.

NAME	STATUS	APP SERVICE PLAN	TRAFFIC %
TestPROD-100	PRODUCTION	Running	100

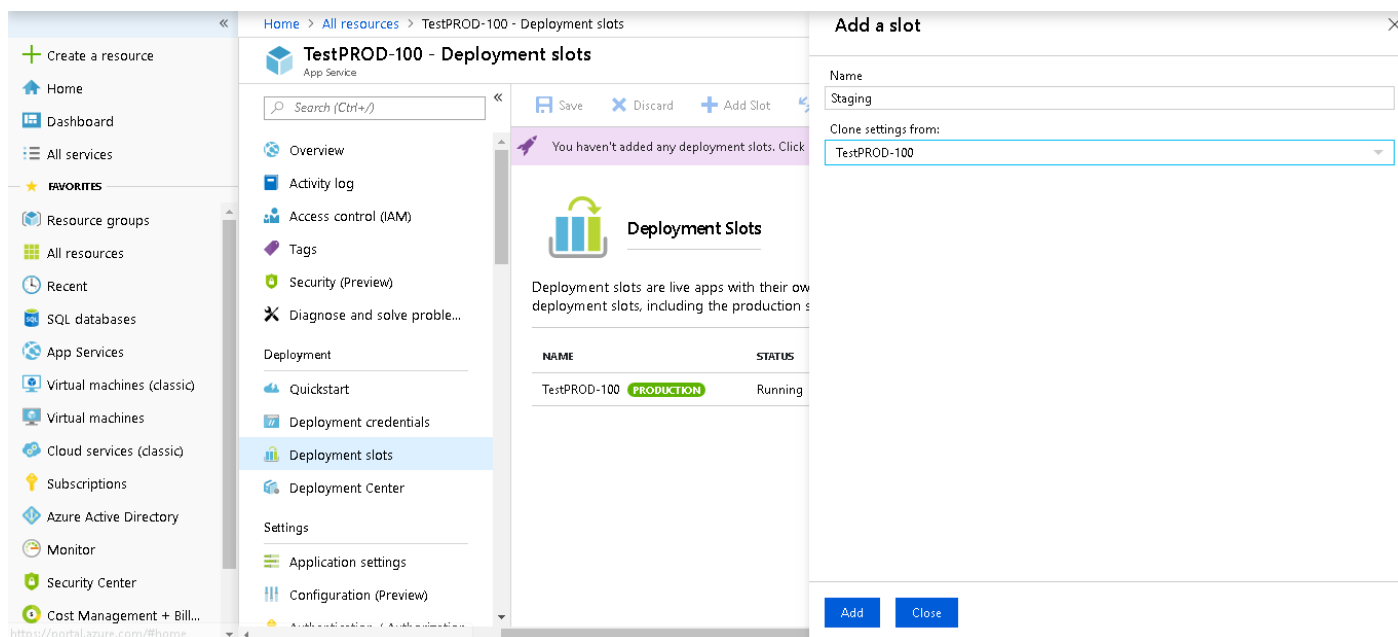
Next page will be deployment slots page. Here we can see that **PROD** environment is running with **100% traffic**. It means all the traffics which are going to access PROD environment will hit to **TestPROD-100 App Service**.

At the top, we have one button as 'Add Slot'. For adding new slot, just click on 'Add Slot'.

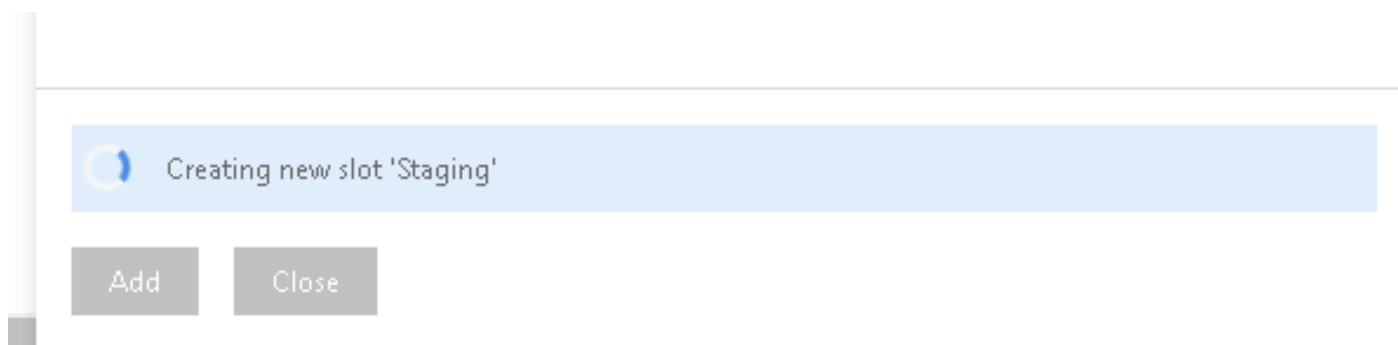
The screenshot shows the 'Deployment slots' page for 'TestPROD-100'. The page includes a header with 'Save', 'Discard', 'Add Slot', 'Swap', and 'Refresh' buttons. A message states: 'You haven't added any deployment slots. Click here to get started.' Below this is a section titled 'Deployment Slots' with a description: 'Deployment slots are live apps with their own hostnames. App content and configurations elements can be swapped between two deployment slots, including the production slot.' A table at the bottom lists deployment slots.

NAME	STATUS	APP SERVICE PLAN	TRAFFIC %
TestPROD-100	PRODUCTION	Running	100

A dialog window will open in right side and ask for the name of the new slot. Here we will give the name as 'Staging' and use the **TestPROD-100** for **clone** setting. It means, new **Staging** environment will be created to clone of **PROD** environment. Let click to **Add** button for adding new Slot.



It will take few minutes to create the new **Staging environment**.



Once it will be done, it will show a **Success** message as follow. Just close this dialog window.

Home > All resources > TestPROD-100 - Deployment slots

TestPROD-100 - Deployment slots

App Service

Search (Ctrl+ /)

Overview
Activity log
Access control (IAM)
Tags
Security (Preview)
Diagnose and solve problem...

Deployment
Quickstart
Deployment credentials
Deployment slots
Deployment Center

Settings
Application settings
Configuration (Preview)
Authentication / Authorization

Deployment Slots

Deployment slots are live apps with their own deployment slots, including the production slot.

NAME	STATUS
TestPROD-100 PRODUCTION	Running
testprod-100(Staging)	Running

Add a slot

Name
Staging

Clone settings from:
TestPROD-100

Successfully created slot 'Staging'

Add Close

Now, we have one Staging environment for **TestPROD-100** as **TestPROD-100(Staging)**. It is also running but right now, there is no any traffic on this.

Save Discard Add Slot Swap Refresh

Deployment Slots

Deployment slots are live apps with their own hostnames. App content and configurations elements can be swapped between two deployment slots, including the production slot.

NAME	STATUS	APP SERVICE PLAN	TRAFFIC %
TestPROD-100 PRODUCTION	Running	ServicePlane	100
testprod-100(Staging)	Running	ServicePlane	0

Let click on the name of staging environment **TestPROD-100(Staging)**. It will open the configuration page for this Staging environment. Here we can see the URL for the staging environment which will be used to access it. We have several options for this Staging environment like **STOP**, **SWAP**, **RESTART**, **DELETE** etc.

The screenshot shows the Azure portal interface for a Web App named 'Staging (testprod-100/Staging)'. The left sidebar contains navigation options like Overview, Activity log, Access control (IAM), Tags, Security (Preview), Diagnose and solve problem..., Deployment, Quickstart, Deployment credentials, Deployment slots, Deployment Center, Settings, Application settings, and Configuration (Preview). The main area displays the app's configuration details:

- Resource group:** testprod-100
- Status:** Running
- Location:** Central US
- Subscription:** Visual Studio Professional with MSDN
- Subscription ID:** [Redacted]
- Tags:** [Redacted]
- URL:** https://testprod-100-staging.azurewebsites.net
- App Service Plan:** ServicePlan [Redacted] (Standard: 1 Small)
- FTP/deployment username:** testprod-[Redacted]
- FTP hostname:** ftp://[Redacted].azurewebsites.windows.net
- FTPS hostname:** ftps://[Redacted].azurewebsites.windows.net

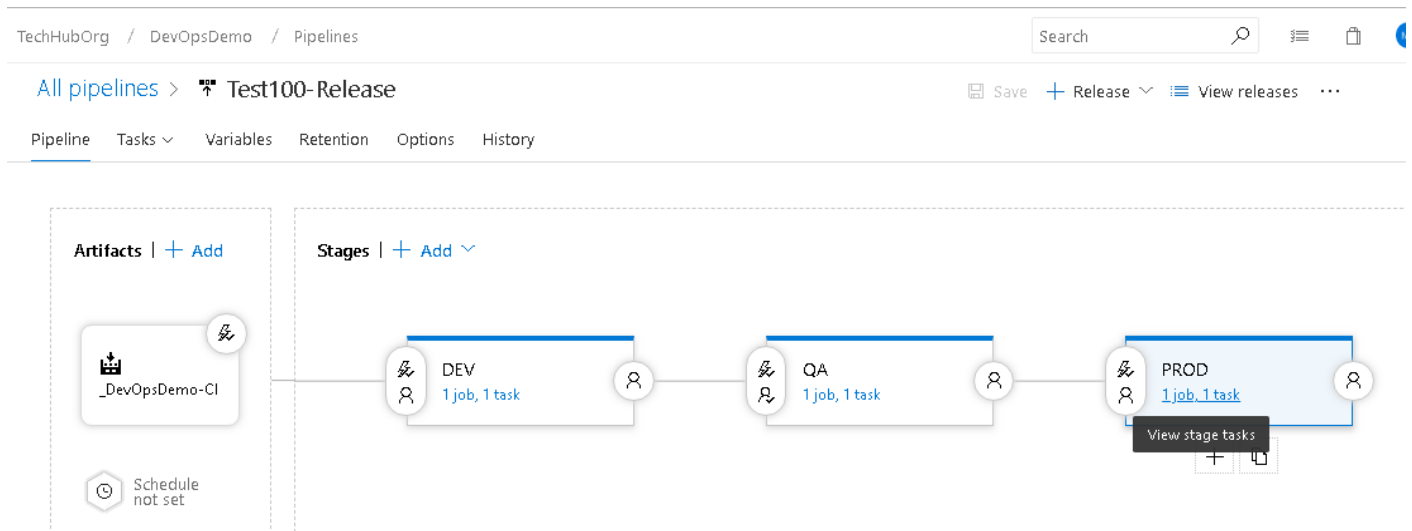
Below the configuration details, there are three tiles: 'Diagnose and solve problems', 'Application Insights', and 'App Service Advisor'.

Staging environment is available now. Let configure it in the **Release** pipeline. So, open <https://dev.azure.com/TechHubOrg/DevOpsDemo> and release pipeline as **Test100-Release** and **Edit** it.

The screenshot shows the Azure DevOps Release pipeline 'Test100-Release' in the 'Releases' tab. The left sidebar shows the pipeline list with 'Test100-Release' selected. The main area displays a table of releases:

Releases	Created	Stages
Release-2 20190228.3 masr	2019-02-28 23:12	DEV QA PROD
Release-1 20190228.2 masr	2019-02-28 22:24	DEV QA

Once **Test100-Release** will open in **Edit** mode. Just click on **PROD** stage. Actually we will configure the Staging environment with PRDO. If any new **build artifact** will be available and going to deploy on **PROD** after **QA** then it will first deploy on Staging environment. Later user can use the **SWAP** functionality to **SWAP** the production environment with staging environment.



In PROD stage, Go to **Tasks** tab and it will open a dialog window in right. Here check the option as **'Deploy to Slot or App Service Environment'** and select the **Resource Group** name as **'TestPROD-100'**. Next option is to **choose the Slot**. So, in the Slot dropdown, we have to select the **'Staging'**.

So far, so good, we have done implementing the slotting in PROD environment. We don't need to change any other options for now. So, now just click on **SAVE** button to save the **Release pipeline**.

The screenshot shows the Azure DevOps Pipelines interface for the 'Test100-Release' pipeline. The 'Tasks' tab is selected, and the 'Deploy Azure App Service' task is highlighted. The task configuration is shown on the right, including the App Service name, Resource group, Slot, and Package or folder.

App Service name *

TestPROD-100

☒ Deploy to Slot or App Service Environment ⓘ

Resource group * ⓘ

TestPROD-100

Slot * ⓘ

Staging

Virtual application ⓘ

Package or folder * ⓘ

\$(System.DefaultWorkingDirectory)/**/*.zip

File Transforms & Variable Substitution Options ▾

Additional Deployment Options ▾

Post Deployment Action ▾

Application and Configuration Settings ▾

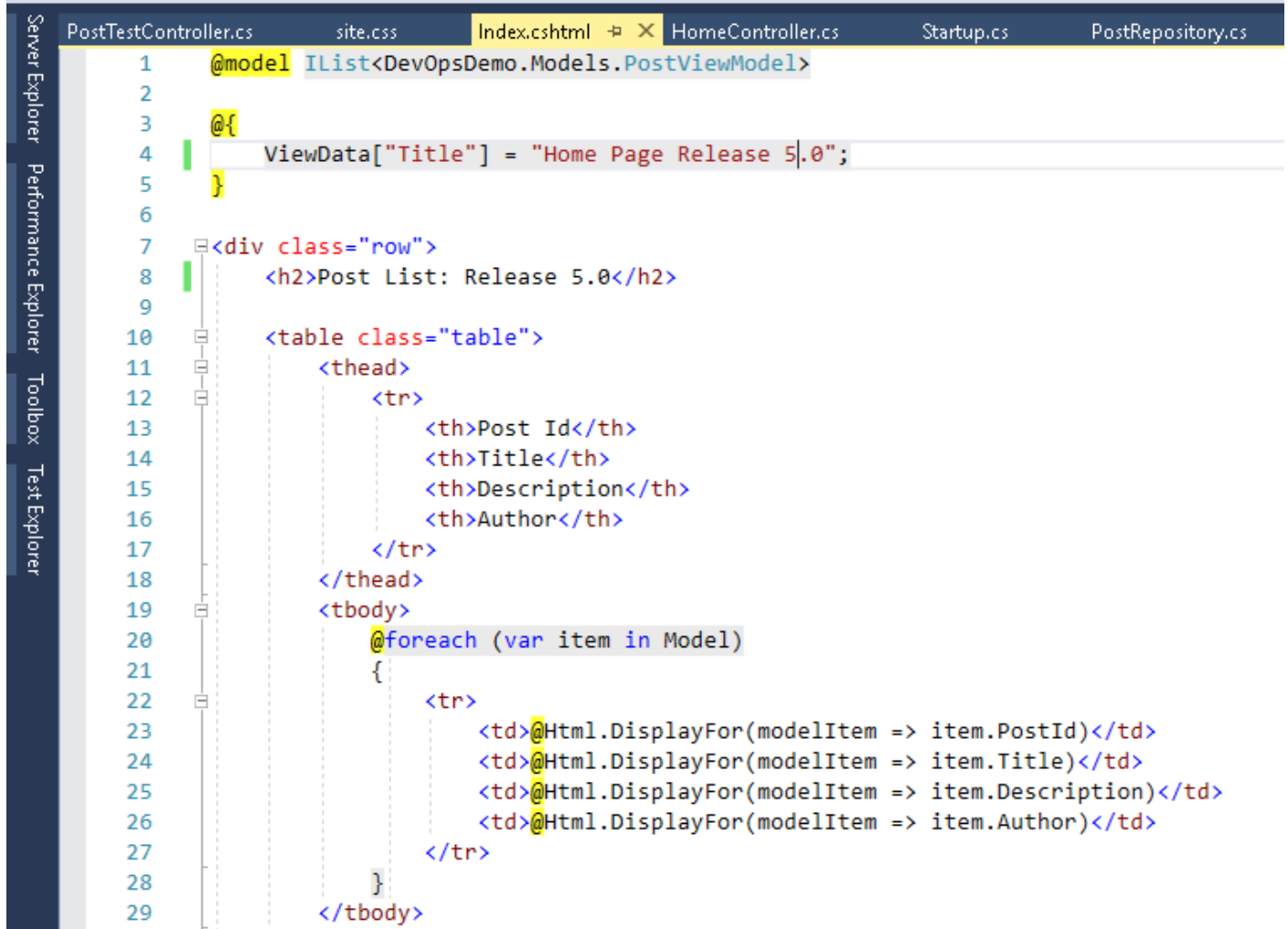
Above we are done with adding the **Slotting functionality with RPDO environment**. So, let make the changes in the code and check in the code. When we will check in the code, what should happen? First, it should make the build

(By: Mukesh Kumar)

artifact in Build Cycle and deploy the artifact on DEV server in Release cycle and after approval, it should deploy on QA and then deploy to Staging environment. It should not deploy anything on **PROD**. We will deploy on the PROD using **SWAP** feature.

10. Run and Test Azure DevOps Pipeline

So, open the **Visual Studio 2017 or higher version** one more time and open the **Index.cshtml**. Here we will only change the version number in page title and post title. So, change version number from **4.0** to **5.0**.

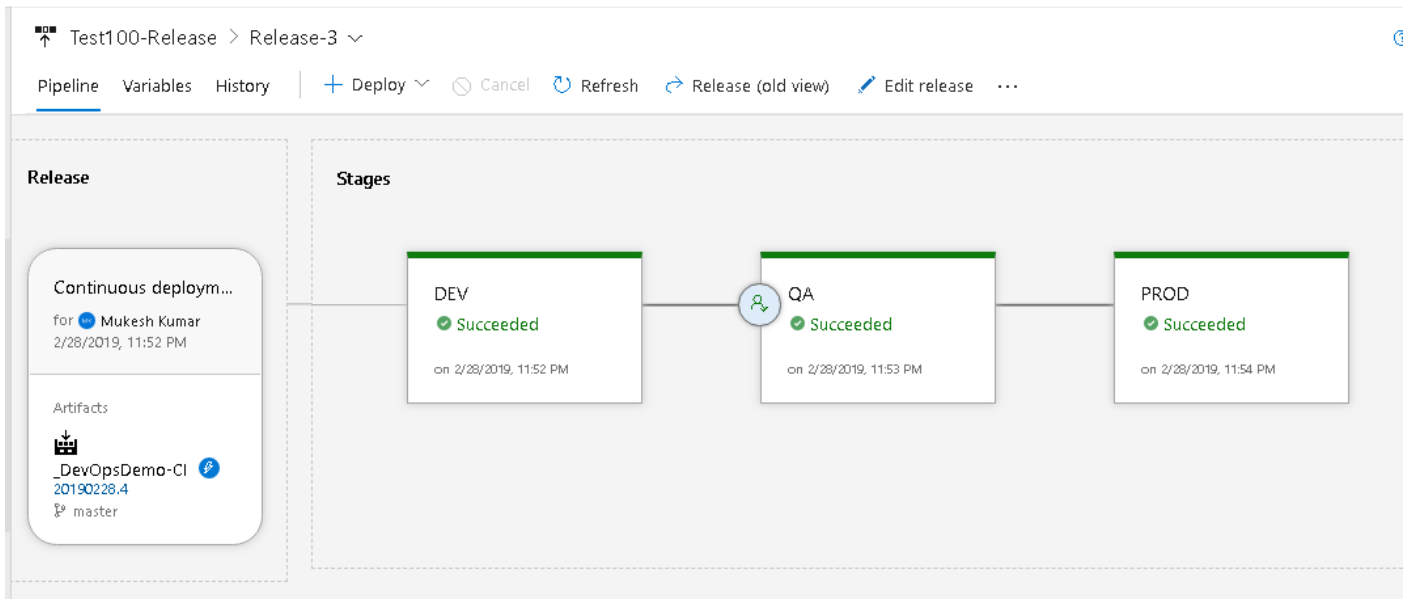


```

1  @model IList<DevOpsDemo.Models.PostViewModel>
2
3  @{
4      ViewData["Title"] = "Home Page Release 5.0";
5  }
6
7  <div class="row">
8      <h2>Post List: Release 5.0</h2>
9
10     <table class="table">
11         <thead>
12             <tr>
13                 <th>Post Id</th>
14                 <th>Title</th>
15                 <th>Description</th>
16                 <th>Author</th>
17             </tr>
18         </thead>
19         <tbody>
20             @foreach (var item in Model)
21             {
22                 <tr>
23                     <td>@Html.DisplayFor(modelItem => item.PostId)</td>
24                     <td>@Html.DisplayFor(modelItem => item.Title)</td>
25                     <td>@Html.DisplayFor(modelItem => item.Description)</td>
26                     <td>@Html.DisplayFor(modelItem => item.Author)</td>
27                 </tr>
28             }
29         </tbody>

```

Save the changes and Go to **Team Explorer** and **check in the code** as we have done previously. Once the code checks in then wait for the completion of **Azure DevOps Build pipeline**. After creating the **build artifact**, it auto will deploy on the **DEV** and ask for approval before deploying on **QA**. Let approve and it will deploy **QA and Staging (PROD)**.



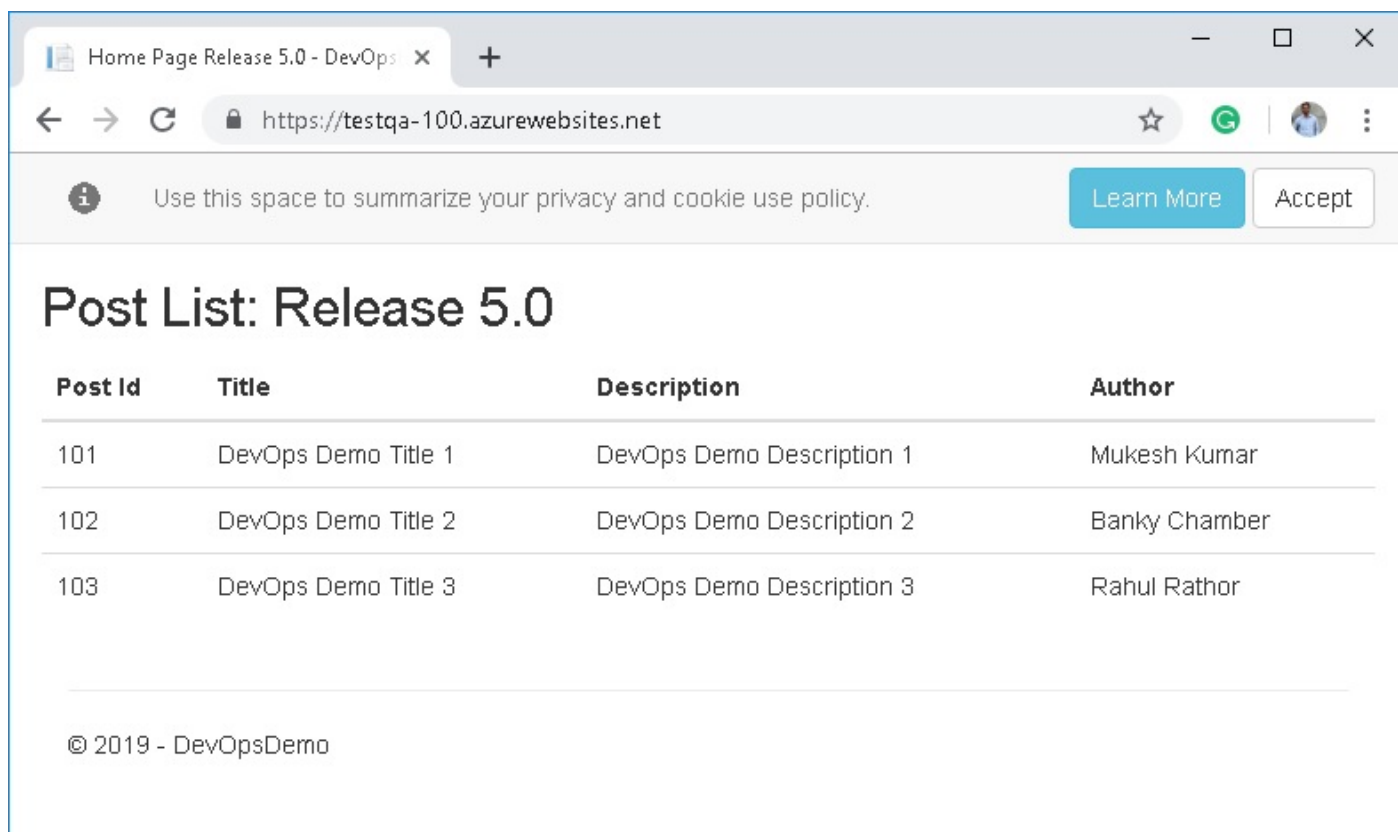
Let check first, how **DEV (TestDEV-100)** is working. Here we can see that our latest changes has deployed on the DEV environment as follows. **[Latest Release is 5.0]**.

The screenshot shows a web browser window with the title 'Home Page Release 5.0 - DevOps'. The address bar shows the URL 'https://testdev-100.azurewebsites.net'. Below the browser window, there is a privacy notice: 'Use this space to summarize your privacy and cookie use policy.' with 'Learn More' and 'Accept' buttons. The main content area is titled 'Post List: Release 5.0' and contains a table with the following data:

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

The footer of the page displays '© 2019 - DevOpsDemo'.

Let open the **QA environment (TestQA-100)** using URL <https://testqa-100.azurewebsites.net> and here we can see. Latest build artifact has deployed successfully on **QA environment** as well.



Home Page Release 5.0 - DevOps x +

← → ↻ https://testqa-100.azurewebsites.net ☆ G |  ⋮

Use this space to summarize your privacy and cookie use policy. [Learn More](#) [Accept](#)

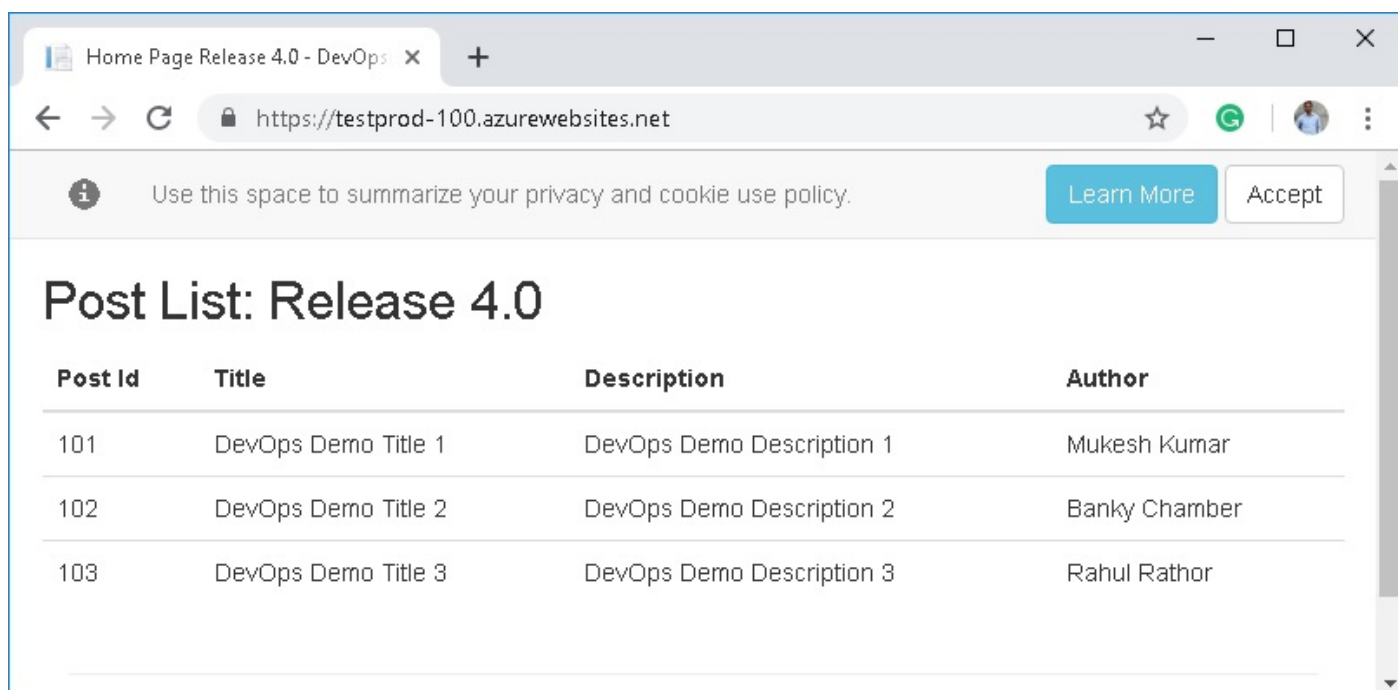
Post List: Release 5.0

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

© 2019 - DevOpsDemo

Now, its time to check the changes has deployed on PROD or not. It should not be deployed. Because we have already configured one stage between QA and PROD. The deployment should heppen on Staging environment after QA. So, let open the **PROD (TestPROD-100)** environment using the URL <https://testprod-100.azurewebsites.net> and here we go.

Good, our **PROD** environment is **not deployed** the latest changes yet.



Home Page Release 4.0 - DevOps x +

← → ↻ https://testprod-100.azurewebsites.net ☆ G |  ⋮

Use this space to summarize your privacy and cookie use policy. [Learn More](#) [Accept](#)

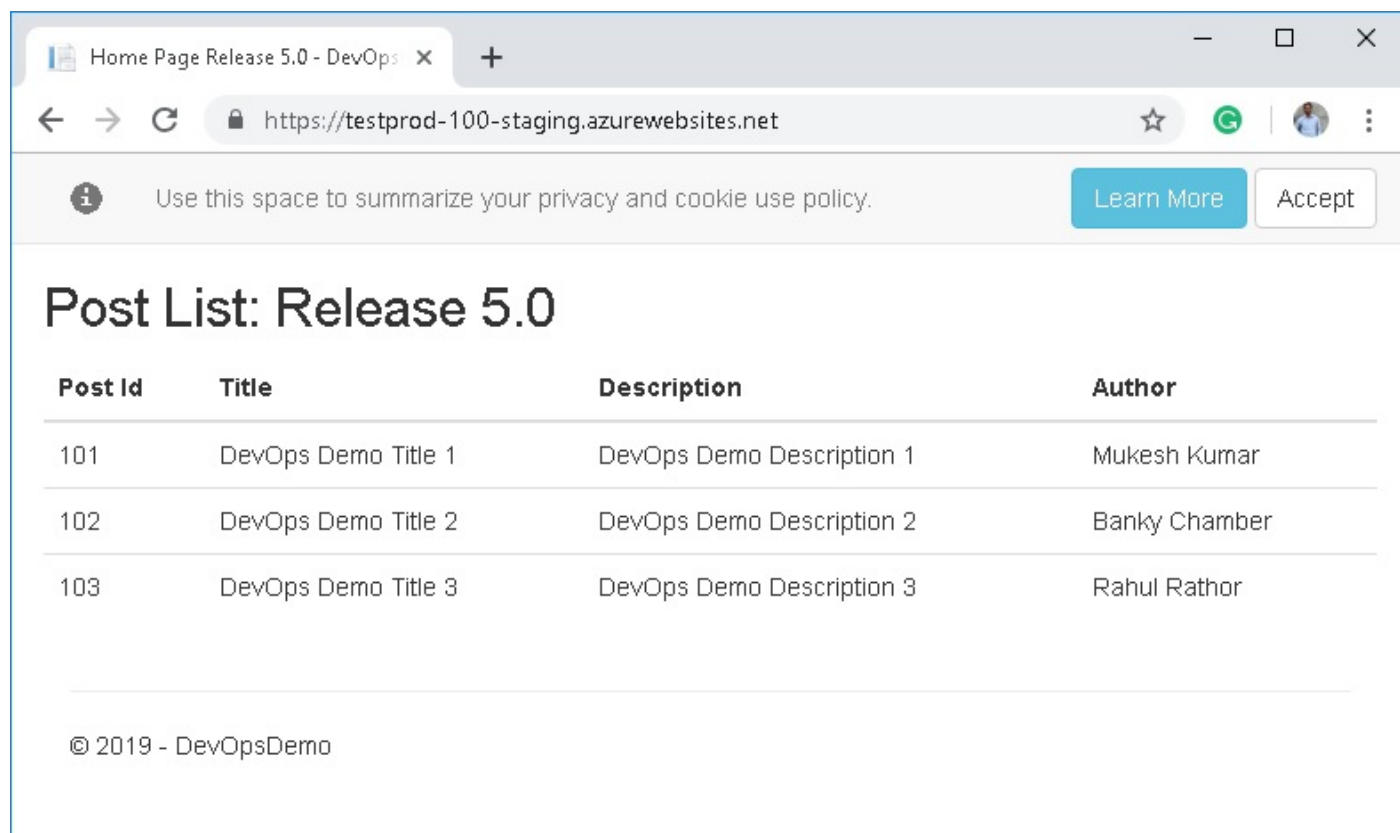
Post List: Release 4.0

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

© 2019 - DevOpsDemo

Let check the Staging environment using URL <https://testprod-100-staging.azurewebsites.net> in the browser and here we go. Yes, latest changes has deployed on Staging environment.

It means, our Release deployment has done in this way **DEV > QA (After Approval) > Staging > PROD (Not Yet Deployed)**.



Home Page Release 5.0 - DevOps x +

← → ↻ <https://testprod-100-staging.azurewebsites.net> ☆ G |  ⋮

 Use this space to summarize your privacy and cookie use policy. [Learn More](#) [Accept](#)

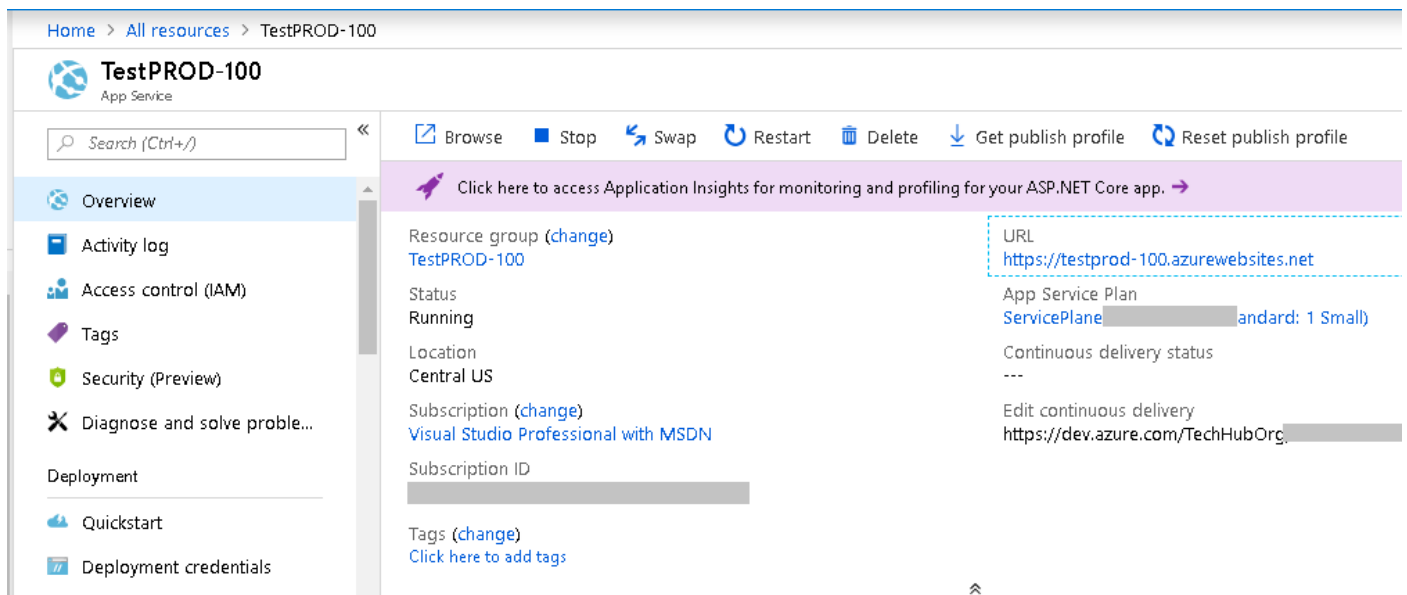
Post List: Release 5.0

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

© 2019 - DevOpsDemo

As we know and learn above that we will use the **SWAP** feature the swap the environment. Here we will **swap between Staging and PROD**. After swapping, Staging environment will become PROD and PROD environment will become the Staging.

Let go to <https://portal.azure.com> and open the **RPOD app service (TestPROD-100)** from the **All resource in Azure Portal**. Now click on the **SWAP** button at the top just after **STOP**.



It will open the **SWAP** dialog window in right. Here we have to provide the **Source environment** and **Target environment**. As our build artifact has already deployed on Staging environment, so Staging will be part of Source and PROD where we have to deploy will be part of Target.

After defining the **Source** and **Target** for **SWAPPING**, we will just click on **SWAP** button.

Swap ✕

● Source

testprod-100(Staging) ▾

● Target

PRODUCTION

TestPROD-100 ▾

i

Swap with preview can only be used with sites that have slot settings enabled

☐ Perform swap with preview

Config Changes

This is a summary of the final set of configuration changes on the source and target slots after the swap has completed.

● Source Changes

SETTING	TYPE	OLD VALUE	NEW VALUE
No Changes			

● Target Changes

Swap

Close

It will take few minutes to **swap** between **Staging** and **Prod**.



Performing swap between slot 'Staging' and slot 'production'

Swap

Close

Once **swapping** will complete, it will give us proper **success** message for confirmation.

Swap



Source

testprod-100(Staging)



Target

PRODUCTION

TestPROD-100



Swap with preview can only be used with sites that have slot settings enabled

☐ Perform swap with preview

Config Changes

This is a summary of the final set of configuration changes on the source and target slots after the swap has completed.

Source Changes

Target Changes

SETTING

TYPE

OLD VALUE

NEW VALUE

No Changes

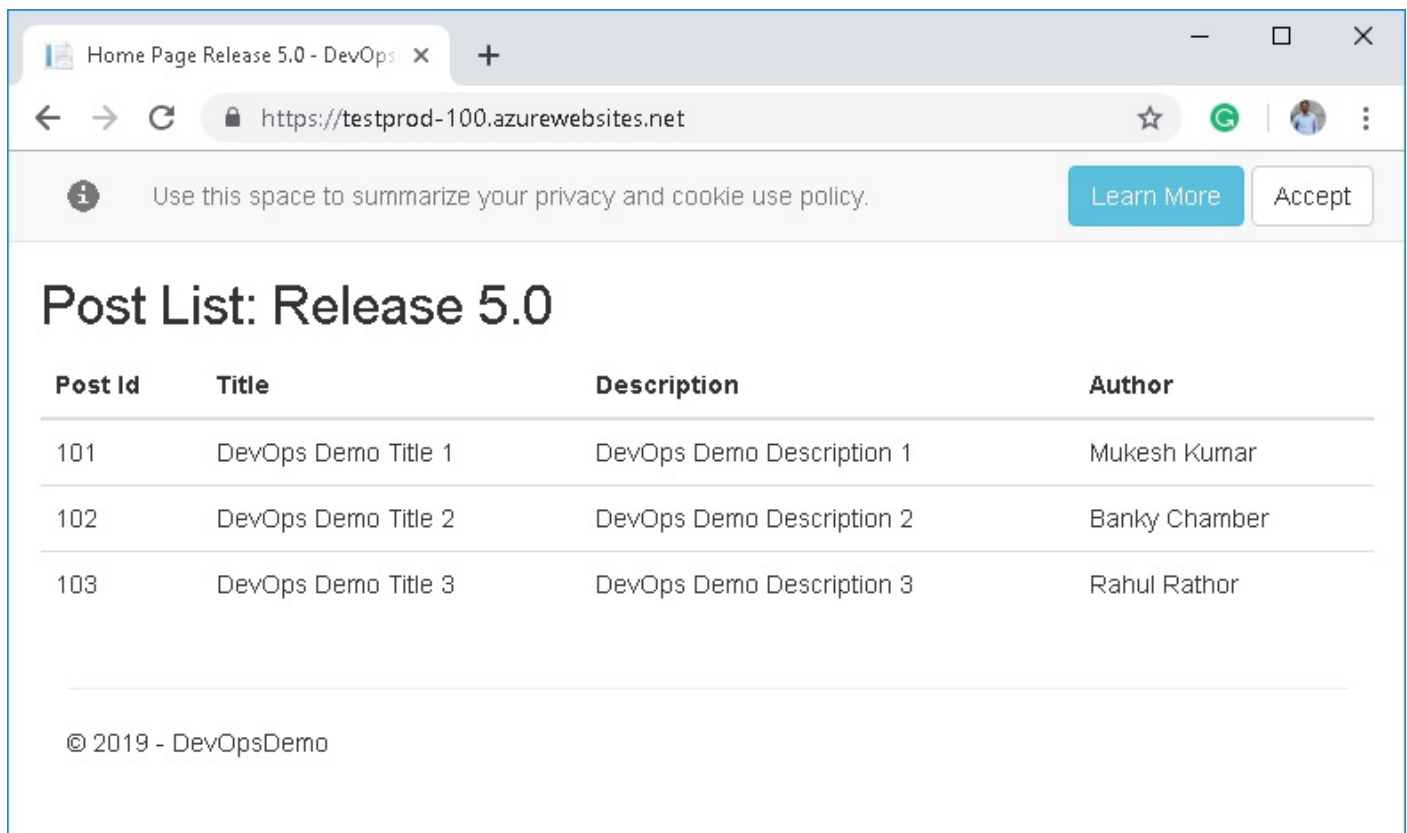


Successfully completed swap between slot 'Staging' and slot 'production'

Swap

Close

So, let open the **PROD (TestPROD-100)** environment and see the changes. Good, we have got the latest changes on the **PROD environment as Release 5.0**.



Home Page Release 5.0 - DevOps x +

← → ↻ https://testprod-100.azurewebsites.net ☆ G |  ⋮

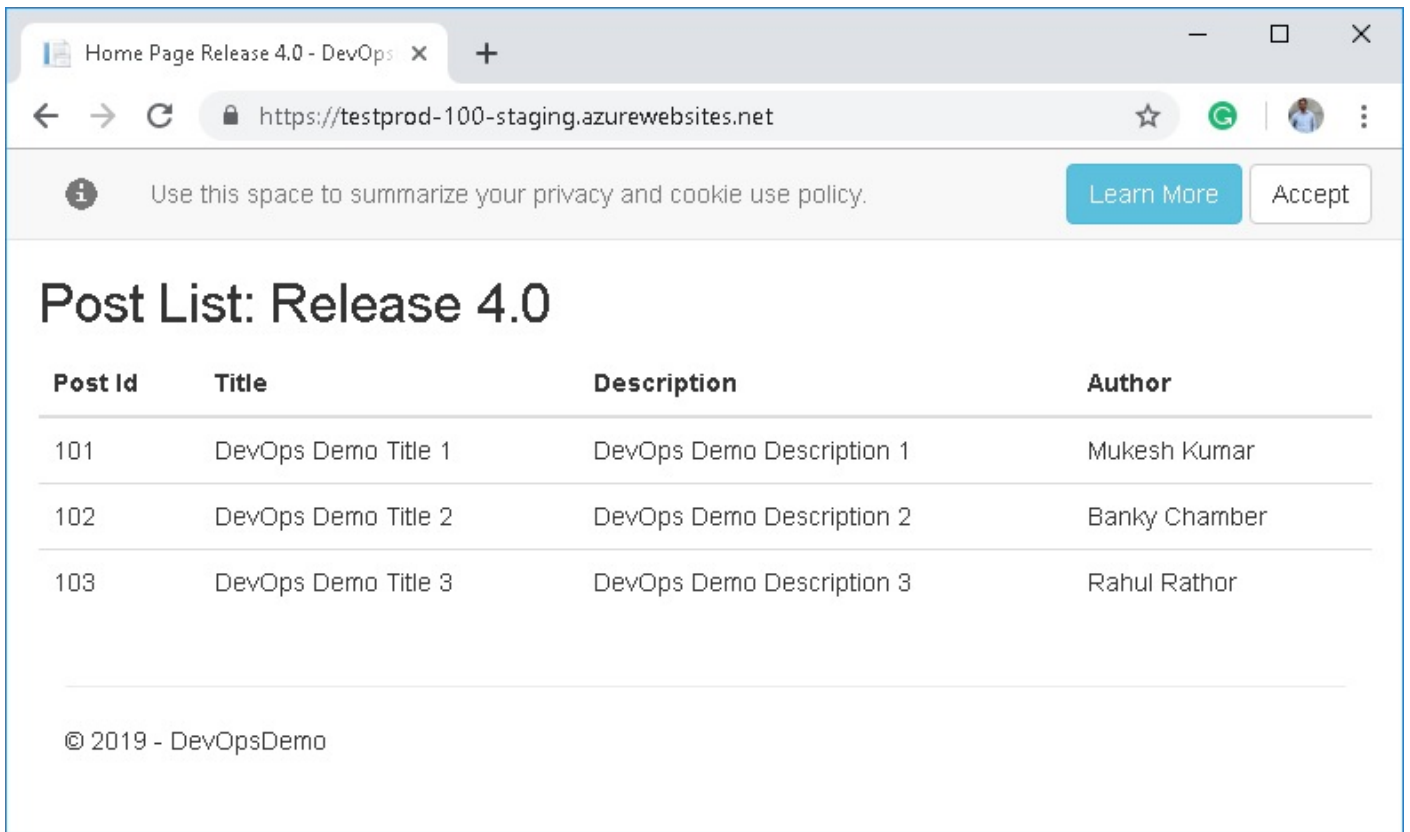
Use this space to summarize your privacy and cookie use policy. [Learn More](#) [Accept](#)

Post List: Release 5.0

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

© 2019 - DevOpsDemo

Let move and check the **Staging** environment and we will find that earlier **PROD** changes has deployed on **Staging** environment. It means, earlier which was **Staging** has become the **PROD** and which was **PROD** has become the **Staging**.



Home Page Release 4.0 - DevOps x +

https://testprod-100-staging.azurewebsites.net

Use this space to summarize your privacy and cookie use policy. [Learn More](#) [Accept](#)

Post List: Release 4.0

Post Id	Title	Description	Author
101	DevOps Demo Title 1	DevOps Demo Description 1	Mukesh Kumar
102	DevOps Demo Title 2	DevOps Demo Description 2	Banky Chamber
103	DevOps Demo Title 3	DevOps Demo Description 3	Rahul Rathor

© 2019 - DevOpsDemo

So, we have learned about Azure DevOps and implement the Continuous Integration and Continuous Delivery using live Asp.NET Core application with step by step. We hope, you have enjoyed a lot and learned a lot.

Thanks.

(By: Mukesh Kumar)